UNIVERSITÀ
DI SIENA
1240

University of Siena

Department of Information Engineering and Mathematics

PathNet: A* Search for Multilayer Perceptron Training

Kevin Gagliano

Jacopo Andreucci

Supervisor:

Prof. Edmondo trentin

Academic Year 2024-2025

Contents

Chapter 1

The A* Search Algorithm

1.1 Overview and Context

The **A* (A-star) search algorithm** is a cornerstone of Artificial Intelligence and computer science, highly effective for pathfinding and optimal graph traversal. Developed in 1968 by Hart, Nilsson, and Raphael as an extension of Dijkstra's algorithm, A* is classified as an *informed search* or *best-first search* method. Unlike uninformed searches, A* strategically explores the solution space by using estimated knowledge about the goal, enabling it to efficiently find the shortest path between a start and goal node. Its classical applications include video game pathfinding, network routing, and logistical planning.

1.2 The Evaluation Function $f(n)$

The central innovation of the A* algorithm lies in its heuristic evaluation function, $f(n)$, which estimates the total cost of the path from the starting point through the current node n to the goal. This function is defined as the sum of two components: the true cost from the start, $g(n)$, and the estimated cost to the goal, $h(n)$.

$$f(n) = g(n) + h(n)$$

1.2.1 Path Cost ($g(n)$)

The function $g(n)$ represents the actual cost of the path taken from the initial starting node to the current node n. This is a known, objective value and is a direct measure of the work done so far. In traditional pathfinding, $g(n)$ might be the distance traveled;

in optimization problems like the one discussed in this report, $g(n)$ represents the cumulative loss differential between two consecutive node while traversing the search graph.

1.2.2 Heuristic Estimate ($h(n)$)

The function $h(n)$ is the heuristic estimate of the cost from the current node n to the final goal node. A heuristic is an educated guess that guides the search towards the most promising regions of the search space. The effectiveness and properties of the A* algorithm are highly dependent on the quality of this heuristic.

The heuristic $h(n)$ must satisfy the admissibility property to guarantee optimality.

Admissibility

A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, meaning for all nodes n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true minimum cost from n to the goal.

If an admissible heuristic is used, A* is guaranteed to find the optimal (lowest-cost) path, provided a path exists.

Consistency

The condition of consistency is a stronger requirement than admissibility. A heuristic $h(n)$ is consistent if, for every node n and every successor node n' connected by an edge with cost $c(n, n')$, the estimated cost from n to the goal is less than or equal to the cost of moving to n' plus the estimated cost from n' to the goal.

$$\textbf{Consistency Condition: } h(n) \leq c(n, n') + h(n')$$

Consistency implies admissibility, provided $h(\text{goal}) = 0$. When a consistent heuristic is used, A* is guaranteed to find the optimal path without needing to re-open nodes (re-examine a node already in the Closed Set), because the f-cost along any optimal path is monotonically non-decreasing.

Optimality Guarantee

If the heuristic $h(n)$ is admissible (and edge costs are non-negative), A* is guaranteed to find the optimal (lowest-cost) path. If the heuristic is consistent, A* is also guaranteed to be optimal and more efficient, as it never needs to process a node more than once.

1.3 Algorithm Mechanics

A* operates by iteratively expanding nodes, maintaining two key data structures:

- **Open Set (or Frontier):** A priority queue containing nodes that have been generated but not yet fully explored. Nodes in the open set are prioritized based on their calculated $f(n)$ value.
- **Closed Set:** A set containing nodes that have already been fully expanded. This prevents the algorithm from revisiting and reprocessing the same node multiple times.

The core loop of the A* algorithm is as follows:

1. Initialize the Open Set with the starting node.
2. While the Open Set is not empty:
    - Select the node n from the Open Set with the lowest $f(n)$ value.
    - If n is the goal state, terminate the search.
    - Move n from the Open Set to the Closed Set.
    - Otherwise, generate all successors of n. For each successor s:
        - Calculate the cost of the path to s via n: $g_{\text{new}}(s) = g(n) + \text{cost}(n, s)$.
        - **Path Evaluation and Update:**
            * If s is in the Open Set, check if $g_{\text{new}}(s) < g(s)$. If true, update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and update its priority in the Open Set.
            * If s is in the Closed Set, check if $g_{\text{new}}(s) < g(s)$. If true, this means a cheaper path to s has been found. Update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and move s back from the Closed Set to the Open Set for re-expansion.
            * If s is not in either set, it is a newly discovered node. Set its parent to n, set $g(s) = g_{\text{new}}(s)$, calculate $f(s) = g(s) + h(s)$, and add it to the Open Set.

1.4 Properties of A*

The A* search algorithm is known for two key theoretical properties:

- **Completeness:** If a solution path exists from the start node to the goal node, A* is guaranteed to find it, provided the branching factor is finite and edge costs are non-negative.
- **Optimality:** A* is guaranteed to find the lowest-cost path if the heuristic function $h(n)$ is admissible. This makes it a preferred method for problems where minimizing the cost (or, in the context of this project, minimizing the training error) is paramount.

The adaptation of A* from a simple pathfinding tool to a machine learning optimization technique, as explored in this report, requires careful consideration of the state space, the definition of the path cost $g(n)$, and the design of an effective, possibly admissible heuristic $h(n)$.

1.5 Tree Search vs. Graph Search

The problem space solved by A* is fundamentally a graph, yet the search strategy can be executed either as a Tree Search or a Graph Search. The difference lies entirely in how the algorithm handles states (nodes) that can be reached via multiple distinct paths, a common occurrence in graphs with cycles or multiple connecting routes.

1.5.1 Tree Search A*

The Tree Search variant treats every path to a state as a unique node in the search tree.

- **Mechanism:** Does not utilize a Closed Set. A node is generated and expanded without checking if an identical state has been reached previously via a different path.
- **Efficiency:** Avoiding the memory overhead of the Closed Set, leads to the redundant expansion of identical states, severely impacting time efficiency, especially in graphs with many cycles or alternative routes.
- **Optimality:** Only guaranteed if and only if the heuristic function $h(n)$ is admissible.

1.5.2 Graph Search A*

The Graph Search variant is the standard implementation of A* because it explicitly tracks visited states to avoid redundant work.

- **Mechanism:** Utilizes both the Open Set and the Closed Set to ensure that, ideally, no state is expanded more than once.
- **Efficiency:** Significantly more time-efficient than Tree Search, as it avoids re-exploring entire subgraphs.
- **Optimality:**
  - If the heuristic $h(n)$ is **consistent**, a node is guaranteed to be reached via the optimal path when it is first placed in the Closed Set. Therefore, no node ever needs to be re-opened.
  - If $h(n)$ is only **admissible**, the optimal path to a state might be discovered after the state has already been placed in the Closed Set. This motivates the re-opening mechanism.

1.5.3 Implementation Choice

In the context of this project, which adapts A* for machine learning optimization, given the experimental nature of the heuristic $h(n)$ and the $g(n)$ cost definition based on training loss, neither strict **admissibility** nor **consistency** can be guaranteed. Therefore, the implementation utilizes the Graph Search A* algorithm, characterized by the explicit use of both the Open Set and the Closed Set. Crucially, the algorithm includes the necessary logic to re-open nodes from the Closed Set if a new, cheaper path to that node is discovered. This ensures the best possible path is found despite the non-ideal properties of the custom heuristic.

Chapter 2

A* Cost Function Adaptation for Training

The successful application of A* to optimization relies entirely on correctly defining the cost components $g(n)$ and $h(n)$ in the context of neural network training.

2.1 Heuristic Function $h(n)$: Current State Loss

In this formulation, the heuristic $h(n)$ directly represents the network's performance at node n and it is defined as the current loss value evaluated on the training data:

$$h(n) = L(n)$$

This heuristic prioritizes configurations that minimize the objective function, guiding the search toward lower-cost states in the weight landscape.

2.2 Path Cost $g(n)$: Accumulated Loss Differential

The cost-to-come, $g(n)$, represents the cumulative cost of the path taken from the initial configuration to the current node n. The cost of each transition (edge) is defined by the improvement or degradation in loss between the parent and the child node n.

The step cost Δg is defined as:

$$\Delta g = L(\text{n}) - L(\text{parent})$$

The total path cost is then updated iteratively:

$$g(n) = g(\text{parent}) + \Delta g$$

By defining Δg in terms of the loss differential, the path cost effectively tracks the total variation in loss encountered during the traversal of the search space.

2.3 Total Evaluation Function $f(n)$

The combined evaluation function $f(n)$ determines the priority of nodes in the search queue:

$$f(n) = g(n) + h(n) = (g(\text{parent}) + (L(n) - L(\text{parent}))) + L(n)$$

By selecting the node with the minimum $f(n)$, the algorithm seeks a path that minimizes both the absolute error of the current state $L(n)$ and the cumulative change in loss required to reach that state $g(n)$.

2.4 Admissibility Analysis

In standard A* search, a heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, i.e., $h(n) \leq h^*(n)$. In this revised framework, the "goal" is implicitly the global minimum of the loss landscape ($L = 0$), and the cost is measured in units of loss differential.

1. **Definition:** Both the heuristic $h(n) = L(n)$ and the true cost $h^*(n)$ are expressed in the same units (loss). The true cost $h^*(n)$ represents the cumulative sum of loss differentials required to reach the global minimum starting from node n.

2. **The Admissibility Challenge:** For $h(n)$ to be admissible, the condition $h(n) \leq h^*(n)$ must hold. Under the current definitions, $h(n) = L(n)$, which is a non-negative value (the loss), while the true cost $h^*(n)$ is the sum of loss differentials: $h^*(n) = \sum_n^{goal} \Delta g$.

   In any scenario where the path toward the goal is monotonically improving, every step cost $\Delta g = L(n') - L(n)$ is strictly negative. Consequently, the true cost-to-go $h^*(n)$ becomes a summation of negative terms, yielding a negative total value. Because $L(n) \geq 0$, the inequality $L(n) \leq h^*(n)$ is fundamentally violated in every improving step of the training process.

This implies that the heuristic is never admissible during a standard training descent. It provides a positive value (L) for a trajectory that accumulation a negative total cost ($\sum \Delta g$). Admissibility could only be restored if the search moved "uphill" into regions of significantly higher loss; however, since the primary goal is loss minimization, this violation is a permanent structural feature of the search design rather than an error.

2.5 Consistency Analysis

A heuristic is consistent if $h(n) \leq c(n, n') + h(n')$, where $c(n, n')$ is the cost of the transition. With our new definitions:

- $h(n) = L(n)$
- $c(n, n') = \Delta g = L(n') - L(n)$
- $h(n') = L(n')$

Substituting these into the consistency inequality:

$$L(n) \leq (L(n') - L(n)) + L(n') \implies L(n) \leq 2L(n') - L(n) \implies 2L(n) \leq 2L(n')$$

Simplifying this, the **Consistency Condition** becomes:

$$L(n) \leq L(n')$$

Analysis of Scenarios w.r.t. Consistency

Unlike the previous version, consistency is now tied directly to the direction of the loss change ΔL:

1. **Scenario 1: Loss Decrease ($\Delta L < 0$):** When an update improves the model, $L(n') < L(n)$. In this case, the consistency condition $L(n) \leq L(n')$ is **violated**. This is a mathematically interesting result: every "good" step in training (one that reduces loss) technically violates the A* consistency property under these definitions.

2. **Scenario 2: Loss Increase ($\Delta L > 0$):** If the update moves into a worse region of the landscape, $L(n') > L(n)$. Here, the consistency condition is **satisfied**. This implies that the search is only "consistent" when it is moving away from the solution.

3. **Scenario 3: Stationary Loss ($\Delta L = 0$):** In plateaus where $L(n') = L(n)$, the heuristic is perfectly consistent.

Chapter 3

The Multilayer Perceptron and Traditional Training Methods

3.1 Multilayer Perceptron (MLP) Architecture

The Multilayer Perceptron (MLP) is a fundamental class of feed-forward artificial neural networks. It is distinguished by having one or more *hidden layers* between the input layer and the output layer, which allows the network to learn complex, non-linear relationships between inputs and outputs.

3.1.1 Neuron Structure and Function

The basic unit of the MLP is the artificial neuron, which receives a set of inputs, each associated with a synaptic weight. The neuron performs two main operations:

1. **Weighted Aggregation:** It calculates the weighted sum of the inputs, to which a *bias* (β) is added:

$$z = \left(\sum_{i=1}^{m} w_i x_i \right) + \beta$$

   where x_i are the inputs, w_i are the weights, and m is the number of inputs.

2. **Activation Function:** It applies a non-linear activation function (σ) to the aggregated result to produce the neuron's output (a):

$$a = \sigma(z)$$

3.1.2 The Loss Function

Training an MLP is an optimization problem that aims to minimize the error between the network's predicted output and the desired target output. This error is quantified by the loss function E.

3.2 The Standard Training Method: Backpropagation

Traditionally, the MLP is trained using the **Backpropagation** algorithm, which utilizes the gradient of the loss function to update the network's weights.

3.2.1 Theoretical Basis: Gradient Descent

The most basic iterative algorithm for weight adjustment is *Gradient Descent*. It moves the weights in the direction of the steepest descent of the cost function:

$$w_{t+1} = w_t - \eta \cdot g_t$$

where η is the *learning rate* and $g_t = \nabla E(w_t)$. While this "Stochastic" Gradient Descent (SGD) forms the historical basis of neural network training, it often suffers from slow convergence and sensitivity to the choice of η.

3.3 Selected Gradient-Based Benchmark: The Adam Optimizer

In this study, rather than utilizing standard SGD, we employ the **Adam (Adaptive Moment Estimation)** optimizer as the representative gradient-based benchmark. Adam is chosen due to its status as the current state-of-the-art for training deep architectures, providing a more robust and efficient baseline for comparison against the proposed A* method.

3.3.1 Mathematical Formulation

Adam optimizes training by maintaining moving averages of both the gradients m_t and the squared gradients v_t:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

To ensure the estimates are unbiased during the initial steps, bias-correction is applied:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The resulting update rule is:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

This adaptive mechanism allows for a more reliable descent in complex loss landscapes compared to fixed-rate methods.

3.4 Motivation for the A* Approach

Despite the sophistication of adaptive optimizers like Adam, gradient-based methods possess intrinsic limitations that motivate the exploration of informed search algorithms like A*:

- **Local Minima and Saddle Points:** Even with the momentum and scaling features of Adam, gradient-based methods are susceptible to stalling in saddle points or becoming trapped in local minima where the local gradient is zero.
- **Local Nature:** Adam remains a *local* optimizer. It relies entirely on the derivative at the current point w_t and its immediate history. It lacks the global "vision" or heuristic foresight of the cost surface that an informed search algorithm like A* can provide by exploring multiple potential paths in the weight space.

This study proposes to frame the MLP training not as a continuous descent problem, but as a pathfinding problem in a quantized weight space. By utilizing a heuristic function $h(n)$ to estimate the distance to the global minimum, A* can potentially navigate the weight space with greater global awareness than the backpropagation-based methods described in this chapter.

Chapter 4

The A* Search Framework for MLP Training

The primary objective of this project is to reformulate the optimization challenge of training a Multilayer Perceptron (MLP) as a **shortest-path problem** within a discrete, high-dimensional weight space. This chapter introduces the framework designed to achieve this, detailing how the continuous weight space is discretized, how the nodes and edges of the search graph are defined, how the challenge of main memory saturation is managed and how the A* algorithm's is adapted to guide the network towards optimal weights.

4.1 Discretization of the Weight Space

The A* algorithm fundamentally operates on a discrete graph. Therefore, the continuous domain of real-valued weight parameters must be converted into a finite, discrete set of possible states. This is handled by the `QuantizedMLP` class.

4.1.1 Quantized State Definition

To define the search nodes, the floating-point parameters (weights and biases) of the MLP are subjected to uniform quantization. Given a predefined **quantization factor** (Q), every weight w_i is rounded to the nearest multiple of $1/Q$.

$$w_{i,\text{quantized}} = \text{round}(w_i \cdot Q)/Q$$

The number of potential discrete weight configurations defines the size of the

search space. To manage the complexity and avoid instability, the parameters are constrained to a defined numerical range [min_range, max_range].

The total number of possible states is:

$$[\mathrm{Q}(\mathrm{min\_range} + \mathrm{max\_range}) + 1]^{N_{\mathrm{params}}}$$

4.1.2 State Representation and Hashing

In the A* implementation, the efficacy of the algorithm relies on quickly checking if a particular weight configuration has been visited before, or if a better path to that configuration has been found. This requires a unique, hashable identifier for each state.

The `QuantizedMLP` utilizes the `get_state_hash()` method, which concatenates all quantized parameters into a single, ordered tuple of integers (by multiplying the quantized weights by the quantization factor Q). This tuple serves as the key in the Closed Set (represented by the `g_costs` dictionary in the implementation) for tracking the minimum cost found so far to reach that state.

4.2 The Search Node

In the context of A* based optimization, a search node n is defined as a discrete point within a high-dimensional graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each vertex $v \in \mathcal{V}$ represents a unique, quantized configuration of the neural network's weights and biases. The edges $\mathcal{E}$ are induced by the possible transitions permitted, which move the network from one quantized state to another.

A single search node n is encapsulated by the `SearchNode` class, which maintains the following state and metadata:

1. **Quantized State:** The current `QuantizedMLP` configuration, representing a unique coordinate in the quantized parameter space.
2. **Cost-to-come ($g(n)$):** The cumulative cost incurred from the initial configuration to the current node, calculated as the sum of loss differentials along the path.
3. **Heuristic Estimate ($h(n)$):** The estimate of the remaining cost to reach the target loss (the "goal"), here defined by the current loss value $L(n)$.
4. **Total Estimated Cost ($f(n)$):** The evaluation function $f(n) = g(n) + h(n)$, which the A* algorithm uses to prioritize node exploration in the Open List.

5. **Parent Pointer:** A reference to the preceding node in the search graph, allowing for the reconstruction of the optimal training trajectory upon convergence.

4.3 Defining Transitions: Neighborhood Generation Strategies

The edges of the search graph are defined by the allowed transitions between weight configurations, generated by the `get_neighbors` function. We implement a versatile framework supporting three distinct strategies for neighbor generation, each offering a different trade-off between structured and stochastic exploration.

In all methods, a neighbor state s is generated from the current state n by selecting a subset of parameters $\theta \in \text{Layer}_\ell$ and applying a discrete quantization step:

$$\theta_s = \theta_n \pm \frac{1}{Q} \quad \forall \theta \in \text{Subset}$$

4.3.1 Strategy 1: Single-Kernel Method

This approach utilizes a unique set of kernel ($H \times W$ for weights, $L \times 1$ for biases) and stride (x_s, y_s) pairs applied consistently across all layers. The algorithm dynamically handles architectural constraints through **dynamic shrinking** (adaptive kernel scaling): if the target tensor dimensions are smaller than the defined kernel, the window is automatically clipped to fit the available parameter space. This ensures a consistent transition logic while maintaining robustness across heterogeneous layer sizes.

4.3.2 Strategy 2: Layer-Wise Kernel Method

To allow for higher degrees of freedom, this method supports per-layer customization, while retaining the **adaptive kernel scaling** mechanism to ensure compatibility with varying tensor dimensions. For each layer ℓ, a specific weight kernel, bias kernel, and stride pair (x_s, y_s) are defined. This enables the search to adapt its "spatial reach" to the specific role of the layer. For instance, wider kernels may be utilized for initial layers to capture broad features, while narrower kernels can be used for final layers to perform fine-grained adjustments.

$$W^{(l)} = \begin{bmatrix} w_{1,1}^{(l)} & w_{1,2}^{(l)} & w_{1,3}^{(l)} & w_{1,4}^{(l)} & w_{1,5}^{(l)} \\ w_{2,1}^{(l)} & w_{2,2}^{(l)} & w_{2,3}^{(l)} & w_{2,4}^{(l)} & w_{2,5}^{(l)} \\ w_{3,1}^{(l)} & w_{3,2}^{(l)} & w_{3,3}^{(l)} & w_{3,4}^{(l)} & w_{3,5}^{(l)} \\ w_{4,1}^{(l)} & w_{4,2}^{(l)} & w_{4,3}^{(l)} & w_{4,4}^{(l)} & w_{4,5}^{(l)} \end{bmatrix}$$

Figure 4.1: Visualization of a 2×2 kernel at position (1,1) in a quantized weight matrix.

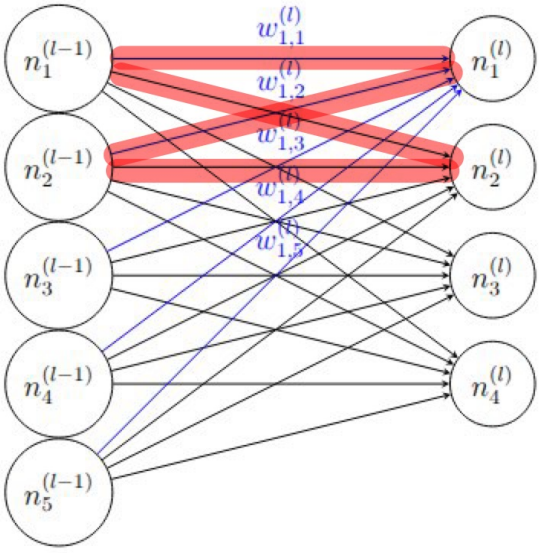

Figure 4.2: Connections within the MLP architecture specifically affected by a localized kernel update.

For both kernel-based strategies, the number of possible transitions (slides) per layer is determined by the valid window positions in an $M \times N$ matrix:

$$\text{Slides}_\ell = \left(\lfloor \frac{M - H}{y_s} \rfloor + 1 \right) \times \left(\lfloor \frac{N - W}{x_s} \rfloor + 1 \right)$$

4.3.3 Dynamic Kernel Reshaping

To address the issue of convergence stagnation during the optimization process, we introduced a heuristic mechanism known as *Dynamic Kernel Reshaping*. This technique is exclusively applied to the kernel-based neighbor generation strategies (*Single-Kernel* and *Layer-Wise Kernels*) and serves to adaptively refine the search granularity when the training loss hits a plateau.

Plateau Detection Mechanism

The algorithm monitors the training trajectory to detect "stalling" conditions where the current kernel configuration fails to produce significant improvements. This detection is governed by two hyperparameters:

- **Loss Improvement Threshold (δ_{loss}):** The minimum required decrement in training loss between two consecutive iterations ($L_t - L_{t+1}$) to be considered a valid improvement.
- **Dynamic Kernel Reshaping Patience ($P_{reshape}$):** A counter that tracks the number of consecutive iterations where the loss improvement has failed to meet δ_{loss}.

Dynamic Shrinking Process

When the accumulated patience counter reaches the defined limit ($P_{reshape}$), the system infers that the search has reached a local optimum relative to the current coarse-grained kernel sizes. To facilitate finer exploration, a **shrinking event** is triggered.

During this event, the dimensions of the kernels (K) and strides (S) are reduced by fixed decrement values:

$$K_{new} = \max(K_{min}, K_{old} - \Delta_k)$$

$$S_{new} = \max(S_{min}, S_{old} - \Delta_s)$$

where Δ_k and Δ_s represent the *kernel decrement* and *stride decrement* parameters (defaulting to 1), and K_{min}, S_{min} represent the minimum allowed shapes to ensure structural validity.

This process allows the A^* search to transition from a broad, global exploration phase to a high-resolution local refinement phase. The reshaping mechanism remains active throughout the training lifecycle; if the patience is exhausted again at the new resolution, the kernels are shrunk further until they reach their minimum bounds or the training completes the maximum allocated iterations.

4.3.4 Adaptive Kernel Scaling

In scenarios where the target network architecture contains layers with parameter tensors smaller than the predefined kernel dimensions, an **adaptive kernel scaling strategy** is employed. This mechanism allows scaling the kernel size (K) and strides (S) to fit the specific geometry of the parent tensor.

2D Weight Matrices

For a weight matrix of shape (M, N), let the initial weight kernel shape be defined as $\mathbf{k}_w = [k_y, k_x]$. If the matrix dimensions are smaller than the kernel size in any axis ($M < k_y$ or $N < k_x$), the updated kernel $\mathbf{k}'_w = [k'_y, k'_x]$ and strides s'_x, s'_y are calculated as follows:

First, we clip the kernel to the maximum available dimensions:

$$k'_{y,\text{clip}} = \min(M, k_y), \quad k'_{x,\text{clip}} = \min(N, k_x)$$

To maintain a search granularity that is finer than the layer itself, we identify the smaller of these dimensions and reduce it by half:

$$\text{index}_{\min} = \text{argmin}(k'_{y,\text{clip}}, k'_{x,\text{clip}})$$

$$k'_{\text{index}_{\min}} = \max\left(1, \lfloor k'_{\text{index}_{\min},\text{clip}}/2 \rfloor\right)$$

Finally, the traversal strides are set equal to the new kernel dimensions to ensure full, non-overlapping coverage of the search space:

$$s'_x = k'_x, \quad s'_y = k'_y$$

1D Bias Vectors

For bias vectors of length L, the 1D kernel k_b is adjusted if $L < k_b$. The new kernel k'_b and stride s'_y are defined by:

$$k'_b = \max\left(1, \lfloor \min(L, k_b)/2 \rfloor\right)$$

$$s'_y = k'_b$$

This adaptation ensures that even for extremely small layers (such as the final output layer of a regressor), the branching factor remains bounded and the search can still identify discrete updates without requiring manual hyperparameter reconfiguration per layer.

4.3.5 Strategy 3: Random Sampling Method

This strategy employs a stochastic approach to define the target set of parameters, governed by two key hyperparameters:

- **Perturbation Ratio (r_p):** Determines the percentage of total parameters within a layer modified in a single transition. For a layer with N_{total} parameters, each neighbor modifies $\lceil r_p \cdot N_{\text{total}} \rceil$ weights selected via random sampling.
- **Search Coverage Ratio (r_c):** Determines the total number of unique neighbors generated for each layer, calculated as $\lceil r_c \cdot N_{\text{total}} \rceil$.

4.3.6 Dynamic Quantization

While Dynamic Kernel Reshaping addresses the *spatial* resolution of the search (i.e., how many parameters are changed), **Dynamic Quantization** addresses the *numerical* resolution of the updates (i.e., how much the parameters change). Unlike kernel reshaping, this technique is universally applicable to all three neighbor generation strategies.

Stagnation and Trigger Logic

Similar to the reshaping mechanism, this strategy employs a heuristic based on training stagnation. The algorithm monitors the loss trajectory using the global *Loss Improvement Threshold*. A dedicated counter, the **Dynamic Quantization Patience**, tracks the number of consecutive iterations where the training loss fails to improve beyond this threshold.

Quantization Refinement

When the patience counter is exhausted, it indicates that the current discretization of the weight space is too coarse to identify a valid descent direction effectively, the "grid" of allowable weight values is straddling the local minimum without landing in it.

To resolve this, the quantization factor Q is **increased**. Since the discrete step size for any parameter update is defined as $\Delta\theta = \pm\frac{1}{Q}$, increasing Q results in a smaller, more precise step size:

$$Q_{new} = Q_{old} * \Delta_Q \quad \implies \quad \frac{1}{Q_{new}} < \frac{1}{Q_{old}}$$

where Δ_Q represents the *Quantization Factor Multiplier* (defaulted to 10).

Operational Impact

By dynamically increasing the quantization factor, the search space transitions from a coarse grid to a fine-grained mesh. This allows the algorithm to perform high-precision "micro-updates," enabling the network to settle into deeper minima that were previously inaccessible due to the rigidity of the initial discrete steps. This process can repeat iteratively, progressively refining the numerical precision until the training concludes.

4.3.7 Mathematical Comparison of Branching Factors

The branching factor B represents the number of successor nodes generated from a single parent node. A comparison of the branching factors between the Layer-wise Kernel and Random Sampling methods highlights the difference in search space density:

1. **Kernel-Wise Branching (B_K):** The branching factor is strictly determined by the geometry of the layers and the chosen kernels and strides:

$$B_K = 2 \cdot \sum_{\ell \in \text{Layers}} (\text{Slides}_{\text{weights},\ell} + \text{Slides}_{\text{biases},\ell})$$

2. **Random Sampling Branching (B_R):** The branching factor is proportional to the total parameter count N of the network:

$$B_R = 2 \cdot \sum_{\ell \in \text{Layers}} \lceil r_c \cdot N_\ell \rceil$$

4.3.8 Comparative Analysis: Structured vs. Stochastic Exploration

The three proposed neighborhood generation strategies represent different philosophies in exploring the high-dimensional quantized parameter space. Their effectiveness is evaluated based on four primary criteria: spatial locality, search space pruning, exploration capacity, and hyperparameter complexity.

- **Single-Kernel Method**
  - **Pros:** Provides extreme *search space pruning* by enforcing a uniform transition rule across the entire network. It maximizes *spatial locality* by ensuring that every weight update is part of a cohesive geometric block.
  - **Cons:** Limited *exploration* flexibility; if the network has layers of vastly different sizes (e.g., a massive hidden layer followed by a tiny output layer), the fixed kernel size may be too coarse for some layers and too fine for others.
- **Layer-Wise Kernel Method**
  - **Pros:** Provides an enhanced degree of architectural flexibility while maintaining the inherent advantages of *spatial locality* through localized weight updates. By tailoring kernel sizes and strides to the specific dimensions of each layer, it optimizes the *search space pruning* for the architecture's specific geometry. This prevents "blind spots" in smaller layers while maintaining high-speed traversal in larger ones.
  - **Cons:** Increases the hyperparameter search space. Determining the optimal (WK, BK, S) triplet for every layer adds significant complexity to the pre-training experimental setup.
- **Random Sampling Method**
  - **Pros:** Superior *exploration* capacity. By breaking the constraint of geometric locality, it can discover non-obvious weight correlations and escape local minima where kernel-based movements might stall. It allows for a dense or sparse search controlled entirely by the *perturbation ratio* and *search coverage ratio* parameters.
  - **Cons:** No *spatial locality* is assumed and because updates are stochastic, the search can become "noisy," leading to lower efficiency as the algorithm

may explore many disjoint directions that do not contribute to a cohesive functional change in the network's output.

Table 4.1: Comparison of Neighbor Generation Strategies

Feature	**Single Kernel**	**Layer-Wise**	**Random**
Locality Preservation	High	High	None
Search Space Pruning	High	Moderate	Variable
Exploration Breadth	Low	Moderate	High
Hyperparameter Complexity	Low	High	Low

4.4 Memory Optimization: Beam Search Integration

A significant challenge in applying the A* search algorithm to neural network optimization is the memory-intensive nature of maintaining the *Open* and *Closed* sets. In large-scale architectures, the branching factor B scales with the total number of parameters, leading to an exponential growth of the search tree. Empirical observations showed that without optimization, the search often encountered premature termination due to system RAM saturation. This phenomenon, in some MLP architectures tested, prevented the algorithm from surpassing 100–200 iterations, leaving the network in a sub-optimal state despite remaining potential for loss reduction.

To mitigate this bottleneck, a **Beam Search memory optimization** was designed and integrated into the search framework. This technique effectively caps the memory footprint by limiting the maximum size of the search frontier.

4.4.1 Periodic Set Pruning and Memory Regulation

The optimization is governed by the *Beam Width* (BW) parameter, which defines the target maximum capacity for the *Open* and *Closed* sets. In this implementation, memory regulation is applied periodically rather than at every discrete time step; in our experimental benchmark, this check occurs every 50 iterations.

Consequently, the dimensions of the search sets are not strictly fixed at every step. Between two pruning cycles, the sets are allowed to grow dynamically to accommodate the branching factor of the network. Once the iteration threshold is reached, the algorithm evaluates the current size of the sets. If they exceed the

defined limit, a pruning operation is triggered to restore the memory footprint to the specified BW:

1. **Ranking:** All nodes currently in the set are sorted according to their evaluation function $f(n) = g(n) + h(n)$.
2. **Truncation:** Only the top BW nodes, those representing the most promising search trajectories, are retained.
3. **Memory Release:** All remaining nodes are purged from the main memory, effectively resetting the search frontier to a manageable scale.

This "pulsed" pruning approach strikes a balance between allowing the search to explore diverse local branches in the short term and maintaining long-term stability in the face of hardware memory constraints.

4.4.2 Impact on Training Longevity

The implementation of the Beam Search optimization has demonstrated a profound impact on the scalability of the training process, particularly in deeper architectures. By enforcing a fixed memory bound, the algorithm can navigate the parameter space for significantly longer durations.

- **Scalability:** Networks that previously exhausted memory within 200 iterations can now reliably reach fixed thresholds of $2,000$ iterations or more.
- **Loss Minimization:** The ability to sustain longer search trajectories allows the A* algorithm to escape local plateaus and locate deeper minima in the loss landscape that were previously unreachable due to hardware constraints.
- **Tuning:** The BW parameter acts as a trade-off between search completeness and memory safety. A higher BW allows for a more exhaustive search closer to the original A*, while a lower BW ensures stability in environments with limited hardware resources.

This optimization transforms the A* approach from a memory-bounded search into a scalable iterative optimizer, making it viable for architectures that would otherwise be computationally prohibitive.

4.5 The A* Training Loop Implementation

The core training logic is contained within the `Trainer` class, which manages the A* search:

1. **Initialization:** The process begins by creating an `initial_node` from the starting MLP weights. This node is placed in the priority queue (`open_set`).
2. **Iteration:** In each iteration, the node with the lowest $f(n)$ is retrieved from the `open_set`.
3. **Optimality Check:** Before expanding the node, the current path cost $g(n)$ is checked against the minimum cost previously recorded for that state in `g_costs` to ensure that A* always processes the optimal path to a given state first.
4. **Expansion:** Neighbors are generated by the transition rule ($\pm 1/Q$ change to a weight subset).
5. **Cost Update:** For each neighbor, the new g and h values are calculated. If the new path to the neighbor results in a lower cost-to-come ($g_{\text{new}} < g_{\text{old}}$), the neighbor is either updated in the search space or added to the `open_set`.
6. **Termination:** The search terminates when either the best node retrieved from the `open_set` satisfies the goal condition ($h(n) = 0$) or the maximum number of iterations is reached.

Chapter 5

Benchmark Datasets

In this chapter, we detail the datasets used to evaluate and compare the performance of the A* training framework against Adam optimizer. To provide a comprehensive assessment, the benchmarks cover both regression and classification tasks, ranging from synthetic functional mappings to high-dimensional real-world data.

For all experiments, a uniform data partitioning strategy was applied, splitting each dataset into training, validation, and test sets according to an 80-10-10 percentage ratio.

5.1 Regression Datasets

5.1.1 Noisy Sine Function

The Noisy Sine dataset is a synthetic benchmark designed to assess the algorithm's ability to capture non-linear, periodic patterns in a low-dimensional setting. The dataset consists of 1,000 samples generated from a one-dimensional sine wave.

- **Data Generation:** The samples are evenly spaced in the range $x \in [0, 4\pi]$.
- **Noise Profile:** To simulate real-world variance, additive Gaussian noise was introduced: $y = \sin(x) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 0.1)$.

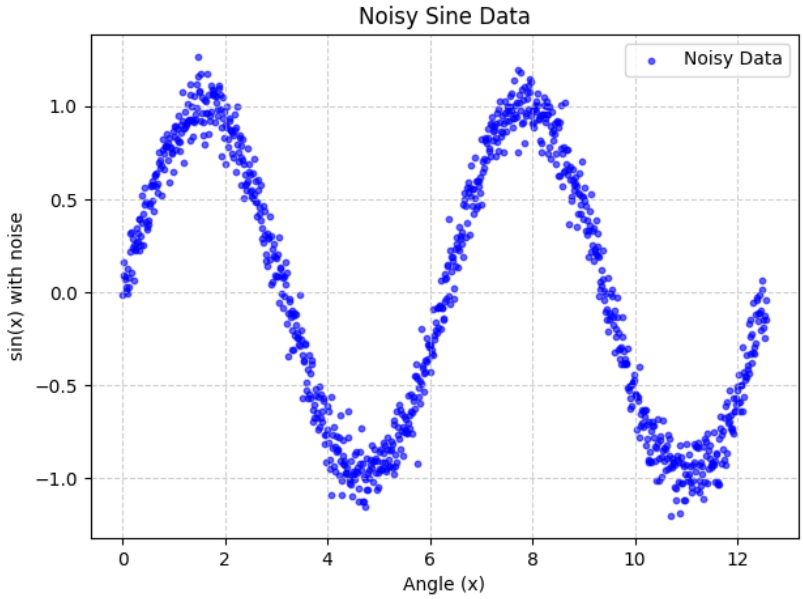

Figure 5.1: Plot of the noisy sine dataset

5.1.2 California Housing

The California Housing dataset provides a high-dimensional, real-world regression challenge. Derived from the 1990 U.S. Census, it contains 20,640 samples, each representing a specific California district.

- **Input Features:** 8 economic and geographic features, including Median Income (MedInc), House Age, Average Rooms, and spatial coordinates (Latitude/Longitude).
- **Target:** The median house value for the district, expressed in units of $100,000.

5.2 Classification Datasets

5.2.1 Iris Flower

The Iris dataset is a classic multiclass classification benchmark. It is a relatively small but well-structured dataset consisting of 150 samples distributed equally across three

species of Iris flowers.

- **Input Features:** 4 physical measurements (sepal length, sepal width, petal length, and petal width).
- **Class Distribution:** Three species: *Setosa*, *Versicolor*, and *Virginica*.

5.2.2 Wine Quality (Red)

The Wine Quality dataset offers a more complex classification task due to its higher dimensionality and larger sample size (1,599 samples) compared to the Iris dataset. It captures the physicochemical properties of Portuguese "Vinho Verde" red wine.

- **Input Features:** 11 physicochemical variables, including fixed acidity, volatile acidity, residual sugar, chlorides, pH, sulphates, and alcohol content.
- **Class Distribution:** The original dataset contains quality scores ranging from 3 to 8. These are shifted to a 0–5 range for modeling.

5.3 Summary of Dataset Characteristics

The table below summarizes the dimensions and objectives of the employed benchmarks.

Table 5.1: Summary of Benchmark Datasets

Dataset	**Type**	**Samples**	**Features**	**Output Classes**
Noisy Sine	Regression	1,000	1	N/A
California Housing	Regression	20,640	8	N/A
Iris Flower	Classification	150	4	3
Wine Quality	Classification	1,599	11	6

Chapter 6

Evaluation Metrics

To rigorously assess the efficacy of the A* search algorithm as a neural network optimizer, we employ two distinct suites of evaluation metrics tailored to regression and classification tasks. For every experiment, performance is measured on the validation and test sets to ensure generalization capability.

Due to the stochastic nature of the initialization and the search trajectories, each experiment is repeated across multiple independent runs. For all metrics defined below, we report the **Mean**, **Standard Deviation**, **Minimum**, and **Maximum** values to capture the stability and potential peak performance of the training process.

6.1 Regression Performance Metrics

In regression tasks, specifically the Noisy Sine and California Housing benchmarks, the objective is to minimize the deviation between the predicted continuous values $\hat{y}$ and the ground-truth values y. We employ four primary metrics:

- **Mean Squared Error (MSE):** Measures the average of the squares of the errors. It is highly sensitive to outliers due to the squaring of the residuals.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- **Root Mean Squared Error (RMSE):** The square root of the MSE, providing an error metric in the same units as the target variable.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **Mean Absolute Error (MAE):** Represents the average of the absolute differences between predictions and actual observations, providing a linear measure of error.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- **Coefficient of Determination (R^2):** This metric quantifies the proportion of the variance in the dependent variable (y) that is predictable from the independent variables (x) via the neural network. It acts as a comparison between the model's error and the variance of a simple mean-based baseline.

  The formula is defined as:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

  Where SS_{res} is the sum of squared residuals (unexplained variance) and SS_{tot} is the total sum of squares (total variance in the data). An R^2 of 1.0 indicates that the model perfectly accounts for all variability in the target data. An R^2 of 0 implies the model is no more informative than the mean of the observed data, while an $R^2 < 0$ indicates that the model provides a worse fit than a horizontal line representing the mean, typically due to a severe overfitting or underfitting.

6.2 Classification Performance Metrics

For classification tasks (Iris and Wine Quality), performance is evaluated based on the model's ability to correctly assign categorical labels. The following metrics are derived from the Confusion Matrix (True Positives TP, True Negatives TN, False Positives FP, and False Negatives FN):

- **Accuracy:** The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** The ability of the classifier not to label as positive a sample that is negative.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** The ability of the classifier to find all the positive samples.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced metric particularly useful in the presence of class imbalance.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

6.3 Statistical Aggregation

For both regression and classification, the reporting of **Standard Deviation** is critical to our analysis. It serves as a proxy for the *reliability* of the A* search under different neighbors generation strategies and optimization techniques . A low standard deviation across runs indicates that the search logic is consistently finding similar basins of attraction in the weight space, regardless of the random initialization.

Chapter 7

Results Visualization and Statistical Evaluation

To ensure a rigorous comparative analysis between the proposed A^* search framework and the Adam optimizer, we utilize a dual-plotting methodology. This approach allows us to observe not only the average performance but also the reliability and variance of the training processes across multiple independent trials. As Adam represents the state-of-the-art in adaptive gradient-based optimization, it serves as the primary benchmark for evaluating the effectiveness of the A^* approach in the weight space.

7.1 Convergence Analysis: Mean Loss and Variance

The first visualization focuses on the temporal evolution of the training process. By plotting the loss as a function of iterations, we observe the convergence velocity and the stability of the optimization path.

7.1.1 Mean Loss Trajectory ($\bar{L}$)

The primary indicator in this plot is the solid line representing the average loss across N independent runs. This trajectory reveals the global behavior of the optimizer:

- **Convergence Speed:** The slope of the curve indicates how rapidly the algorithm minimizes error.

- **Plateau Level:** The terminal value of the line represents the average expected performance upon reaching $I_{\max}$.

7.1.2 Standard Deviation Shading ($\pm 1\sigma$)

The shaded region surrounding the mean line represents the standard deviation (σ). This area is a critical metric for **algorithmic robustness**:

- **High Consistency (Narrow Band):** A narrow band suggests that the algorithm is insensitive to initial weight configurations and consistently follows a similar optimization path.
- **High Volatility (Wide Band):** A wide band indicates that the search outcome is highly dependent on initialization, suggesting that the algorithm may frequently encounter diverse local minima.

7.2 Distributional Analysis: Final Loss Box Plots

While convergence plots show the "how," box plots provide a definitive "what" regarding the final state of the models. These plots summarize the statistical distribution of the terminal loss values achieved at the end of training.

7.2.1 Component Interpretation

The box plot provides a five-number summary of the terminal results:

- **The Median:** The central horizontal line represents the 50th percentile. This is our primary metric for "typical" performance, as it remains unaffected by isolated training failures.
- **The Interquartile Range (IQR):** The vertical box spans from the 25th (Q_1) to the 75th (Q_3) percentile. A smaller IQR indicates that the algorithm's results are highly clustered.
- **Whiskers:** These represent the spread of the data excluding extreme cases, providing a view of the reliable range of the optimizer.

7.2.2 Formal Identification of Outliers

To maintain statistical integrity, we utilize a standardized method for identifying anomalous runs (outliers). This prevents rare, failed training attempts from skewing the interpretation of the algorithm's general capability.

The length of the whiskers is determined by the IQR, where $IQR = Q_3 - Q_1$. Any data point L_{final} is considered an **outlier** if it falls outside the following boundaries:

$$\text{Lower Boundary} = Q_1 - 1.5 \times IQR$$

$$\text{Upper Boundary} = Q_3 + 1.5 \times IQR$$

In our analysis, outliers appearing above the upper whisker are particularly noteworthy, as they represent runs where the search failed to escape high-loss regions of the parameter space.

Chapter 8

Preliminary Study for Hyperparameters Tuning

The application of the A^* search algorithm to the training of neural networks introduces a unique set of challenges that distinguish it fundamentally from traditional gradient-based optimization. While Gradient Descent operates in a continuous landscape, A^* explores a discretized state-space of weights and biases, where the efficiency of the search is governed by the logic of neighbor generation and the management of computational resources.

This chapter details an extensive experimental campaign, spanning across various network complexities and architectural strategies, aimed at identifying the optimal balance between convergence precision and computational feasibility. Given the exponential growth of the search tree, a "vanilla" implementation of A^* is insufficient for deep architectures. Consequently, this study evaluates several advanced techniques designed to mitigate the "curse of dimensionality" and prevent hardware exhaustion.

The tuning process is structured into four primary investigative areas:

1. **Structural Adaptation:** Testing dynamic versus static kernel reshaping to simplify the architectural search space.

2. **Search Resolution:** Analyzing adaptive quantization factors to allow the algorithm to refine its search as it approaches local minima.

3. **Stochastic Exploration:** A grid search on random sampling parameters to evaluate the effectiveness of non-deterministic neighbor generation.

4. **Resource Optimization:** The integration of Beam Search to transition from

an exhaustive memory-heavy search to a resource-constrained optimization process.

By evaluating these components across three distinct network scales, this study establishes a robust hyperparameter baseline that ensures the A^* algorithm can effectively train neural networks within realistic hardware constraints.

8.1 Experimental Setup

To ensure that the results obtained across the various test phases are consistent and comparable, a standardized experimental environment was established. This setup focuses on isolating the effects of hyperparameter and optimization techniques changes while keeping the dataset and evaluation metrics constant.

8.1.1 Benchmark Dataset and Network Architectures

The **California Housing Dataset** was selected as the sole benchmark for this tuning phase. This choice provides a regression task of sufficient complexity to highlight the convergence differences between strategies. The data distribution consists of 16,512 training samples, 2,064 validation samples, and 2,064 test samples, following a standard $[80\%, 10\%, 10\%]$ split.

To assess how the algorithm scales with parameter count, three MLP architectures were defined:

- **Small Net**: $[8, 32, 16, 1]$ (coarse exploration baseline).
- **Medium Net**: $[8, 64, 32, 16, 1]$ (increased depth and width).
- **Big Net**: $[8, 128, 64, 32, 16, 1]$ (testing memory-intensive limits).

8.1.2 Evaluation Metrics and Baseline Constants

The primary performance indicator is the final average **Mean Squared Error (MSE)** calculated on the training set after 2,000 iterations. Predictive accuracy is further validated through the MSE, RMSE, MAE and R^2 scores on the validation set.

A set of default parameters, derived from early empirical trials, is maintained as a constant baseline for all tests unless specific variations are required by the experiment logic (see Table 8.1).

Table 8.1: Default hyperparameters and baseline constraints.

Parameter	**Value**	**Parameter**	**Value**
Max Iterations	2000	Parameter Range	[-10, 10]
Independent Runs	5	Early Stopping Patience	200
Min Quantization Factor	10	Max Quantization Factor	10,000
Loss Improvement Threshold	0.001	Kernels/Stride Decrement	1
Dynamic Logic Patience	100	Quantization Multiplier	10

8.1.3 Categorization of Neighbor Generation Strategies

The most critical aspect of the search algorithm is how it defines its "neighbors" (the next possible configurations of weights). This chapter compares three philosophies:

- **Single Kernel Strategy**: Uses a unified weight/bias kernel that reshapes itself to fit different layers. It represents a low-parameter deterministic approach.
- **Layer-wise Kernels Strategy**: Employs dedicated kernels for each layer, allowing for high structural customization at the cost of more hyperparameters to tune.
- **Random Neighbors Generation**: Introduces stochasticity by selecting a percentage of parameters to perturb (*perturbation ratio*) and generating a specific number of children (*search coverage ratio*).

8.1.4 Specific Tuning Experiments

Following the setup, the chapter is organized into detailed subsections for each experiment. We first examine **Kernel Dynamics**, comparing if dynamic reshaping offers any advantage over fixed kernels. We then transition to the **Quantization Factor Study**, which tests the hypothesis that increasing search resolution during stalls helps the algorithm escape plateaus.

Subsequently, a comprehensive **Grid Search** is performed on Random Sampling parameters to find the ideal balance between perturbation intensity and search breadth. Finally, we conclude with a critical analysis of **Beam Search Optimization**, where we evaluate how limiting the "beam width" affects training, specifically looking at how it prevents the system-level memory failures inherent in standard A^* searches.

8.2 Dynamic Kernels Reshaping Results

This section presents the experimental results obtained by applying the dynamic kernel reshaping technique for the **single kernel strategy** across three different network architectures. The primary objective of this evaluation is to compare the performance of dynamically reshaped kernels against statically defined ones.

Specifically, this test seeks to determine whether the dynamic approach offers superior performance or, at minimum, achieves comparable results to the static counterpart. An equivalent performance would be a significant finding, as it would simplify the architectural design process by reducing the necessity for manual selection of optimal kernel dimensions.

8.2.1 Small Net Results

The configuration parameters for the dynamic and static tests are outlined here:

- **Weight Kernel Size:** Max $[4, 4]$, Min $[2, 2]$.
- **Bias Kernel Size:** Max [4], Min [2] .
- **Strides:** Max 4, Min 2 .

Visual Analysis of Training and Stability

The first point of analysis concerns the training trajectory. As illustrated in Figure 8.1, all models benefit from an initial rapid loss reduction. However, a significant divergence occurs around iteration 250.

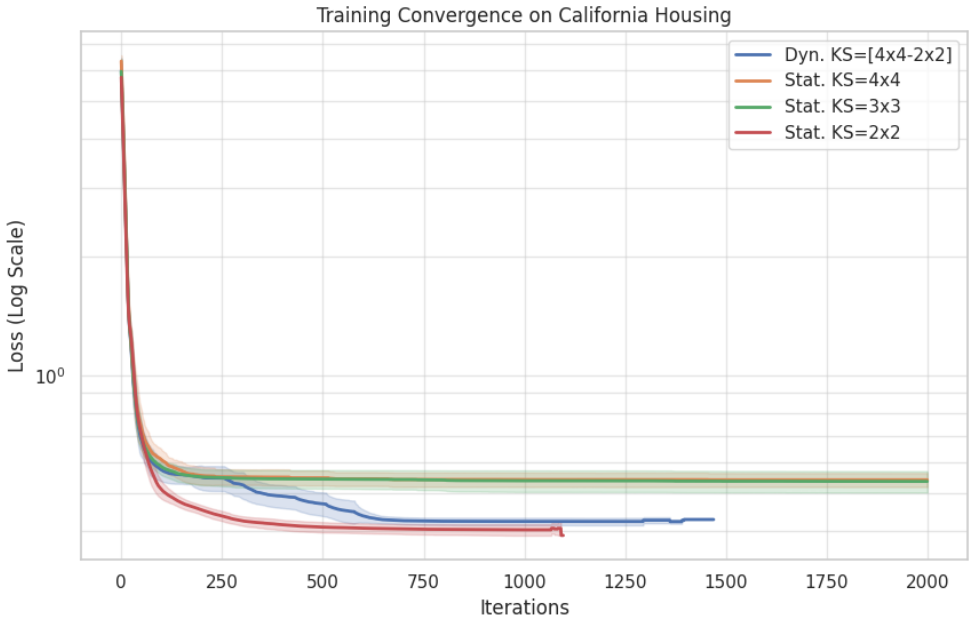

Figure 8.1: Training convergence comparison on a logarithmic scale.

The *Dynamic* approach (blue line) demonstrates a superior ability to adapt compared to larger static kernels (4×4 and 3×3), which plateau prematurely. The dynamic reshaping allows the network to "escape" these local minima, eventually settling into a performance range very close to the optimal 2×2 static configuration.

To assess the reliability of these results, Figure 8.2 displays the distribution of the final loss values across the 5 independent runs.

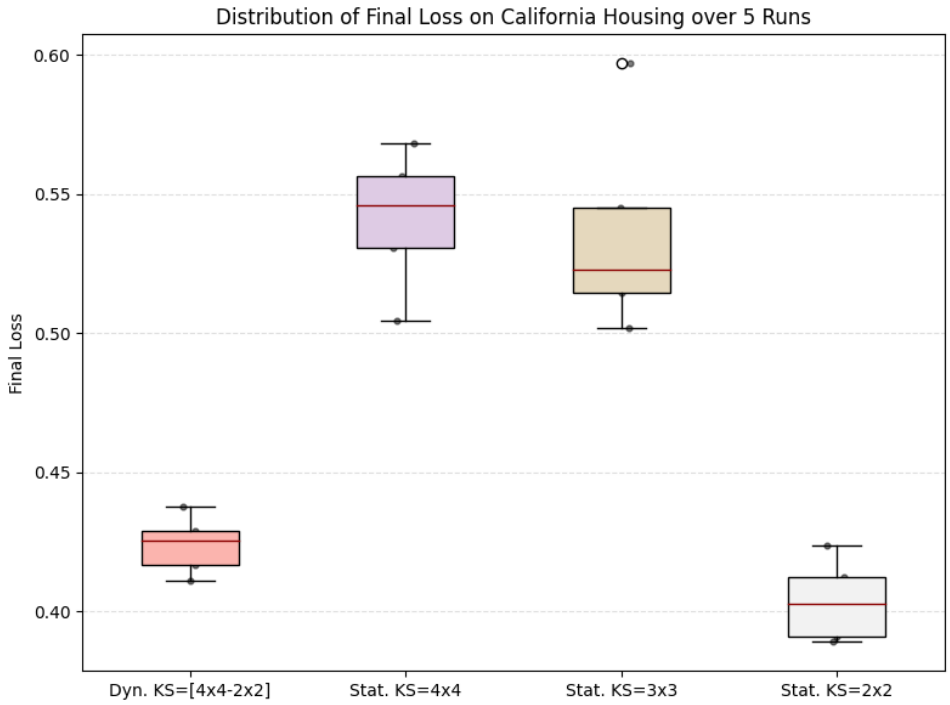

Figure 8.2: Distribution of Final Loss over 5 runs.

The boxplot highlights the stability of the *Dynamic* method, which maintains a very tight distribution. In contrast, the static 3×3 and 4×4 kernels show not only higher average loss but also greater variance and presence of outliers, suggesting a higher sensitivity to weight initialization.

Finally, we examine the broader predictive performance through the distribution of regression metrics (MSE, RMSE, MAE, and R^2), shown in Figure 8.3.

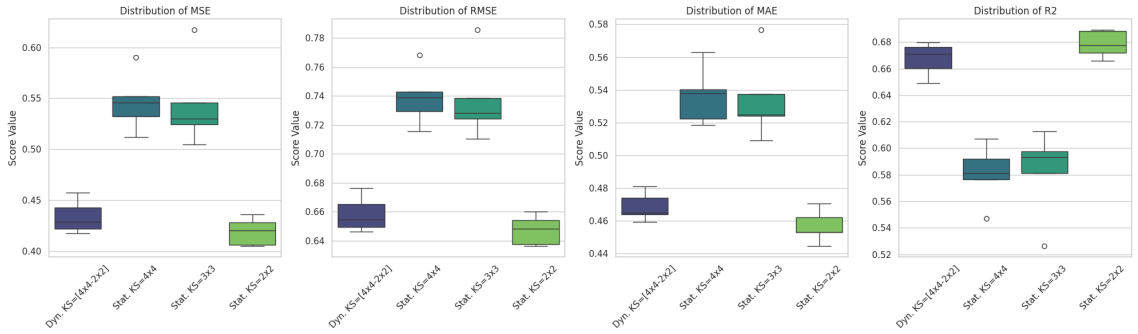

Figure 8.3: Boxplots for MSE, RMSE, MAE, and R^2 across all tested configurations.

The metrics confirm that the Dynamic approach consistently matches or exceeds the performance of large static kernels across all indicators. Notably, the R^2 distribution for the dynamic method is centered around 0.67, proving that it can autonomously find a near-optimal configuration for the California Housing task.

Summary Statistics

The numerical data summarized in Table 8.2 provides the final confirmation of these observations, illustrating the average performance across all metrics.

Metric (Avg)	**Dynamic [4x4-2x2]**	**Static 4x4**	**Static 3x3**	**Static 2x2**
Metrics computed on Training Set:				
Final Loss	0.4239	0.5411	0.5362	**0.4036**
Time (s)	736.10	**387.30**	527.97	721.80
Metrics computed on Validation Set:				
MSE	0.4337	0.5464	0.5444	**0.4190**
RMSE	0.6585	0.7390	0.7374	**0.6473**
MAE	0.4686	0.5363	0.5344	**0.4567**
R^2	0.6673	0.5808	0.5823	**0.6785**

Table 8.2: Average performance summary for Small Net.

8.2.2 Medium Net Results

The configuration parameters for the dynamic and static tests are outlined here:

- **Weight Kernel Size:** Max $[6, 6]$, Min $[2, 2]$.
- **Bias Kernel Size:** Max [6], Min [2] .
- **Strides:** Max 6, Min 2.

Visual Analysis of Training and Stability

As illustrated in Figure 8.4, the training trajectory for the Medium Net shows a clear advantage for the dynamic approach. While large static kernels (6×6 and 5×5) suffer from significant plateaus early in the training, the dynamic reshaping allows the model to continuously refine its representation.

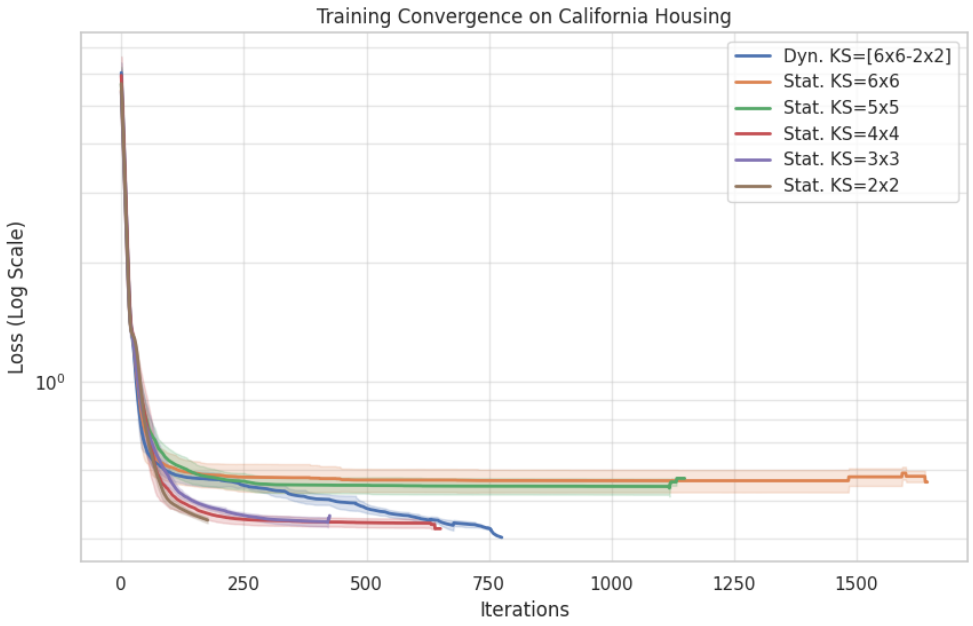

Figure 8.4: Training convergence comparison for the Medium Net on a logarithmic scale.

The *Dynamic* approach (blue line) begins to diverge from the suboptimal larger kernels after approximately 150 iterations, eventually achieving a lower final loss than even the best-performing static configurations (4×4, 3×3 and 2×2).

To assess the reliability of these results, Figure 8.5 displays the distribution of the final loss values across the 5 independent runs.

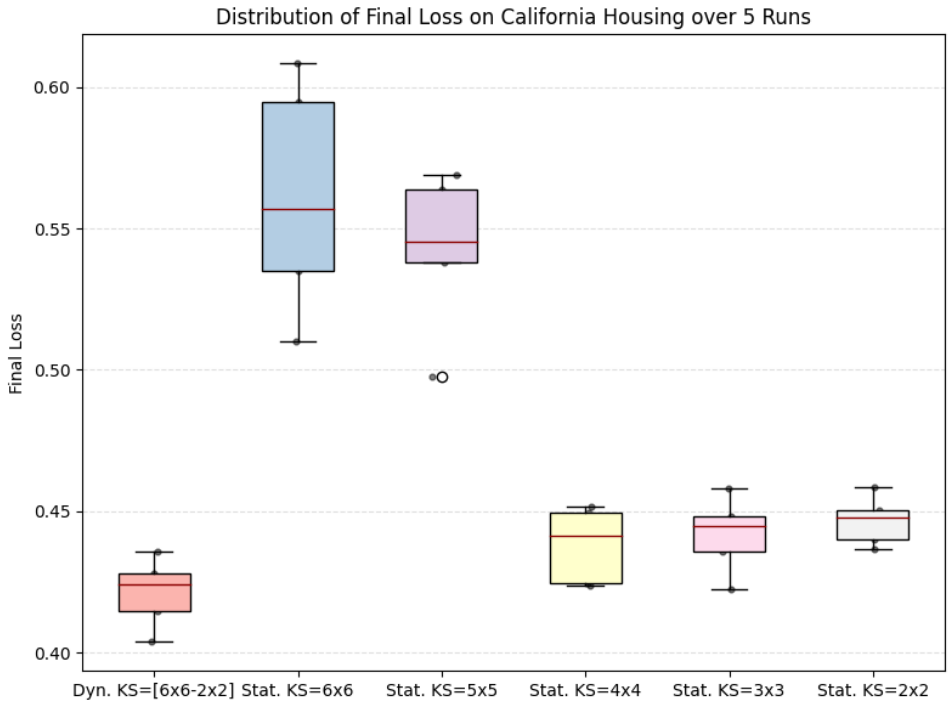

Figure 8.5: Distribution of Final Loss over 5 runs for the Medium Net.

The boxplot confirms that the *Dynamic* method is the most stable and effective configuration for this architecture. It exhibits the lowest median loss and a remarkably tight distribution, whereas the larger static kernels (6×6 and 5×5) show both poor performance and higher variance.

Finally, we examine the broader predictive performance through the distribution of regression metrics (MSE, RMSE, MAE, and R^2), shown in Figure 8.6.

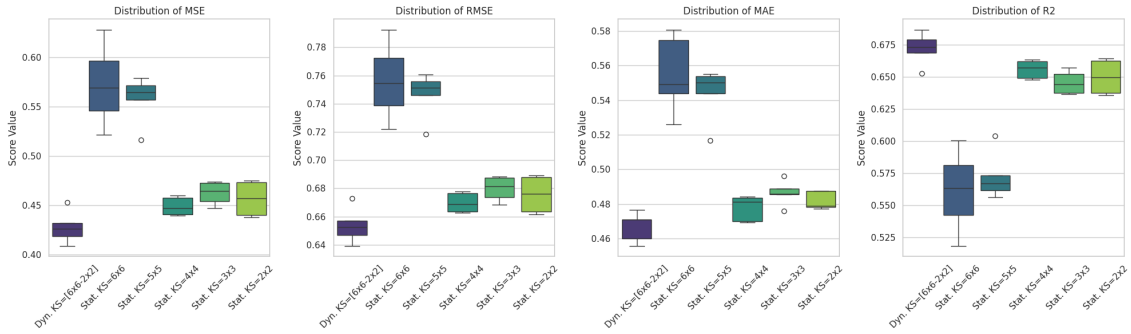

Figure 8.6: Boxplots for MSE, RMSE, MAE, and R^2 across all Medium Net configurations.

The metrics highlight a critical finding: for the Medium Net, the Dynamic approach consistently outperforms all static kernels across every indicator. Notably, the R^2 distribution for the dynamic method is centered above 0.67, while all static configurations, including the previously optimal smaller kernels, fail to reach the same level of predictive accuracy. Smaller static configurations, such as the 2x2 kernel, are structurally unable to reach optimal loss levels due to implemented early stopping. These kernels saturate available RAM prematurely, forcing training to terminate before the optimization process can effectively refine the network weights.

Summary Statistics

The numerical data in Table 8.3 provides the final confirmation. The Dynamic approach achieves the best average R^2 (0.6720) and the lowest average MSE (0.4275), proving its superiority in more complex architectures.

Metric (Avg)	**Dyn. [6x6-2x2]**	**Stat. 6x6**	**Stat. 5x5**	**Stat. 4x4**	**Stat. 3x3**	**Stat. 2x2**
Metrics computed on Training Set:						
Final Loss	**0.4212**	0.5610	0.5427	0.4382	0.4419	0.4467
Time (s)	810.86	**745.08**	745.80	751.14	758.06	756.91
Metrics computed on Validation Set:						
MSE	**0.4275**	0.5720	0.5574	0.4487	0.4622	0.4565
RMSE	**0.6537**	0.7559	0.7465	0.6698	0.6798	0.6756
MAE	**0.4670**	0.5549	0.5440	0.4777	0.4866	0.4820
R^2	**0.6720**	0.5611	0.5723	0.6557	0.6454	0.6497

Table 8.3: Average performance summary for the Medium Net architecture.

8.2.3 Big Net Results

The Big Net architecture represents the most complex scenario, evaluated with the following setup:

- **Weight Kernel Size:** Max $[8, 8]$, Min $[3, 3]$.
- **Bias Kernel Size:** Max [8], Min [3] .
- **Strides:** Max 8, Min 3.

Visual Analysis of Training and Stability

The training trajectory for the Big Net, shown in Figure 8.7, illustrates the difficulty of optimizing larger architectures. We observe a significant failure in the *Static 3x3* configuration, which remains trapped in a high-loss state.

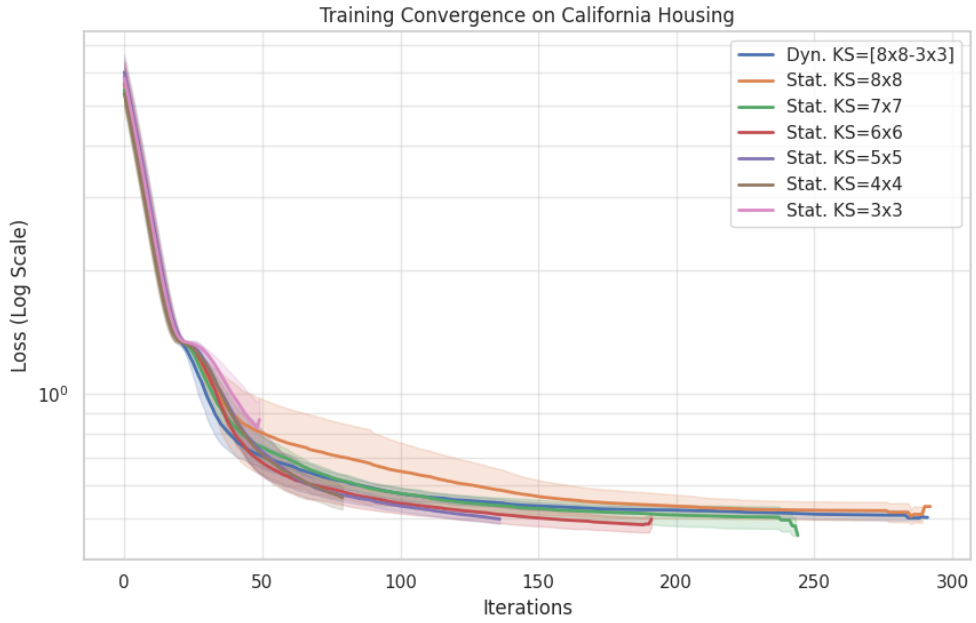

Figure 8.7: Training convergence comparison for the Big Net on a logarithmic scale.

The *Dynamic* approach (blue line) successfully avoids the initialization issues that plague the smallest static kernels. It follows a stable descent path, mirroring the behavior of the most effective static kernels (6×6 and 7×7), and proves that

starting with a larger receptive field before reshaping is beneficial for deeper or wider networks.

Figure 8.8 highlights the distribution of final loss values, providing insights into the robustness of each method.

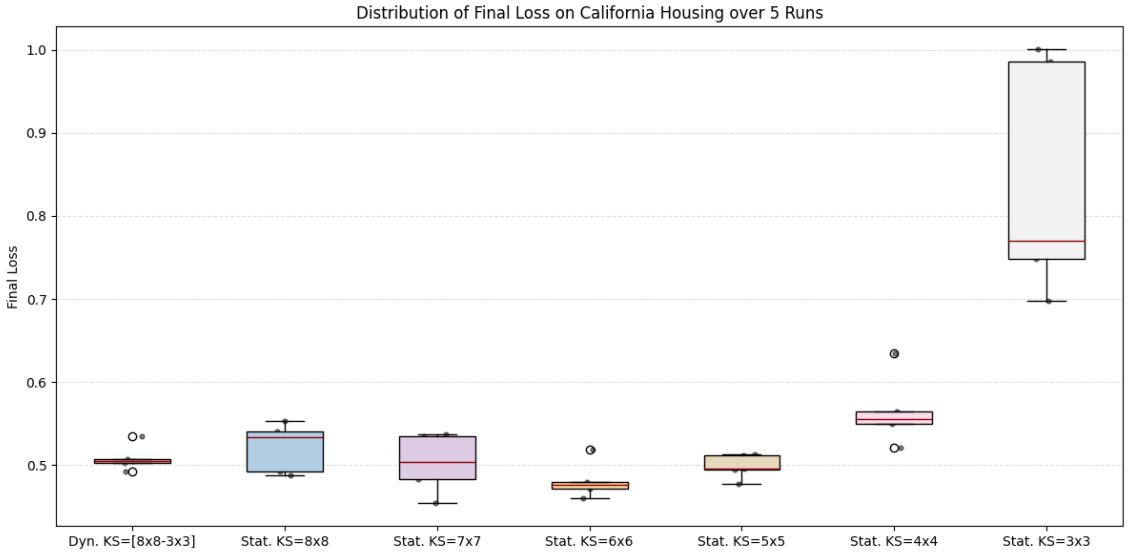

Figure 8.8: Distribution of Final Loss over 5 runs for the Big Net.

The boxplot clearly shows the large variance of the *Static 3x3* configuration. In contrast, the *Dynamic* method remains extremely stable. Although the *Static 6x6* achieved the absolute lowest median loss in this specific test, the Dynamic approach provides a nearly identical performance level without any manual tuning, acting as a reliable "safety net" against suboptimal kernel choices.

The broader predictive performance across regression metrics is presented in Figure 8.9.

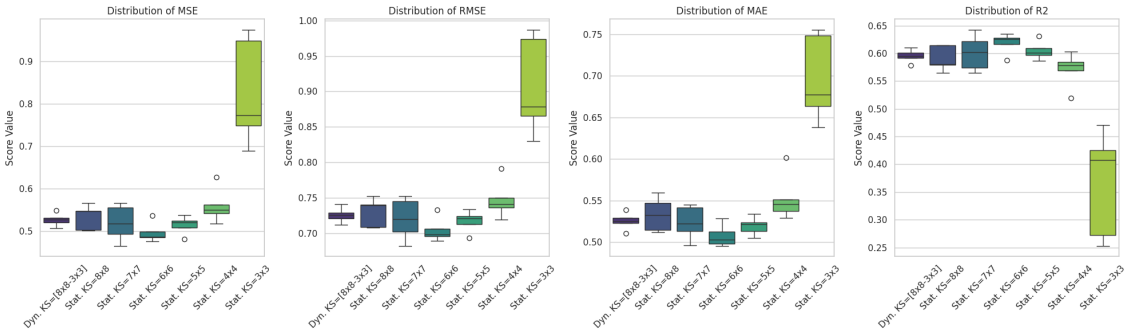

Figure 8.9: Boxplots for MSE, RMSE, MAE, and R^2 across all Big Net configurations.

These distributions confirm the general trend: the Dynamic approach matches the predictive power of the best static configurations (6x6 and 7x7). As seen in the R^2 plot, the dynamic method centers its performance around 0.60, effectively outperforming the 8x8, 4x4, and 3x3 static setups. Smaller static configurations, such as the 3x3 and 4x4 kernels, are structurally unable to reach optimal performance levels, as said before, due to the early stopping. These configurations saturate the available RAM prematurely, forcing the training to terminate before the optimization process can effectively refine the network weights to reach the best possible configuration.

Summary Statistics

Table 8.4 summarizes the average metrics for the Big Net. The results prove that dynamic reshaping is a viable strategy for large networks, offering a balanced trade-off between execution time and predictive accuracy.

Metric (Avg)	**Dyn. [8x8-3x3]**	**Stat. 8x8**	**Stat. 7x7**	**Stat. 6x6**	**Stat. 5x5**	**Stat. 4x4**	**Stat. 3x3**
Metrics computed on Training Set:							
Final Loss	0.5083	0.5214	0.5025	**0.4813**	0.4985	0.5652	0.8406
Time (s)	704.31	708.07	716.08	712.44	702.59	698.35	**696.42**
Metrics computed on Test Set:							
MSE	0.5274	0.5335	0.5201	**0.4972**	0.5149	0.5600	0.8263
RMSE	0.7262	0.7302	0.7207	**0.7050**	0.7174	0.7479	0.9069
MAE	0.5252	0.5333	0.5236	**0.5075**	0.5196	0.5530	0.6964
R^2	0.5954	0.5907	0.6010	**0.6185**	0.6050	0.5705	0.3660

Table 8.4: Average performance summary for the Big Net architecture.

8.2.4 Final Considerations on Dynamic Kernel Reshaping

The comparative analysis across Small, Medium, and Big architectures provides a comprehensive evaluation of the Dynamic Kernel Reshaping approach. The experimental results lead to several key conclusions regarding its performance, stability, and robustness against hardware constraints.

Performance and Adaptability

The primary objective was to determine if a dynamic approach could match or exceed the performance of statically defined kernels. The data suggests a nuanced but positive outcome:

- **Near-Optimal Performance:** In the Small Net, the dynamic approach achieved results nearly identical to the best static configuration (2×2), effectively closing the gap after a brief adaptation period.
- **Architectural Superiority:** In the Medium Net, the dynamic method demonstrated its maximum potential by *outperforming all static configurations*. This suggests that as network complexity increases, the ability to adapt the receptive field during training becomes a significant competitive advantage.
- **Robustness against Memory Constraints:** For the Big Net, while numerical results were comparable to the best-performing static kernels (6×6), the dynamic approach proved essential in avoiding the convergence failures observed with smaller static kernels.

A critical finding of this study is the relationship between kernel size and memory management. Smaller static configurations, such as the 2×2 kernel in the Medium Net or the 3×3 and 4×4 kernels in the Big Net, were structurally prevented from reaching optimal performance levels.

These configurations suffer from a high node-generation rate that leads to premature **RAM saturation**. Consequently, the **early stopping** forces the training to terminate before the optimization process can effectively refine the network weights. The dynamic approach successfully mitigates this by starting with larger receptive fields that are more memory-efficient, only downsizing for refinement once a stable loss region is reached.

In the end the most significant practical benefit of Dynamic Kernel Reshaping is the elimination of exhaustive hyperparameter searches. Selecting the optimal kernel size is typically a trial-and-error process that varies significantly with network depth and task complexity.

The dynamic schema acts as an **automated optimizer**: by starting with a larger receptive field to favor global feature exploration and gradually shrinking to smaller dimensions for fine-tuning, the model autonomously navigates toward a high-performance state while simultaneously avoiding the memory dangers that halt smaller static kernels.

8.3 Dynamic Quantization Factor Results

This section investigates the impact of dynamically adjusting the quantization factor during the training process for the **single kernel approach**. The primary hypothesis is that increasing the quantization resolution when the loss improvement stalls can allow the A^* algorithm to perform finer weight adjustments, potentially refining the solution quality.

The dynamic approach is compared against several static quantization factors ($QF \in \{10^1, 10^2, 10^3, 10^4\}$) to evaluate whether an adaptive strategy can achieve the high precision of a large QF without the computational search burden or the convergence difficulties often associated with high-resolution discrete spaces.

8.3.1 Small Net Results

For the Small Net architecture, the following parameters were utilized to establish the baseline and dynamic behavior:

- **Weight Kernel Size:** $[2, 2]$ (Static).
- **Bias Kernel Size:** $[2]$ (Static).
- **Strides:** 2.
- **Dynamic QF Range:** Min 10, Max $10,000$ (Multiplier: 10).

Visual Analysis of Training and Stability

The convergence behavior, as depicted in Figure 8.10, reveals a stark contrast between low and high quantization factors. Low static factors (10^1 and 10^2) lead to rapid initial convergence. In contrast, static factors of 10^3 and 10^4 struggle significantly, failing to achieve a competitive loss within the same iteration limit.

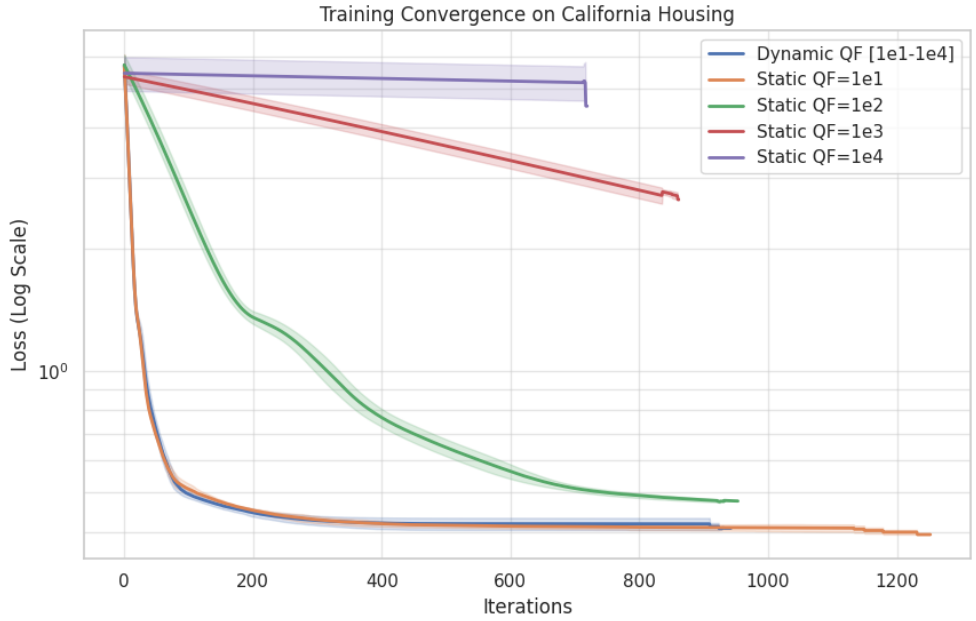

Figure 8.10: Training convergence comparison for various Quantization Factors (Log Scale).

The *Dynamic QF* strategy (blue line) successfully mirrors the rapid descent of the $QF = 10$ configuration. This suggests that starting with a lower resolution allows for broad, efficient exploration of the weight space, while the dynamic increases ensure the model maintains progress.

Figure 8.11 illustrates the distribution of the final loss. The Dynamic QF approach demonstrates high stability, with a median loss of approximately 0.41, nearly identical to the best-performing static model ($QF = 10$).

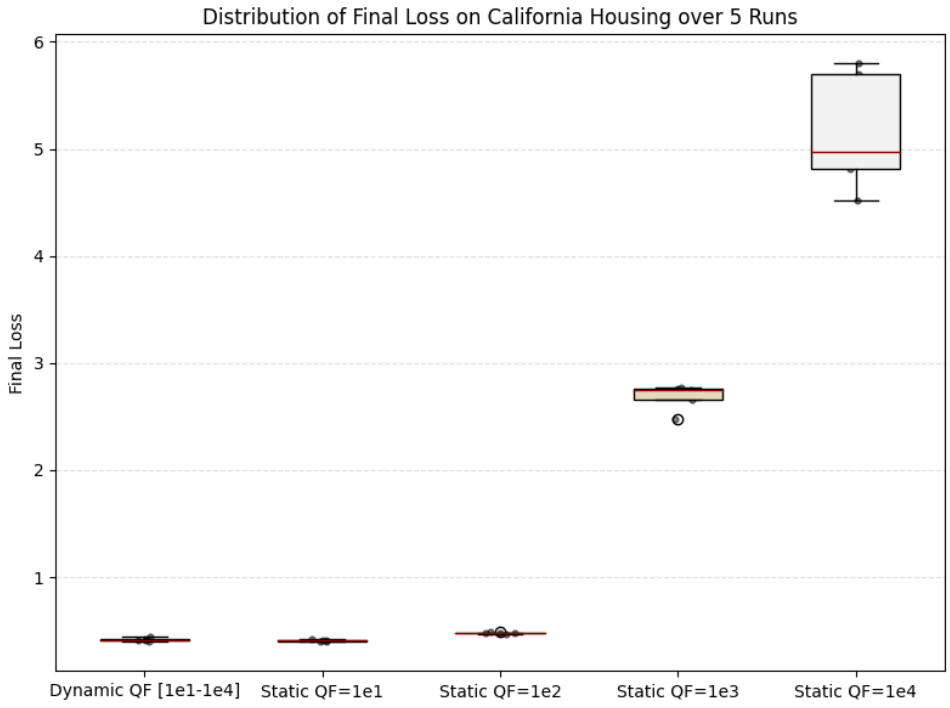

Figure 8.11: Distribution of Final Loss over 5 runs for Quantization Factor tests.

As the static quantization factor increases beyond 10^2, we observe not only a higher average loss but also a significant increase in variance. This confirms that excessively high quantization at the start of training makes the search landscape too complex for the algorithm to navigate effectively.

The regression metrics across the 5 runs are summarized in Figure 8.12.

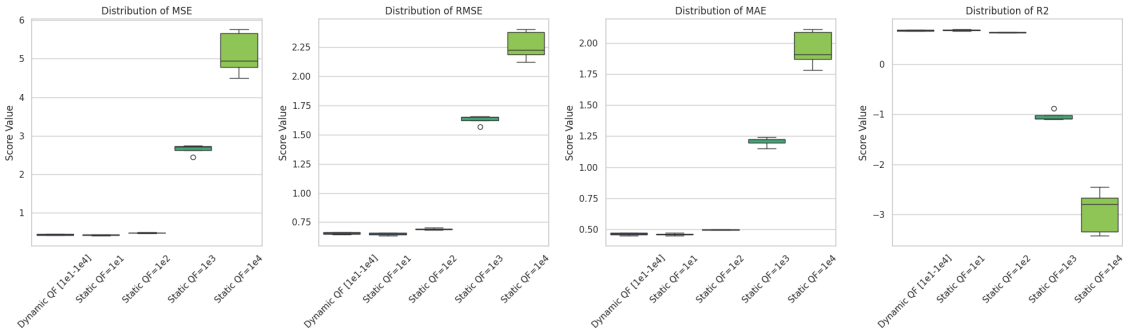

Figure 8.12: Regression metrics distribution (MSE, RMSE, MAE, and R^2) for Quantization Factor configurations.

The metrics highlight that the *Dynamic QF* and *Static QF=10, 100* are the only configurations achieving a positive and high R^2 score (approximately 0.67). For static factors of 10^3 and 10^4, the R^2 values become negative, indicating that the models perform worse than a horizontal baseline (mean of target values), further justifying the need for a dynamic or low-starting quantization approach.

Summary Statistics

The average values for all metrics, compiled in Table 8.5, provide a definitive comparison of the performance levels.

Metric (Avg)	**Dynamic [1e1-1e4]**	**Static 1e1**	**Static 1e2**	**Static 1e3**	**Static 1e4**
Metrics computed on Training Set:					
Final Loss	0.4184	**0.4085**	0.4770	2.6815	5.1592
Time (s)	667.96	777.83	644.30	579.98	**492.87**
Metrics computed on Validation Set:					
MSE	0.4268	**0.4208**	0.4788	2.6506	5.1290
RMSE	0.6532	**0.6486**	0.6919	1.6277	2.2621
MAE	0.4634	**0.4618**	0.4983	1.2077	1.9496
R^2	0.6725	**0.6772**	0.6327	-1.0336	-2.9352

Table 8.5: Average performance summary for Small Net.

8.3.2 Medium Net Results

The evaluation of the Dynamic Quantization Factor (QF) was extended to the Medium Net architecture to assess how increased model complexity interacts with search space resolution. The configuration remained consistent with the previous test to ensure comparability:

- **Weight Kernel Size:** $[4, 4]$ (Static).
- **Bias Kernel Size:** $[4]$ (Static).
- **Strides:** 4.
- **Dynamic QF Range:** Min 10, Max $10,000$ (Multiplier: 10).

Visual Analysis of Training and Stability

As shown in Figure 8.13, the convergence trends for the Medium Net reinforce the observations from the Small Net but with more pronounced performance gaps. Static factors of 10^3 and 10^4 show almost no effective optimization, maintaining a high loss throughout the iterations.

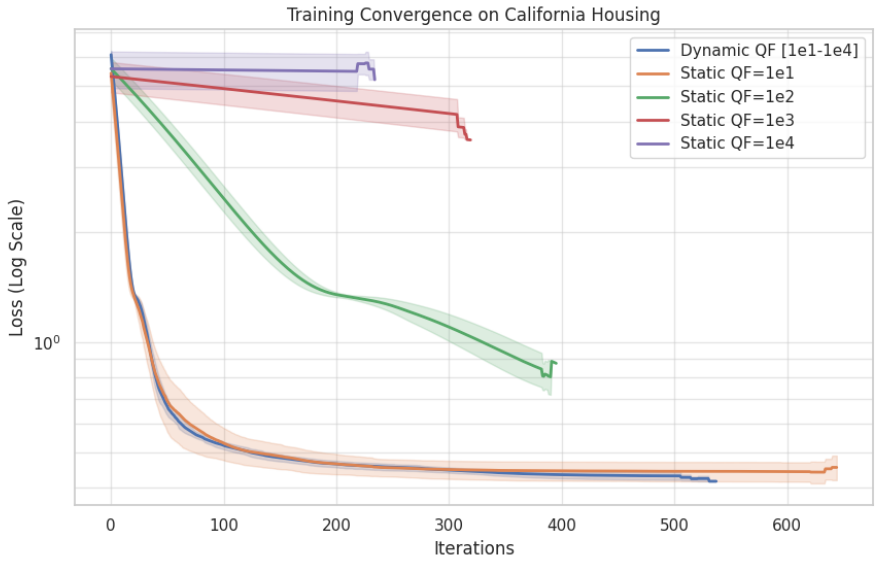

Figure 8.13: Training convergence comparison for Medium Net (Log Scale).

Interestingly, the *Dynamic QF* strategy (blue line) demonstrates a highly efficient trajectory, converging rapidly and reaching a final loss state that is slightly lower than the $QF = 10$ static baseline. This suggests that as the network grows in size, the ability to dynamically refine weight updates becomes increasingly valuable for fine-tuning.

Figure 8.14 highlights the stability of these results. The Dynamic QF approach maintains a very low variance (Std: 0.0108), outperforming the Static $QF = 10$ in terms of consistency.

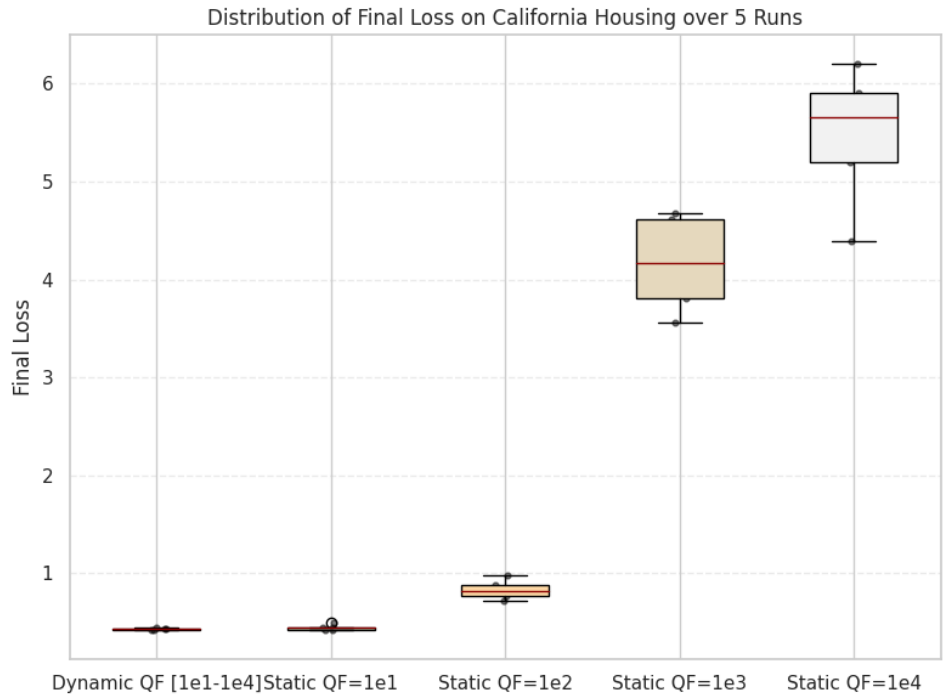

Figure 8.14: Distribution of Final Loss for Medium Net configurations.

A critical observation is the behavior of the Static $QF = 10^2$ configuration. While it was somewhat competitive in the Small Net, its performance degrades significantly in the Medium Net (Avg Loss 0.83 vs 0.47 in Small Net). This indicates that higher-dimensional networks are more sensitive to finer parameter space discretization.

The regression metrics for the Medium Net are visualized in Figure 8.15.

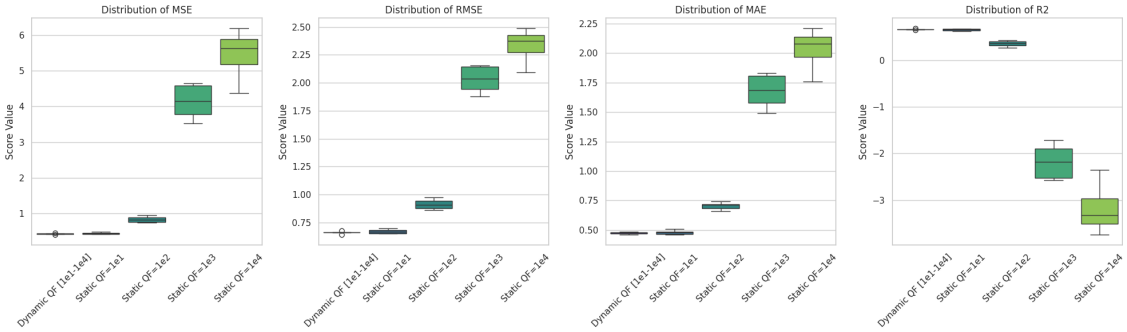

Figure 8.15: Regression metrics distribution for Medium Net configurations.

The metrics confirm that the *Dynamic QF* achieves the best overall performance, with an average R^2 of 0.6646, slightly surpassing the 0.6535 achieved by the Static $QF = 10$. Configurations with $QF \geq 10^3$ fail to produce meaningful predictive models, as evidenced by the high MSE and negative R^2 values.

Summary Statistics

Table 8.6 provides the numerical breakdown of the average metrics for the Medium Net.

Metric (Avg)	**Dynamic [1e1-1e4]**	**Static 1e1**	**Static 1e2**	**Static 1e3**	**Static 1e4**
Metrics computed on Training Set:					
Final Loss	**0.4308**	0.4423	0.8327	4.1656	5.4733
Time (s)	618.54	739.54	465.80	379.06	**283.61**
Metrics computed on Validation Set:					
MSE	**0.4371**	0.4516	0.8373	4.1377	5.4457
RMSE	**0.6611**	0.6717	0.9140	2.0312	2.3295
MAE	**0.4742**	0.4794	0.7030	1.6773	2.0293
R^2	**0.6646**	0.6535	0.3576	-2.1746	-3.1782

Table 8.6: Average performance summary for Medium Net.

8.3.3 Big Net Results

Finally, the study concludes with the Big Net architecture. As the number of parameters increases, the search space becomes exponentially more complex, making the initial selection of a quantization factor a determining factor for training success. The experimental parameters remained as follows:

- **Weight Kernel Size:** $[8, 8]$ (Static).
- **Bias Kernel Size:** $[8]$ (Static).
- **Strides:** 8.
- **Dynamic QF Range:** Min 10, Max $10,000$ (Multiplier: 10).

Visual Analysis of Training and Stability

The convergence patterns for the Big Net, shown in Figure 8.16, demonstrate that the A^* algorithm's efficiency is highly dependent on low-resolution starts for large architectures. Similar to previous networks, $QF = 10^3$ and $QF = 10^4$ are unable to achieve meaningful optimization, plateauing almost immediately.

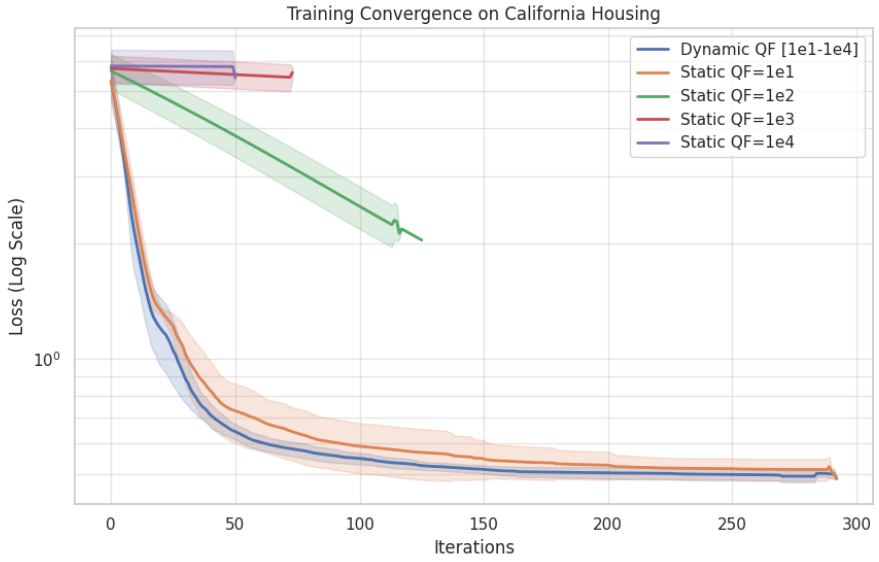

Figure 8.16: Training convergence comparison for Big Net (Log Scale).

The *Dynamic QF* strategy (blue line) exhibits a superior convergence rate even compared to the best static configuration ($QF = 10$). In this high-dimensional setting, the dynamic approach proves to be the most robust, navigating the complex loss landscape more effectively than any fixed-resolution method.

Figure 8.17 confirms this superiority. Not only does the Dynamic QF achieve the lowest average loss, but it also demonstrates significantly lower variance compared to the $QF = 10$ static run, which shows several higher-loss outliers.

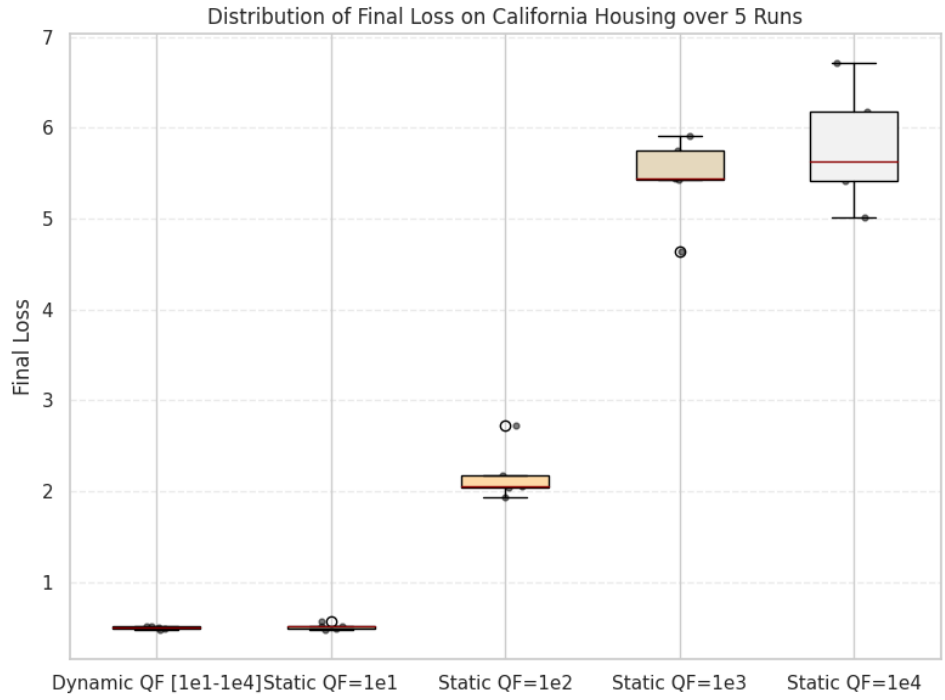

Figure 8.17: Distribution of Final Loss for Big Net configurations.

The failure of the $QF = 10^2$ configuration is even more evident here than in the Medium Net; while it manages a slow descent, its final loss (Avg 2.18) remains far from the optimal region, highlighting that higher parameter counts require even more careful discretization management.

The comparative regression metrics are presented in Figure 8.18.

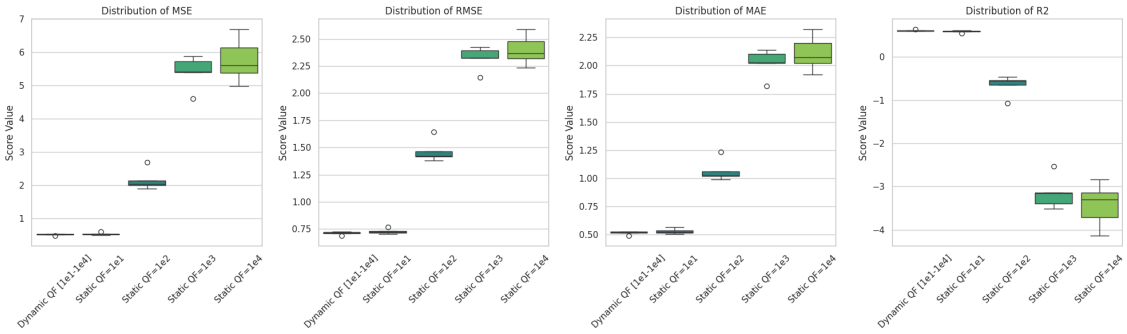

Figure 8.18: Regression metrics distribution for Big Net configurations.

The *Dynamic QF* achieves an average R^2 of 0.6106, outperforming the 0.5931 of the static $QF = 10$ case. The significant drop in R^2 for $QF = 10^2$ (which becomes negative at -0.65) underscores the "discretization trap" where a slightly too-high resolution prevents the algorithm from finding the global descent direction in large models.

Summary Statistics

The numerical results for the Big Net are detailed in Table 8.7.

Metric (Avg)	**Dynamic [1e1-1e4]**	**Static 1e1**	**Static 1e2**	**Static 1e3**	**Static 1e4**
Metrics computed on Training Set:					
Final Loss	**0.4959**	0.5133	2.1827	5.4331	5.7914
Time (s)	703.85	716.39	300.36	190.66	**135.31**
Metrics computed on Validation Set:					
MSE	**0.5075**	0.5303	2.1525	5.4062	5.7643
RMSE	**0.7123**	0.7279	1.4642	2.3231	2.3977
MAE	**0.5146**	0.5290	1.0635	2.0227	2.1074
R^2	**0.6106**	0.5931	-0.6515	-3.1478	-3.4226

Table 8.7: Average performance summary for Big Net.

8.3.4 Conclusion on Dynamic Quantization Factor Study

The experimental campaign across three architectures of increasing complexity provides a clear understanding of how the quantization factor (QF) influences the A^* search space in neural network training.

- **Robustness against the "Discretization Trap":** The most significant finding is that while low static quantization ($QF = 10$) performs well, static factors from 10^2 upwards lead to a rapid degradation of performance as the network size increases. The *Dynamic QF* strategy effectively mitigates this risk by starting with a coarse resolution to find the global descent direction and only increasing precision when local stalls occur.

- **Performance Trends in Scalability:** For Medium and Big Net architectures, the *Dynamic QF* approach did not only match but consistently **outperformed** the best static configurations in terms of final MSE and R^2 score. While the margin of improvement in the Small Net was negligible, the benefits became more pronounced as the parameter space expanded, suggesting that adaptive precision is a key requirement for scaling A^*-based training.

- **Operational Efficiency:** Beyond the raw metrics, the primary advantage of the dynamic approach is the **elimination of the hyperparameter search process**. Manually identifying the optimal QF for a specific architecture is a computationally expensive trial-and-error task. The dynamic strategy offers a "set-and-forget" solution that guarantees near-optimal or superior performance across different network depths.

- **Convergence Stability:** As evidenced by the boxplots, the Dynamic QF approach significantly reduced the variance of the final loss compared to static methods. This stability is crucial for ensuring reliable training outcomes across multiple runs, regardless of weight initialization.

In conclusion, the **Dynamic Quantization Factor** is an essential optimization. It provides the algorithm with the flexibility to balance broad exploration with high-precision refinement, making the training process more accurate removing a critical hyperparameter from manual tuning.

8.4 Random Neighbors Generation: Grid Search on Sampling Parameters

This section explores the *Random Neighbors Generation* strategy, which introduces a stochastic approach to weight optimization. Unlike the deterministic kernel-based methods, this scheme relies on two primary hyperparameters to define the search neighborhood: the *Perturbation Ratio* and the *Search Coverage Ratio*.

The *Perturbation Ratio* determines the cardinality of the subset of parameters updated simultaneously within each layer. It is expressed as a percentage of the total number of parameters per layer ($|W_{layer}| \times$ Perturbation Ratio). The *Search Coverage Ratio* governs the total number of child nodes generated for each search step, similarly defined as a percentage of the total parameters in the layer.

The objective of this grid search is to identify the optimal balance between the intensity of each update (perturbation) and the breadth of the local search (coverage) to maximize convergence efficiency in a discrete state-space.

8.4.1 Small Net Results

For the Small Net architecture, a grid search was conducted using the following parameter combinations:

- **Perturbation Ratios:** $[0.01, 0.05, 0.1]$ $(1\%, 5\%, 10\%)$.
- **Search Coverage Ratios:** $[0.05, 0.1, 0.2]$ $(5\%, 10\%, 20\%)$.

Visual Analysis of Convergence and Loss Distribution

The training trajectories for all nine configurations are illustrated in Figure 8.19.

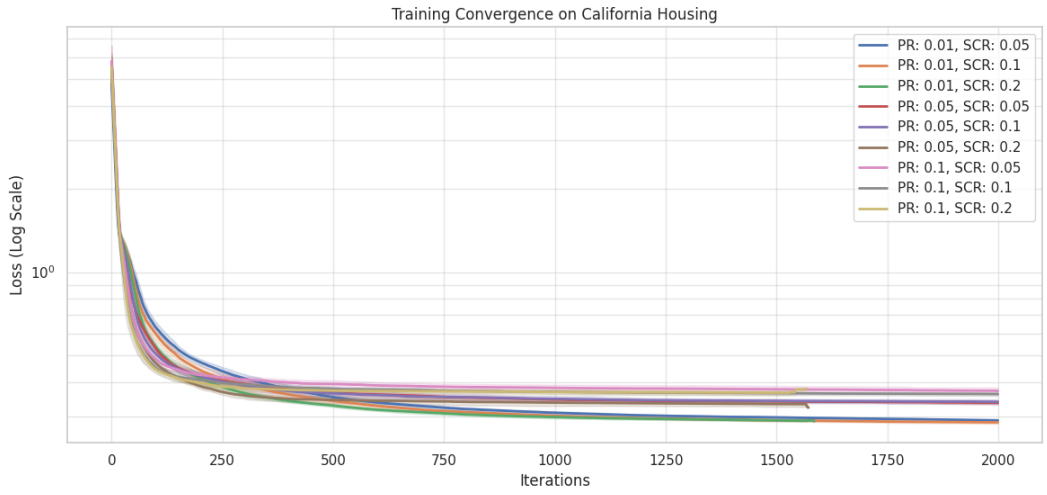

Figure 8.19: Training convergence for Small Net using various Random Sampling configurations (Log Scale).

The results indicate a clear sensitivity to the perturbation ratio. The configurations with a lower perturbation ratio (1%, depicted in blue and green) exhibit the most stable and deepest descent. As the perturbation ratio increases to 5% and 10%, the algorithm struggles to refine the solution, likely due to "jumping" over narrow local minima.

The stability of these stochastic updates is further examined in the final loss distribution shown in Figure 8.20.

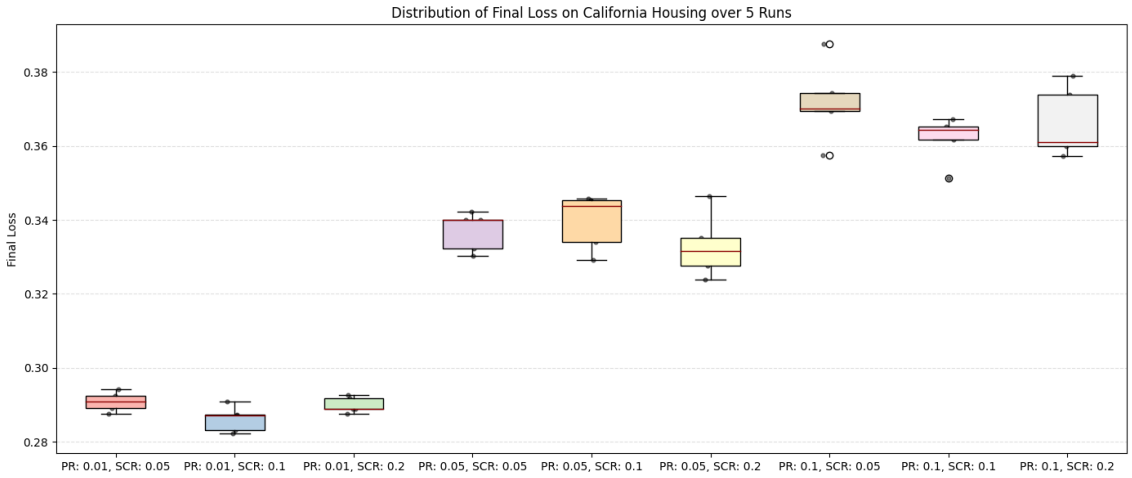

Figure 8.20: Distribution of Final Loss over 5 runs across the grid search space.

The best performing configuration is **Perturbation Ratio: 0.01** with a **Search Coverage Ratio: 0.1**, which achieves the lowest average loss with remarkably low variance. Interestingly, increasing coverage (Search Ratio) from 0.1 to 0.2 does not significantly improve the final loss for the 0.01 perturbation case, but it substantially increases the computational time.

The regression metrics across the grid are summarized in Figure 8.21.

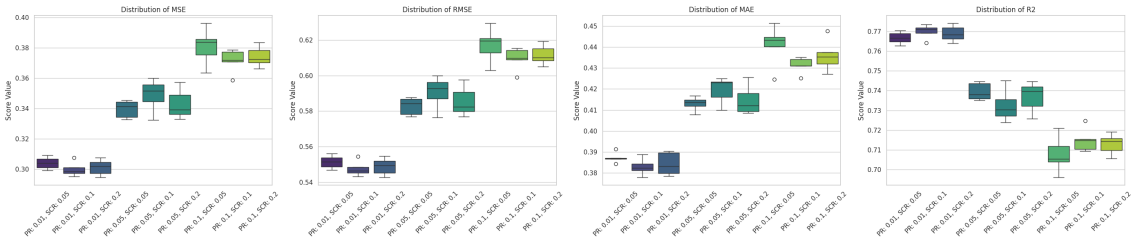

Figure 8.21: Distribution of MSE, RMSE, MAE, and R^2 for the Random Sampling Grid Search.

The R^2 scores confirm that smaller perturbations are vastly superior for this task, with configurations at 1% perturbation reaching R^2 values near 0.77. This suggests that for a network of this size, high-precision, small-scale stochastic adjustments are more effective than larger, more aggressive updates.

Summary Statistics

Tables below provide the numerical breakdown of the average performance for each tested configuration.

Pert. / Search	**0.01 / 0.05**	**0.01 / 0.1**	**0.01 / 0.2**	**0.05 / 0.05**	**0.05 / 0.1**
Metrics computed on Training Set:					
Avg Final Loss	0.2909	**0.2862**	0.2900	0.3369	0.3396
Avg Time (s)	325.44	562.40	855.63	**278.16**	535.81
Metrics computed on Validation Set:					
Avg MSE	0.3040	**0.2998**	0.3012	0.3396	0.3488
Avg RMSE	0.5514	**0.5476**	0.5488	0.5827	0.5905
Avg MAE	0.3872	**0.3829**	0.3843	0.4130	0.4195
Avg R^2	0.7668	**0.7700**	0.7689	0.7395	0.7324

Table 8.8: Average statistical metrics for Small Net Random Sampling (Part 1).

Pert. / Search	**0.05 / 0.2**	**0.1 / 0.05**	**0.1 / 0.1**	**0.1 / 0.2**
Metrics computed on Training Set:				
Avg Final Loss	0.3329	0.3718	0.3620	0.3662
Avg Time (s)	823.20	270.92	524.14	805.04
Metrics computed on Validation Set:				
Avg MSE	0.3429	0.3810	0.3715	0.3741
Avg RMSE	0.5855	0.6172	0.6095	0.6116
Avg MAE	0.4148	0.4408	0.4313	0.4359
Avg R^2	0.7369	0.7077	0.7150	0.7130

Table 8.9: Average statistical metrics for Small Net Random Sampling (Part 2).

8.4.2 Medium Net Results

For the Medium Net architecture, the grid search was conducted using the same parameter combinations to assess how increased network complexity influences sampling dynamics:

- **Perturbation Ratios:** $[0.01, 0.05, 0.1]$ $(1\%, 5\%, 10\%)$.
- **Search Coverage Ratios:** $[0.05, 0.1, 0.2]$ $(5\%, 10\%, 20\%)$.

Visual Analysis of Convergence and Loss Distribution

The training trajectories for the nine configurations on the Medium Net are illustrated in Figure 8.22.

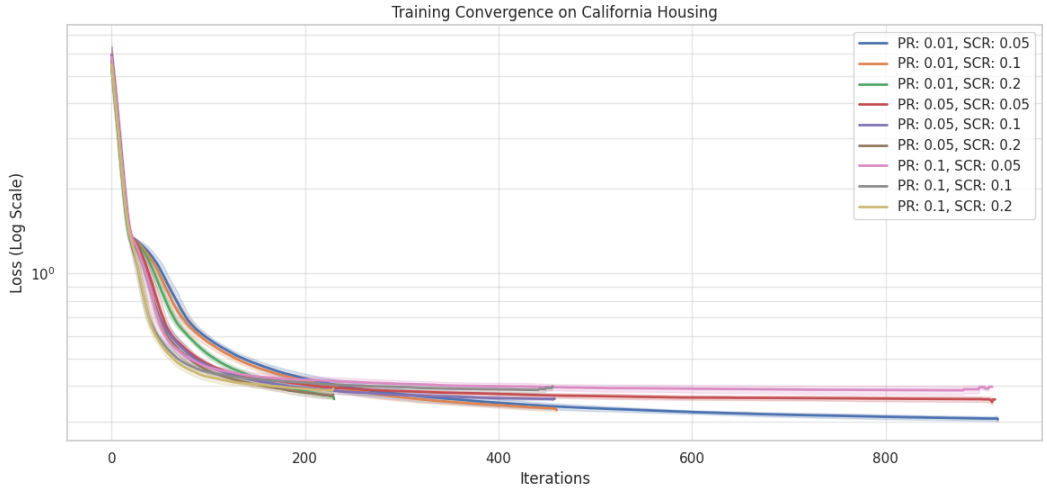

Figure 8.22: Training convergence for Medium Net using various Random Sampling configurations (Log Scale).

In this architecture, sensitivity to the *perturbation ratio* remains the dominant factor. It is observed that the configuration with 1% perturbation and 5% coverage (Pert. Ratio: 0.01, Search Ratio: 0.05) produces the most rapid and deepest descent. Compared to the Small Net, the increased number of parameters seems to make a very narrow local search (low Search Ratio) more effective, indicating that in higher-dimensional spaces, over-exploration through too many random directions can be less efficient than precision in individual steps.

The stability of these stochastic updates is further examined in the final loss distribution shown in Figure 8.23.

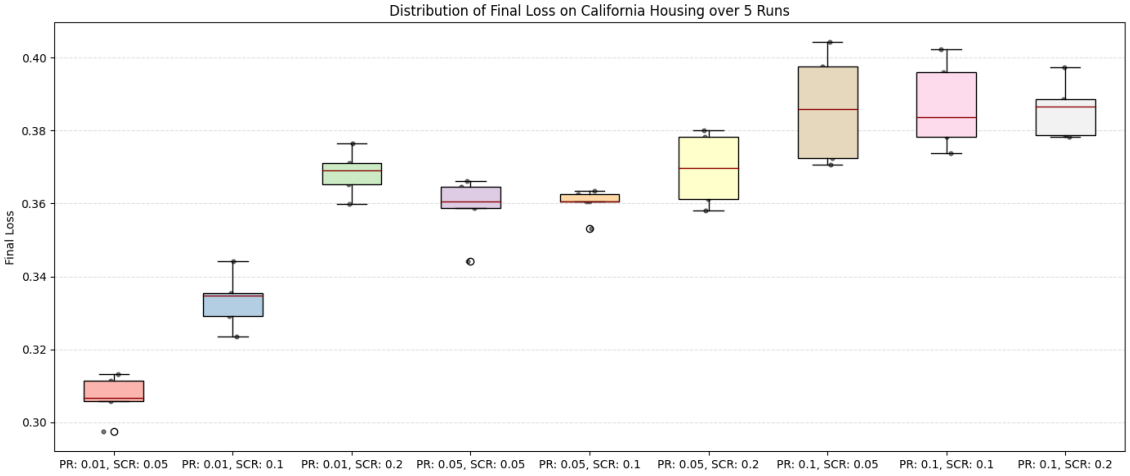

Figure 8.23: Distribution of Final Loss over 5 runs across the grid search space for Medium Net.

The **Perturbation Ratio: 0.01** with **Search Ratio: 0.05** configuration is confirmed as the best performer, achieving the lowest average loss (0.3068) with extremely low variance (0.000030). Interestingly, for this network, increasing the coverage (Search Ratio) to 0.1 or 0.2 actually degrades the final loss, highlighting that higher values lead to premature training stops due to main memory saturation.

The regression metrics for the Medium Net are summarized in Figure 8.24.

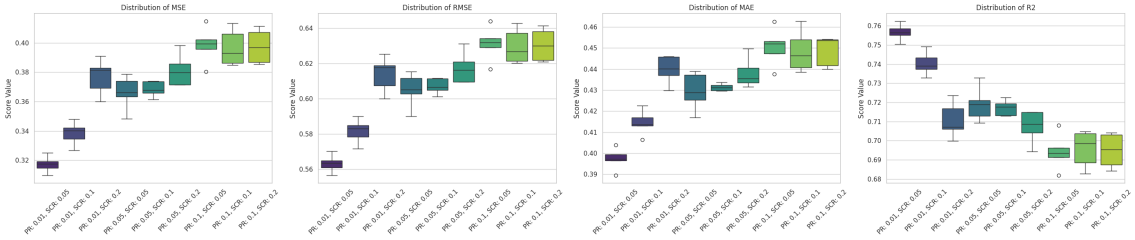

Figure 8.24: Distribution of MSE, RMSE, MAE, and R^2 for the Medium Net Random Sampling Grid Search.

The R^2 scores highlight that predictive performance decreases linearly as the perturbation increases. The optimal configuration reaches an average R^2 of ap-

proximately 0.7566, demonstrating that even with a deeper network, random sampling with small increments is capable of effectively modeling the California Housing dataset.

Summary Statistics

The tables below provide the numerical breakdown of the average performance for each tested configuration on the Medium Net architecture.

Pert. / Search	**0.01 / 0.05**	**0.01 / 0.1**	**0.01 / 0.2**	**0.05 / 0.05**	**0.05 / 0.1**
Metrics computed on Training Set:					
Avg Final Loss	**0.3068**	0.3334	0.3684	0.3588	0.3601
Avg Time (s)	802.43	790.59	781.27	788.71	783.09
Metrics computed on Validation Set:					
Avg MSE	**0.3173**	0.3384	0.3770	0.3662	0.3686
Avg RMSE	**0.5632**	0.5817	0.6139	0.6051	0.6071
Avg MAE	**0.3973**	0.4145	0.4397	0.4295	0.4313
Avg R^2	**0.7566**	0.7404	0.7108	0.7191	0.7172

Table 8.10: Average statistical metrics for Medium Net Random Sampling (Part 1).

Pert. / Search	**0.05 / 0.2**	**0.1 / 0.05**	**0.1 / 0.1**	**0.1 / 0.2**
Metrics computed on Training Set:				
Avg Final Loss	0.3695	0.3862	0.3868	0.3859
Avg Time (s)	777.98	**774.91**	775.43	779.09
Metrics computed on Validation Set:				
Avg MSE	0.3814	0.3985	0.3967	0.3976
Avg RMSE	0.6175	0.6312	0.6297	0.6305
Avg MAE	0.4382	0.4505	0.4485	0.4486
Avg R^2	0.7074	0.6942	0.6957	0.6949

Table 8.11: Average statistical metrics for Medium Net Random Sampling (Part 2).

8.4.3 Big Net Results

For the Big Net architecture, the grid search explores how stochastic optimization scales with a significantly larger parameter space. The test conditions remain consistent with the previous architectures:

- **Perturbation Ratios:** $[0.01, 0.05, 0.1]$ $(1\%, 5\%, 10\%)$.
- **Search Coverage Ratios:** $[0.05, 0.1, 0.2]$ $(5\%, 10\%, 20\%)$.

Visual Analysis of Convergence and Loss Distribution

The training trajectories for the Big Net are presented in Figure 8.25.

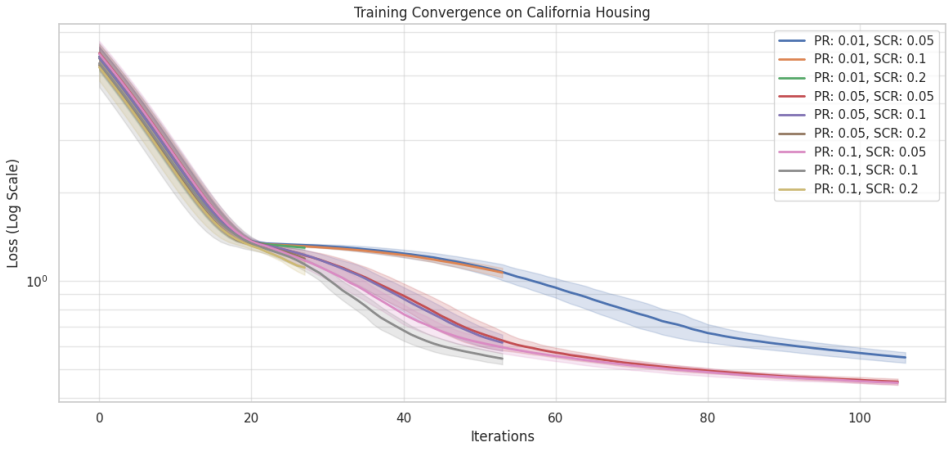

Figure 8.25: Training convergence for Big Net using various Random Sampling configurations (Log Scale).

A significant shift in behavior is observed compared to the smaller networks. While in the Small Net a low perturbation was universally better, the Big Net shows that a higher *Perturbation Ratio* (0.05 or 0.1), when paired with a very low *Search Coverage Ratio* (0.05), results in the most effective optimization. This suggests that in high-dimensional spaces, a lower search coverage ratio (0.05) combined with a higher perturbation ratio is more effective. This configuration minimizes memory consumption, allowing the training process to proceed longer and ultimately achieve better results than a wide, fine-grained search breadth.

The distribution of final loss values is shown in Figure 8.26, highlighting a clear trend between the two analyzed parameters.

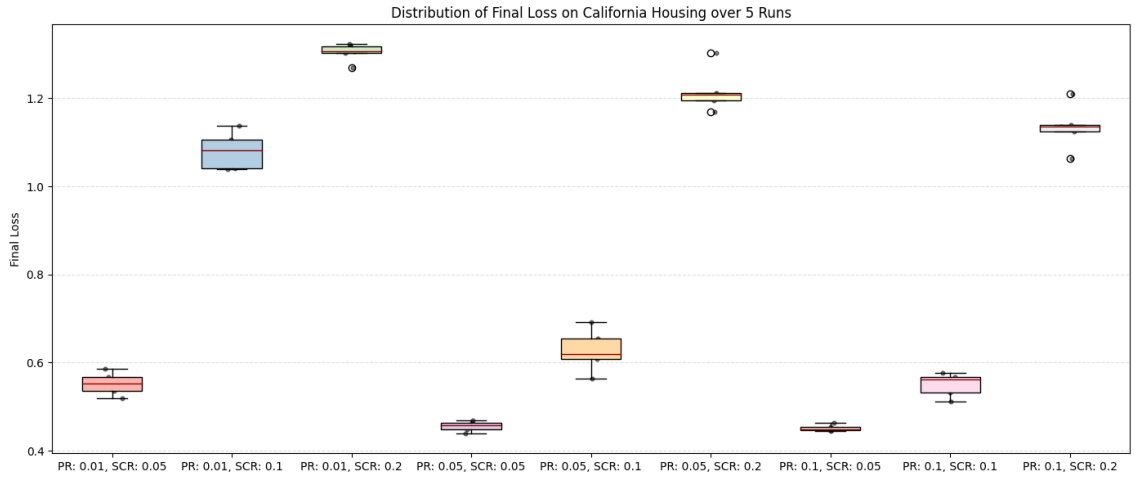

Figure 8.26: Distribution of Final Loss over 5 runs across the grid search space for Big Net.

The best-performing configurations are **Perturbation Ratio: 0.1 / Search Coverage Ratio: 0.05** (Avg Loss: 0.4518) and **Perturbation Ratio: 0.05 / Search Coverage Ratio: 0.05** (Avg Loss: 0.4554). Conversely, any configuration with a high Search Ratio (0.2) performs poorly, with losses exceeding 1.1, indicating that searching too many random directions exhausts main memory prematurely; this results in a lack of refinement as the model is unable to complete its full training cycle.

The regression metrics for the Big Net are visualized in Figure 8.27.

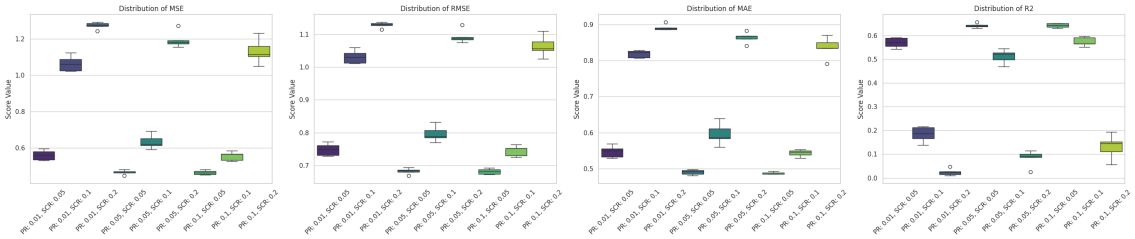

Figure 8.27: Distribution of MSE, RMSE, MAE, and R^2 for the Big Net Random Sampling Grid Search.

The R^2 scores confirm that the model's predictive power is highly sensitive to the search coverage ratio in large networks. While optimal configurations achieve an R^2 of approximately 0.6428, increasing the Search Ratio to 0.2 causes the R^2 to drop

dramatically toward zero (0.02 to 0.13). This collapse is not a failure of the search logic itself, but a consequence of premature training termination; the broader search leads to rapid main memory exhaustion, which forces a premature stop and prevents the model from reaching convergence.

Summary Statistics

The tables below summarize the average statistical performance for all tested configurations on the Big Net.

Pert. / Search	**0.01 / 0.05**	**0.01 / 0.1**	**0.01 / 0.2**	**0.05 / 0.05**	**0.05 / 0.1**
Metrics computed on Training Set:					
Avg Final Loss	0.5523	1.0809	1.3036	0.4554	0.6274
Avg Time (s)	733.17	**726.02**	739.29	727.94	729.05
Metrics computed on Validation Set:					
Avg MSE	0.5609	1.0637	1.2724	0.4671	0.6351
Avg RMSE	0.7488	1.0312	1.1280	0.6834	0.7967
Avg MAE	0.5471	0.8186	0.8918	0.4898	0.5959
Avg R^2	0.5696	0.1839	0.0238	0.6416	0.5127

Table 8.12: Average statistical metrics for Big Net Random Sampling (Part 1).

Pert. / Search	**0.05 / 0.2**	**0.1 / 0.05**	**0.1 / 0.1**	**0.1 / 0.2**
Metrics computed on Training Set:				
Avg Final Loss	1.2176	**0.4518**	0.5501	1.1347
Avg Time (s)	739.12	727.02	730.05	735.02
Metrics computed on Validation Set:				
Avg MSE	1.1942	**0.4656**	0.5557	1.1316
Avg RMSE	1.0927	**0.6823**	0.7453	1.0634
Avg MAE	0.8643	**0.4882**	0.5433	0.8360
Avg R^2	0.0837	**0.6428**	0.5737	0.1318

Table 8.13: Average statistical metrics for Big Net Random Sampling (Part 2).

8.4.4 Conclusion on Random Neighbors Generation Study

The grid search analysis conducted across Small, Medium, and Big Net architectures provides several key insights into the behavior of stochastic training within a discrete state-space search.

- **Architecture-Dependent Sensitivity:** The study demonstrates that the optimal hyperparameter configuration is strictly linked to the network's dimensionality. For the **Small Net**, a fine-grained approach (1% perturbation with 10% coverage) yielded the best results. However, as the parameter space expanded in the **Big Net**, the algorithm required more aggressive updates (5-10% perturbation) but a much narrower search breadth (5% coverage) to avoid premature RAM saturation.

- **The Optimal Hyperparameter Window:** Despite architectural variances, the most effective configurations consistently fell within a specific range: **Perturbation Ratios** between $[0.01, 0.1]$ and **Search Coverage Ratios** between $[0.05, 0.1]$. Deviating outside these bounds, particularly by increasing search coverage to 20%, consistently resulted in poor performance.

- **Stochastic vs. Deterministic Search:** Most significantly, the random sampling approach consistently outperformed the kernel-based methods tested in previous sections. While kernels provide a structured update, random sampling allows the A^* algorithm to explore a more diverse set of directions in the weight landscape. This stochasticity appears to be a **primary driver for performance**, enabling the model to achieve higher R^2 scores and lower MSE values across all tested architectures.

In conclusion, **Random Neighbors Generation** transforms the training process into a more effective stochastic optimization task. By balancing the intensity of updates with a controlled search breadth, this method provides a superior ability to navigate the complex loss surfaces of deep learning models compared to rigid, kernel-driven architectures.

8.5 Neighbors Generation Strategies Comparison

This section evaluates and compares the three proposed neighbor generation strategies (*Single Kernel*, *Layer-wise Kernels*, and *Random Neighbors Generation*) using the optimal hyperparameters identified in the previous tuning phases. To evaluate the raw performance of each generation logic without external enhancements, we omit all architectural and memory optimizations such as dynamic kernel reshaping, dynamic quantization factor scaling, and Beam Search memory management from these tests.

The goal is to determine which strategy provides the most effective exploration of the weight space and achieves the best performace on the California Housing benchmark.

8.5.1 Small Net Results

The Small Net architecture was tested using the following optimized configurations:

- **Single Kernel:** Weight kernel $[2, 2]$, Bias kernel $[2]$, Stride 2.
- **Layer-wise Kernels:**
  - Weight kernels: $[[2, 2], [2, 2], [1, 2]]$
  - Bias kernels: $[[2], [2], [1]]$
  - Weight strides: $[[2, 2], [2, 2], [2, 1]]$
  - Bias strides: $[[2], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.1.

Visual Analysis of Training and Stability

The training convergence comparison is illustrated in Figure 8.28.

While all strategies exhibit a similar initial descent, a significant gap emerges after the first 250 iterations.

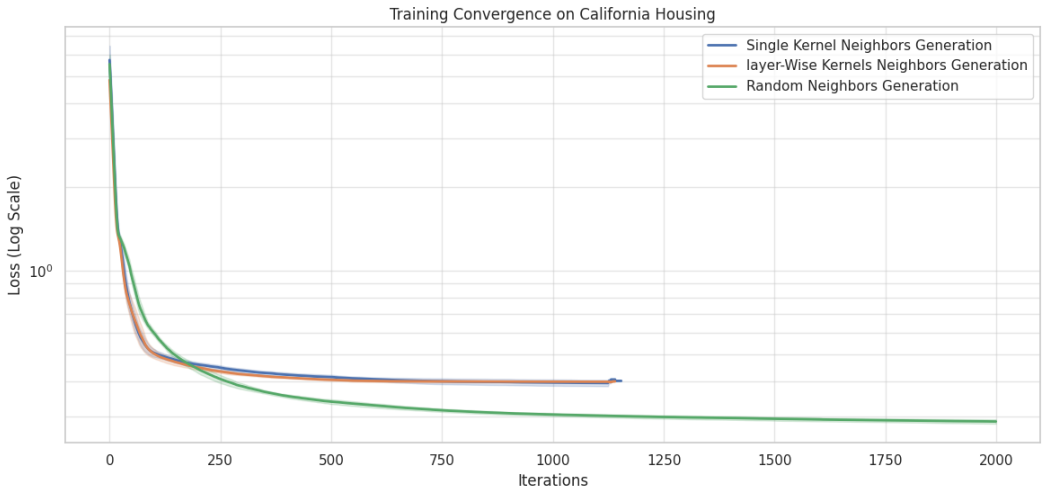

Figure 8.28: Training convergence comparison between Single Kernel, Layer-wise, and Random strategies (Log Scale).

The *Random Neighbors Generation* strategy (green line) exhibits a markedly superior convergence profile. Notably, the plot reveals a steeper slope in loss reduction compared to kernel-based methods, indicating more efficient optimization per iteration. While the kernel methods trigger earlier memory saturation and premature stopping, the stochastic approach maintains this downward trajectory for longer, ultimately achieving a significantly lower final training loss.

The distribution of the final loss values across 5 independent runs is shown in Figure 8.29.

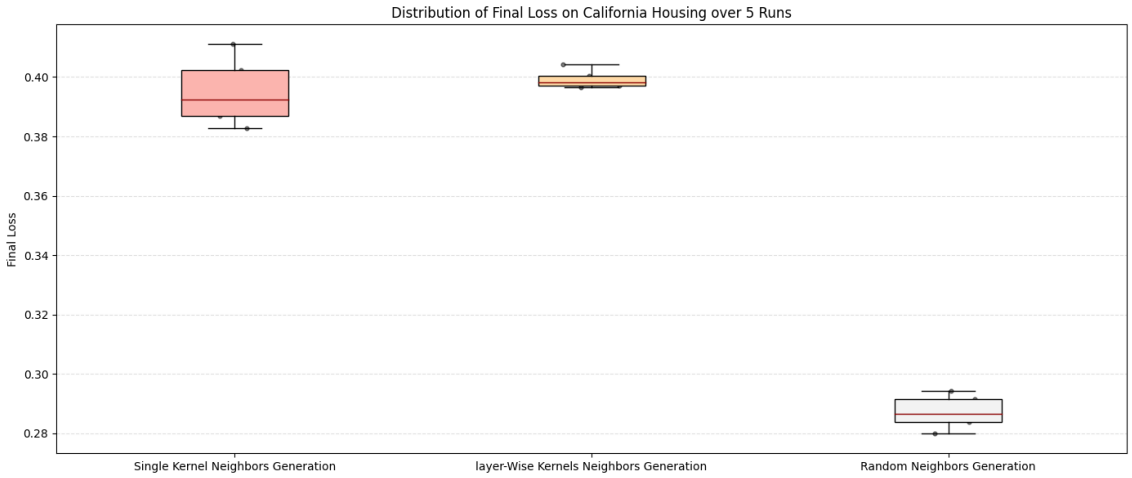

Figure 8.29: Distribution of Final Loss for the three generation strategies on Small Net.

The boxplot confirms that the Random strategy is not only more effective but also highly stable, maintaining a lower median loss (0.286) compared to the Single Kernel (0.392) and Layer-wise (0.398) approaches.

The regression metrics for the Small Net are visualized in Figure 8.30.

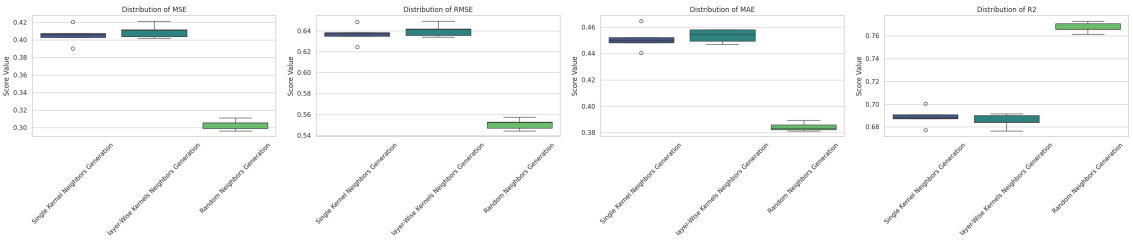

Figure 8.30: Regression metrics distribution (MSE, RMSE, MAE, R^2) across generation strategies.

The R^2 scores further validate the superiority of the Random approach, which achieves values centered around 0.77, whereas both kernel strategies cluster significantly lower, near 0.69.

Summary Statistics

The numerical performance summary in Table 8.14 highlights the clear advantage of stochastic sampling for this architecture.

Metric	Random Sampling	Single Kernel	Layer-wise Kernels
Metrics computed on Training Set:			
Avg Final Loss	**0.2872**	0.3951	0.3993
Avg Time (s)	**535.60**	777.80	753.39
Metrics computed on Validation Set:			
Avg MSE	**0.3035**	0.4055	0.4101
Avg RMSE	**0.5508**	0.6368	0.6404
Avg MAE	**0.3844**	0.4511	0.4534
Avg R^2	**0.7672**	0.6889	0.6853

Table 8.14: Average statistical comparison of generation strategies for Small Net.

8.5.2 Medium Net Results

The Medium Net architecture was evaluated using the optimal settings derived from the previous grid search. The configurations compared are:

- **Single Kernel:** Weight kernel $[4, 4]$, Bias kernel $[4]$, Stride 4.
- **Layer-wise Kernels:**
  - Weight kernels: $[[4, 4], [4, 4], [4, 4], [1, 4]]$
  - Bias kernels: $[[4], [4], [4], [1]]$
  - Weight strides: $[[4, 4], [4, 4], [4, 4], [4, 1]]$
  - Bias strides: $[[4], [4], [4], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

The convergence dynamics for the Medium Net, shown in Figure 8.31, reveal that the gap between stochastic and deterministic strategies widens as network depth increases.

Figure 8.31: Training convergence comparison for Medium Net (Log Scale).

The *Random Neighbors Generation* (green line) continues to refine the loss effectively, whereas both the *Single Kernel* and *Layer-wise* methods reach a performance

plateau relatively early, around iteration 400. This confirms that for larger architectures, the rigid geometric updates of kernels struggle to find effective descent directions in a more complex parameter space.

The stability of these results is analyzed through the distribution of the final loss in Figure 8.32.

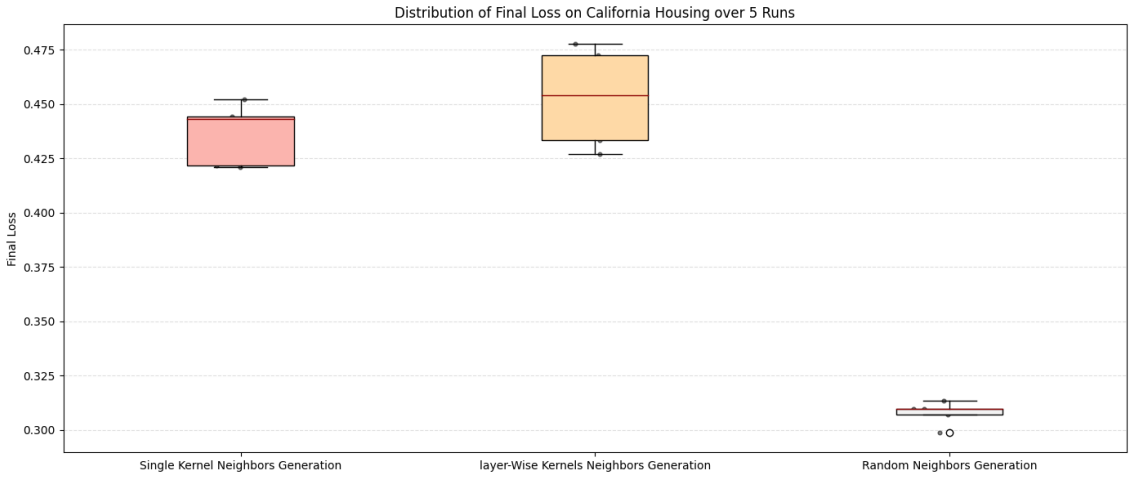

Figure 8.32: Distribution of Final Loss for Medium Net generation strategies.

The Random approach maintains a significantly lower and more consistent final loss (0.3076). In contrast, the *Layer-wise* method exhibits the highest average loss (0.4529), suggesting that the additional complexity of per-layer kernels does not translate to better optimization in this specific architecture.

The distribution of regression metrics for the Medium Net is summarized in Figure 8.33.

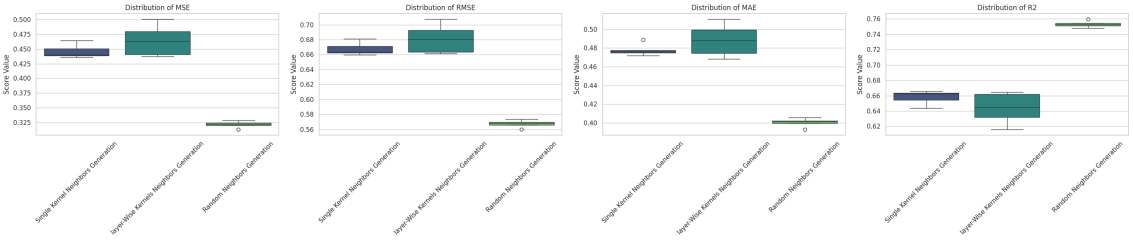

Figure 8.33: Regression metrics distribution across strategies for Medium Net.

The metrics highlight a superior predictive capability for the Random sampling strategy, achieving an average R^2 of 0.7531, compared to 0.6580 and 0.6438 for the

Single and Layer-wise kernel methods, respectively. This confirms that stochastic neighbors provide a more robust exploration of the error surface for deeper networks.

Summary Statistics

Table 8.15 provides the numerical performance breakdown for the Medium Net.

Metric	**Random Sampling**	**Single Kernel**	**Layer-wise Kernels**
Metrics computed on Training Set:			
Avg Final Loss	**0.3076**	0.4364	0.4529
Avg Time (s)	776.89	**727.95**	728.50
Metrics computed on Validation Set:			
Avg MSE	**0.3218**	0.4458	0.4643
Avg RMSE	**0.5672**	0.6676	0.6812
Avg MAE	**0.4002**	0.4780	0.4883
Avg R^2	**0.7531**	0.6580	0.6438

Table 8.15: Average statistical comparison of generation strategies for Medium Net.

8.5.3 Big Net Results

The Big Net architecture represents the most complex search space in this study. The three strategies were compared using the parameters found to be most effective for this scale:

- **Single Kernel:** Weight kernel $[6, 6]$, Bias kernel $[6]$, Stride 6.
- **Layer-wise Kernels:**
  - Weight kernels: $[[6, 2], [6, 6], [4, 4], [4, 4], [1, 2]]$
  - Bias kernels: $[[6], [6], [4], [2], [1]]$
  - Weight strides: $[[2, 6], [6, 6], [4, 4], [4, 4], [2, 1]]$
  - Bias strides: $[[6], [6], [4], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.1, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

The convergence patterns for the Big Net, illustrated in Figure 8.34, confirm the trends observed in smaller networks but with higher absolute loss values due to the increased dimensionality.

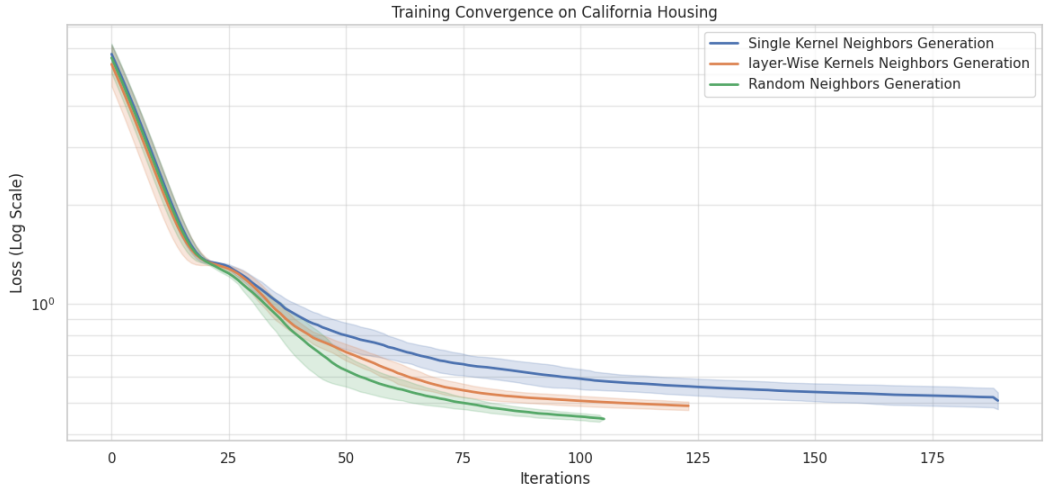

Figure 8.34: Training convergence comparison for Big Net (Log Scale).

The *Random Neighbors Generation* (green line) continues to be the most effective strategy, achieving a final loss significantly lower than its deterministic counterparts. In this high-parameter setting, the *Single Kernel* method (blue line) shows the most difficulty in refining weights, plateauting at a higher loss value compared to the *Layer-wise* approach.

The stability of these outcomes is summarized in the boxplot distribution in Figure 8.35.

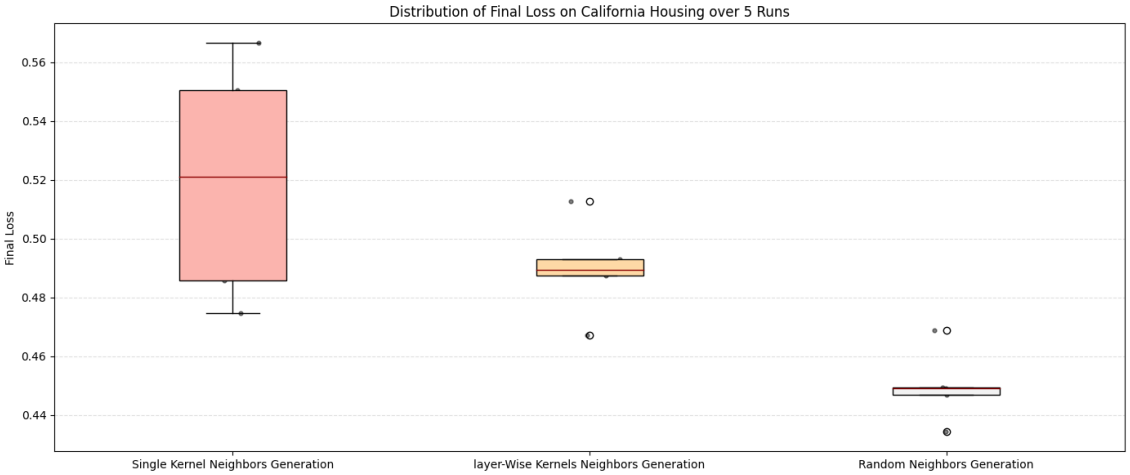

Figure 8.35: Distribution of Final Loss for Big Net generation strategies.

The Random strategy exhibits the lowest average loss (0.4497) and the lowest standard deviation (0.0110), indicating a high degree of reliability in high-dimensional spaces. While the *Layer-wise* method outperforms the *Single Kernel*, it still remains nearly 9% behind the Random strategy in terms of average final loss.

Figure 8.36 displays the regression metrics across the three strategies for the Big Net.

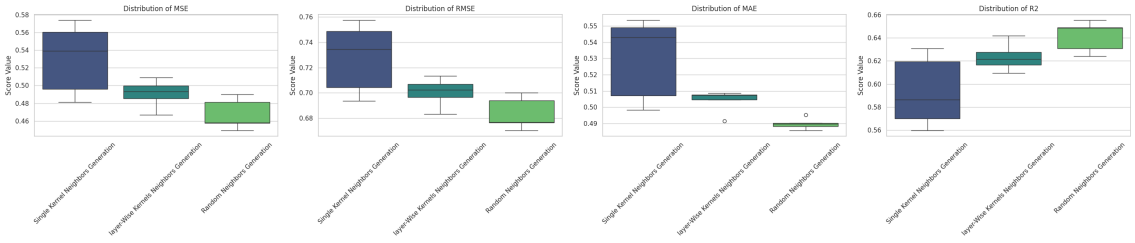

Figure 8.36: Regression metrics distribution across strategies for Big Net.

The R^2 scores demonstrate the scaling advantage of stochastic search: the Random strategy achieves an average R^2 of 0.6416, whereas the Single Kernel approach struggles to maintain predictive power, dropping below 0.60. This suggests that as network size increases, the flexibility of the neighbor generation becomes the primary factor in model performance.

Summary Statistics

Table 8.16 provides the final average numerical comparison for the Big Net architecture.

Metric	**Random Sampling**	**Single Kernel**	**Layer-wise Kernels**
Metrics computed on Training Set:			
Avg Final Loss	**0.4497**	0.5197	0.4899
Avg Time (s)	720.60	703.09	**702.34**
Metrics computed on Validation Set:			
Avg MSE	**0.4672**	0.5302	0.4908
Avg RMSE	**0.6834**	0.7277	0.7005
Avg MAE	**0.4898**	0.5302	0.5040
Avg R^2	**0.6416**	0.5932	0.6234

Table 8.16: Average statistical comparison of generation strategies for Big Net.

8.5.4 Conclusion on Neighbors Generation Strategies

The comparative analysis across all three network architectures provides conclusive evidence regarding the effectiveness of the proposed neighbor generation strategies.

- **Dominance of Random Sampling:** The results consistently demonstrate that the *Random Neighbors Generation* approach is the most effective strategy for weight optimization in an A^*-based framework. Across all architectures, it yielded significantly lower final loss and Mean Squared Error (MSE) compared to kernel-based methods. For instance, in the Small Net, the Random strategy achieved an MSE of 0.3035, nearly 25% lower than the best kernel-based result (0.4055).

- **Limitations of Structural Kernels:** The *Single Kernel* and *Layer-wise* approaches produced comparable results that were consistently inferior to stochastic sampling. This suggests that the rigid structural constraints imposed by kernels (weights and biases updated in fixed blocks) limit the search algorithm's ability to navigate the complex, non-linear error surfaces of neural networks. In contrast, the stochastic nature of random sampling provides the flexibility needed to find more efficient descent directions.

- **Scalability and Predictive Power:** The superiority of the Random approach is further validated by the R^2 scores, which remained robust even as network complexity increased. While kernel-based methods saw a notable drop in predictive performance when transitioning to larger networks, the Random strategy maintained high R^2 values (e.g., 0.7531 in Medium Net and 0.6416 in Big Net), highlighting its superior scalability.

In summary, the Random Neighbors Generation strategy is identified as the key driver for high-performance training in this study, primarily due to its superior convergence stability and predictive accuracy. However, it is **important to note** that the current performance of kernel-based methods is constrained by memory-saving measures; specifically, strides are currently set equal to kernel dimensions to delay hardware saturation. Reducing these strides to allow for kernel overlapping would enable a much finer level of refinement within the parameter matrices. While such overlapping is currently resource-prohibitive, it represents a promising path for future optimization. For now, the random sampling approach remains the most effective and recommended choice for achieving high-dimensional optimization under realistic memory constraints.

8.6 Beam Search Optimization on Single Kernel Approach

In this preliminary phase, we investigate the implementation of the **Beam Search** strategy as a potential solution to the critical limitation of main memory saturation. In a standard A^* search, the Open and Closed sets grow exponentially, often leading to RAM exhaustion before an optimal solution is reached.

Due to significant time and hardware constraints, this study focuses exclusively on the *Single Kernel* generation method. This choice allows us to analyze and isolate the impact of Beam Search before scaling to the other proposed methods. The experiment compares the performance of the classic A^* training against three different beam widths (100, $1,000$, and $10,000$) to evaluate whether this memory-efficient approach can maintain optimization quality while preventing hardware-related interruptions.

8.6.1 Small Net Results

For the Small Net architecture, the test was conducted to establish a baseline for how Beam Search impacts optimization when memory saturation is not yet a critical factor. The following parameters were applied:

- **Weight Kernel Size:** $[2, 2]$, **Bias Kernel Size:** $[2]$, **Strides:** 2.
- **Quantization Factor:** 10 (Static).
- **Beam Widths tested:** $100, 1,000, 10,000$.

Visual Analysis of Training and Stability

The convergence trajectories for the Small Net are shown in Figure 8.37.

Due to the low computational overhead of this architecture, memory constraints are not a factor, allowing all configurations to follow a uniform path toward convergence.

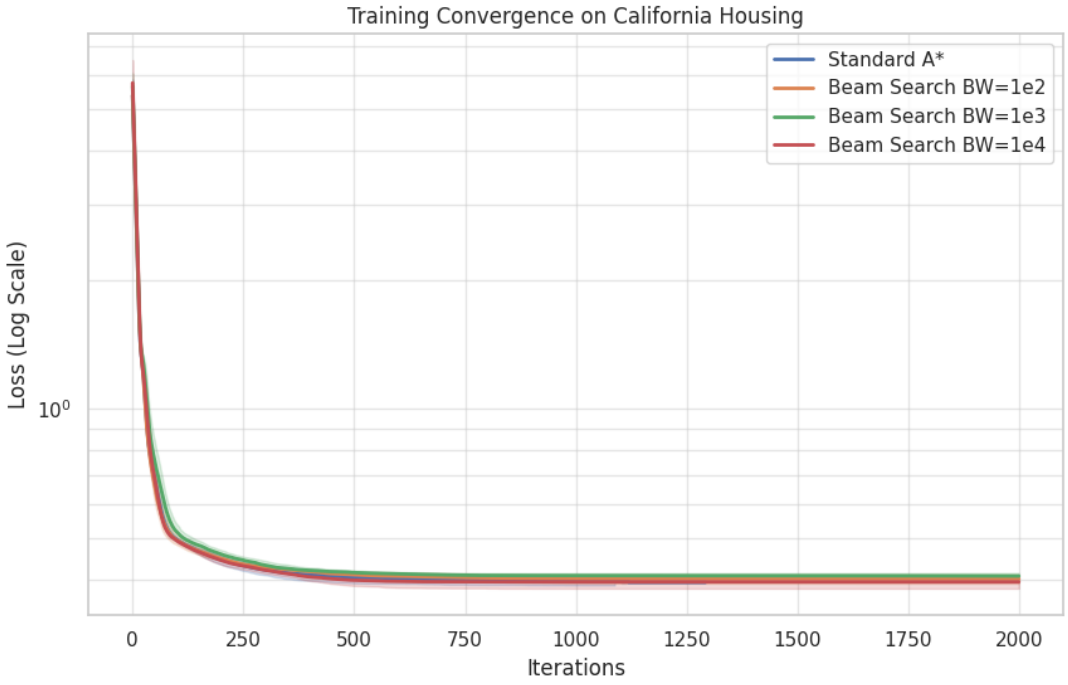

Figure 8.37: Training convergence comparison: Standard A* vs. various Beam Widths (Log Scale).

The *Standard A** and *Beam Search BW=10,000* show the most overlap, indicating that a sufficiently wide beam width can effectively approximate the full search space for smaller models.

Figure 8.38 displays the distribution of final loss values. It is observed that as the Beam Width increases, the final loss values tend to consolidate and improve.

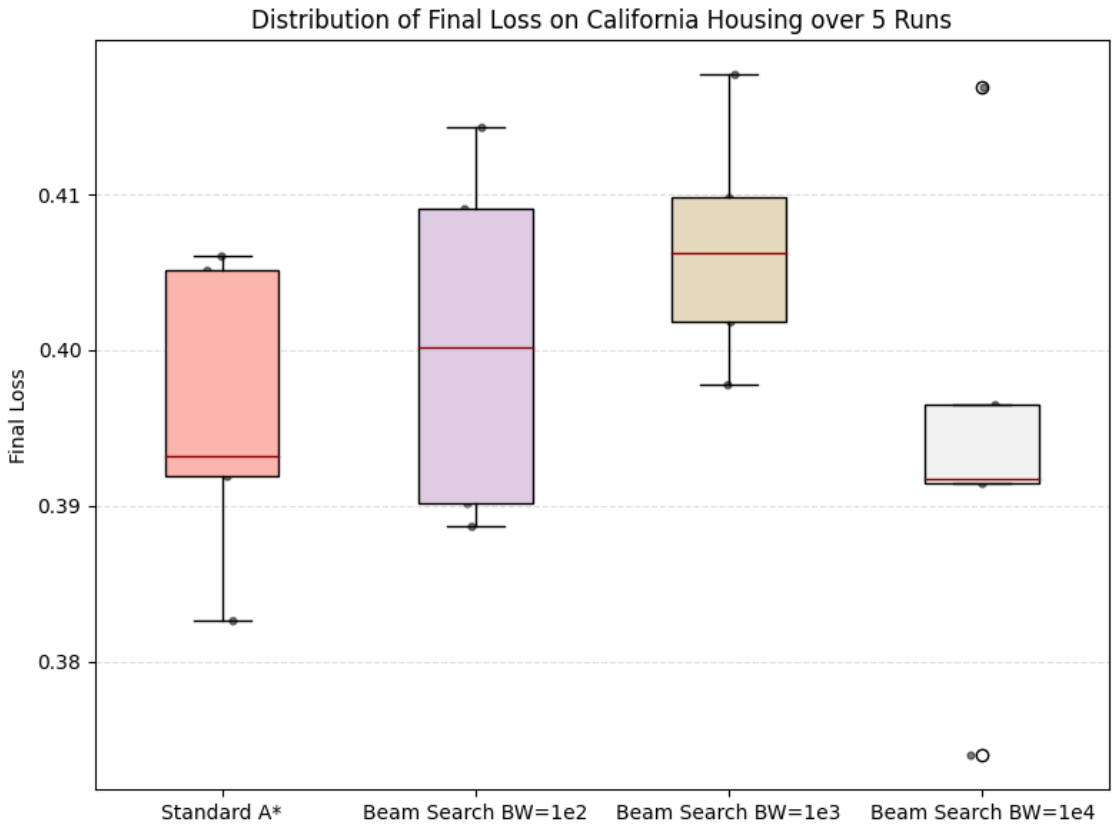

Figure 8.38: Distribution of Final Loss for Standard A* and Beam Search configurations.

Interestingly, the Standard A^* exhibits a wider variance in the Small Net compared to the $BW = 10,000$ configuration, suggesting that the pruning mechanism of Beam Search might occasionally filter out high-variance paths that Standard A^* would otherwise explore.

The regression metrics for the Small Net are summarized in Figure 8.39.

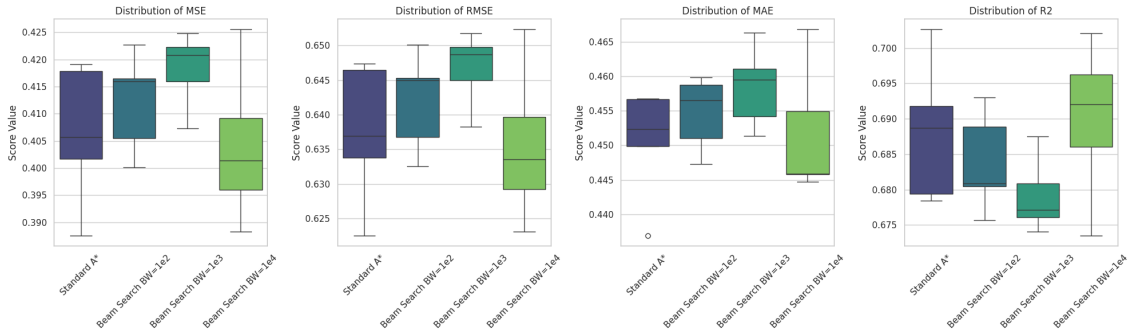

Figure 8.39: Regression metrics distribution across Beam Search configurations for Small Net.

The average R^2 values remain extremely stable across all tests, hovering around 0.68-0.69. This confirms that for architectures of this size, implementing Beam Search does not significantly compromise the model's ability to model the California Housing data.

Summary Statistics

Table 8.17 provides the numerical breakdown of the performance, highlighting the trade-off between search complexity and computational time.

Metric	**Standard A***	**BW=1e2**	**BW=1e3**	**BW=1e4**
Metrics computed on Training Set:				
Avg Final Loss	0.3958	0.4005	0.4067	**0.3941**
Avg Time (s)	**772.46**	1485.36	1514.21	1679.42
Metrics computed on Validation Set:				
Avg MSE	0.4064	0.4121	0.4182	**0.4041**
Avg RMSE	0.6374	0.6420	0.6467	**0.6356**
Avg MAE	**0.4505**	0.4547	0.4585	0.4516
Avg R^2	0.6882	0.6838	0.6791	**0.6900**

Table 8.17: Average statistical comparison: Standard A* vs. Beam Search for Small Net.

8.6.2 Medium Net Results

The evaluation of the Beam Search strategy was extended to the Medium Net architecture to observe the trade-offs between memory management and optimization efficiency in a higher-dimensional state space. The experimental parameters remained consistent with the Small Net tests:

- **Weight Kernel Size:** $[4, 4]$, **Bias Kernel Size:** $[4]$, **Strides:** 4.
- **Quantization Factor:** 10 (Static).
- **Beam Widths tested:** $100, 1,000, 10,000$.

Visual Analysis of Training and Stability

The convergence patterns for the Medium Net, illustrated in Figure 8.40, show a remarkable degree of overlap across all configurations.

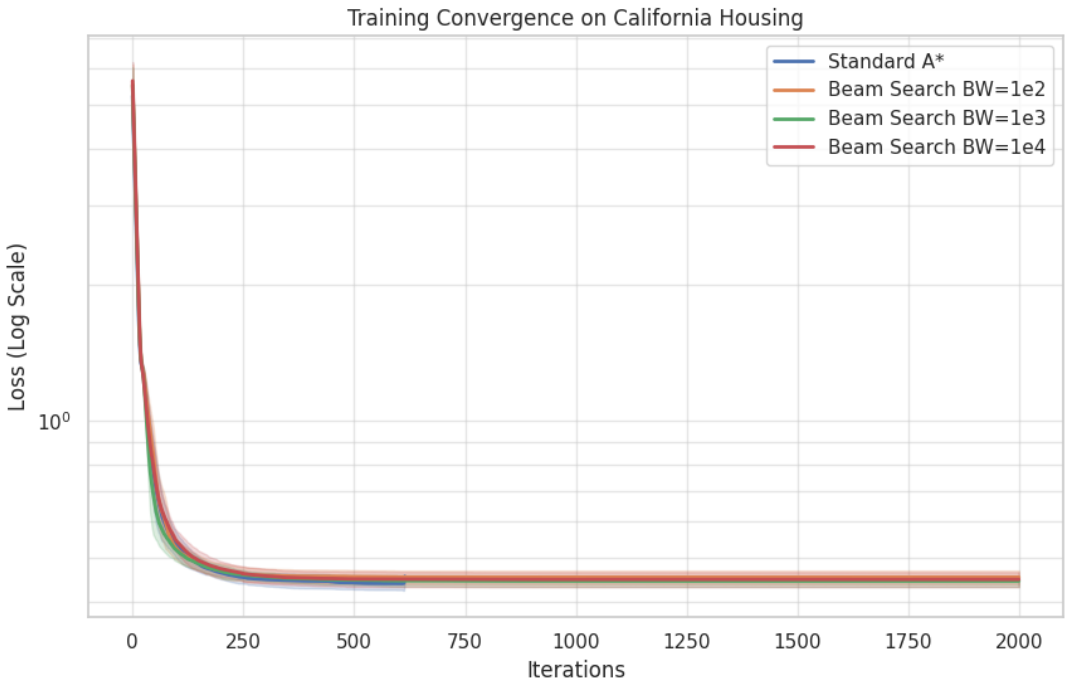

Figure 8.40: Training convergence comparison for Medium Net: Standard A* vs. various Beam Widths (Log Scale).

Even with the increased complexity of the Medium Net, the pruning mechanism of the Beam Search does not prevent the algorithm from following a convergence trajectory nearly identical to the Standard A^* search. This indicates that the most promising descent directions are correctly preserved within the beam, even at a width of 100.

Figure 8.41 displays the distribution of final loss values.

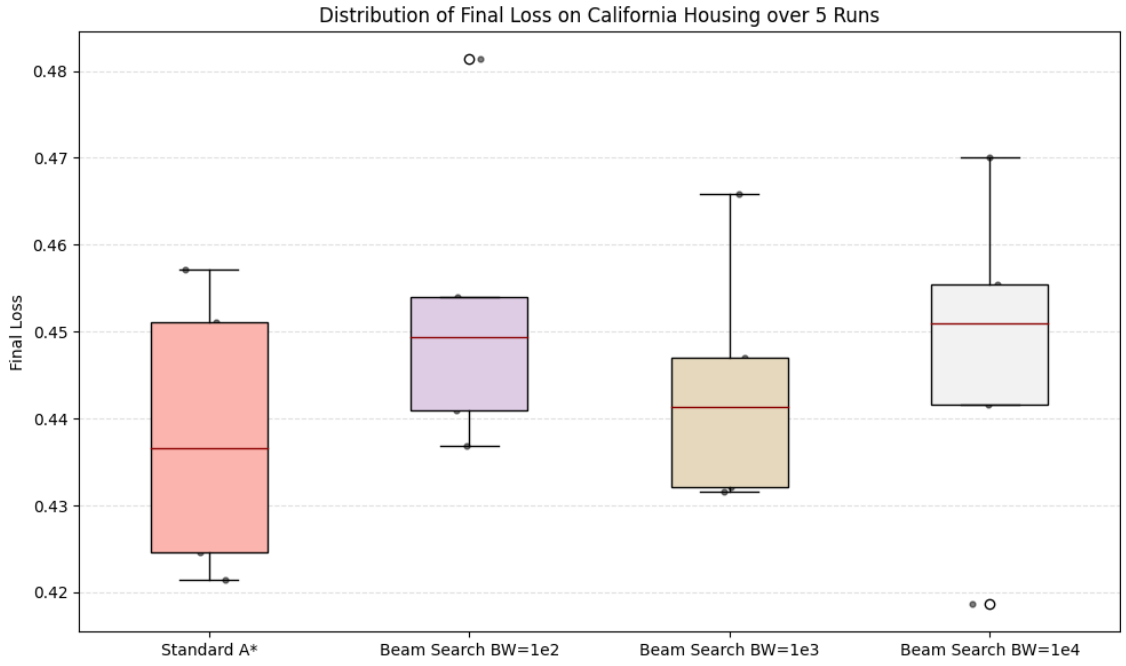

Figure 8.41: Distribution of Final Loss for Medium Net across Beam Search configurations.

While the Average Final Loss for $BW = 100$ (0.4525) is slightly higher than the Standard A^* (0.4382), the difference is marginal, confirming that Beam Search is a viable alternative for larger networks where memory limitations become a bottleneck.

The predictive accuracy metrics for the Medium Net are visualized in Figure 8.42.

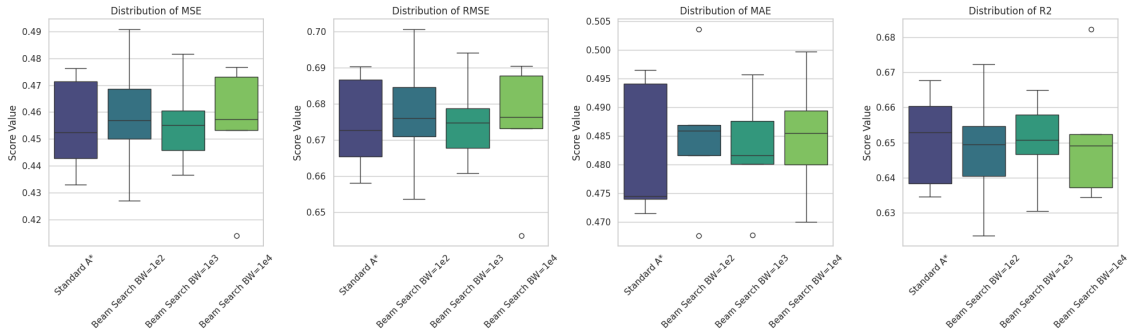

Figure 8.42: Regression metrics distribution across Beam Search configurations for Medium Net.

The metrics confirm that optimization quality is preserved despite the state-space pruning. The average R^2 values remain remarkably consistent, with the $BW = 10,000$ configuration achieving an average R^2 of 0.6510, virtually identical to the 0.6508 achieved by the Standard A^* search.

Summary Statistics

Table 8.18 provides the numerical performance summary, highlighting the significant time disparity between the classic search and the beam-limited search.

Metric	**Standard A***	**BW=1e2**	**BW=1e3**	**BW=1e4**
Metrics computed on Training Set:				
Avg Final Loss	**0.4382**	0.4525	0.4436	0.4473
Avg Time (s)	**730.16**	2682.06	2888.02	3332.55
Metrics computed on Validation Set:				
Avg MSE	0.4552	0.4587	0.4560	**0.4549**
Avg RMSE	0.6746	0.6771	0.6752	**0.6742**
Avg MAE	**0.4822**	0.4851	0.4826	0.4849
Avg R^2	0.6508	0.6481	0.6502	**0.6510**

Table 8.18: Average statistical comparison: Standard A* vs. Beam Search for Medium Net.

8.6.3 Big Net Results

The evaluation of the Beam Search strategy on the Big Net architecture represents the most significant test case for memory management. With a considerably larger state space, the Standard A^* algorithm is prone to premature termination due to RAM saturation. The experimental parameters were configured as follows:

- **Weight Kernel Size:** $[8, 8]$, **Bias Kernel Size:** $[8]$, **Strides:** 8.
- **Quantization Factor:** 10 (Static).
- **Beam Widths tested:** $100, 1,000, 10,000$.

Visual Analysis of Training and Stability

The convergence trajectories for the Big Net are illustrated in Figure 8.43. In this complex architecture, the trajectories remain tightly clustered, indicating that the pruning of the search space does not adversely affect the primary descent path.

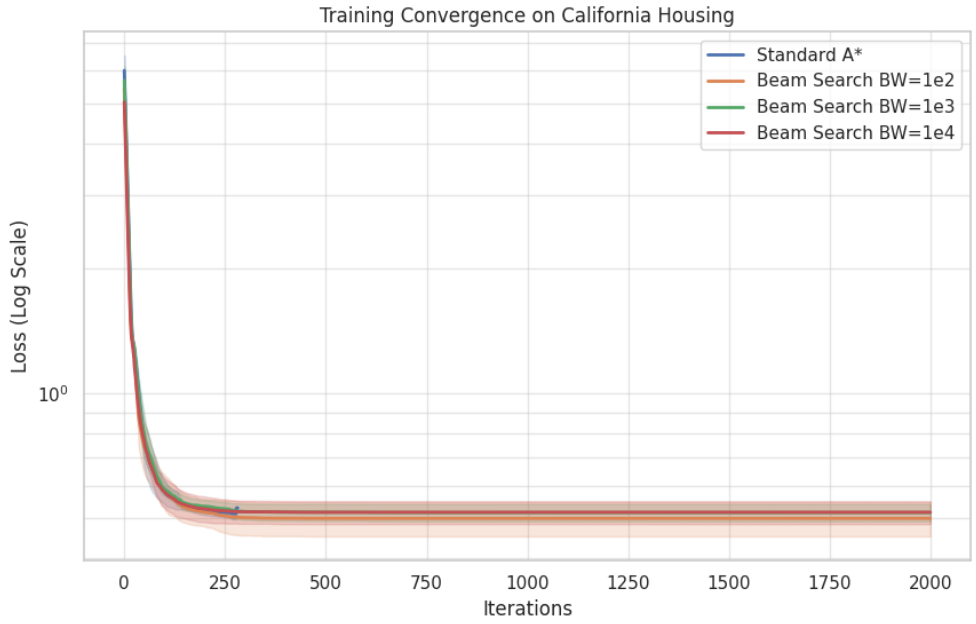

Figure 8.43: Training convergence comparison for Big Net: Standard A* vs. various Beam Widths (Log Scale).

A critical observation is found in the execution time. The *Standard A** completes its execution in significantly less time ($\approx$ 691s) compared to the Beam Search variants ($\approx$ 5300-6200s). This disparity is due to the vanilla method reaching hardware memory limits rapidly and halting, whereas Beam Search manages the sets to allow for a more memory-efficient optimization process.

The distribution of final loss values is shown in Figure 8.44.

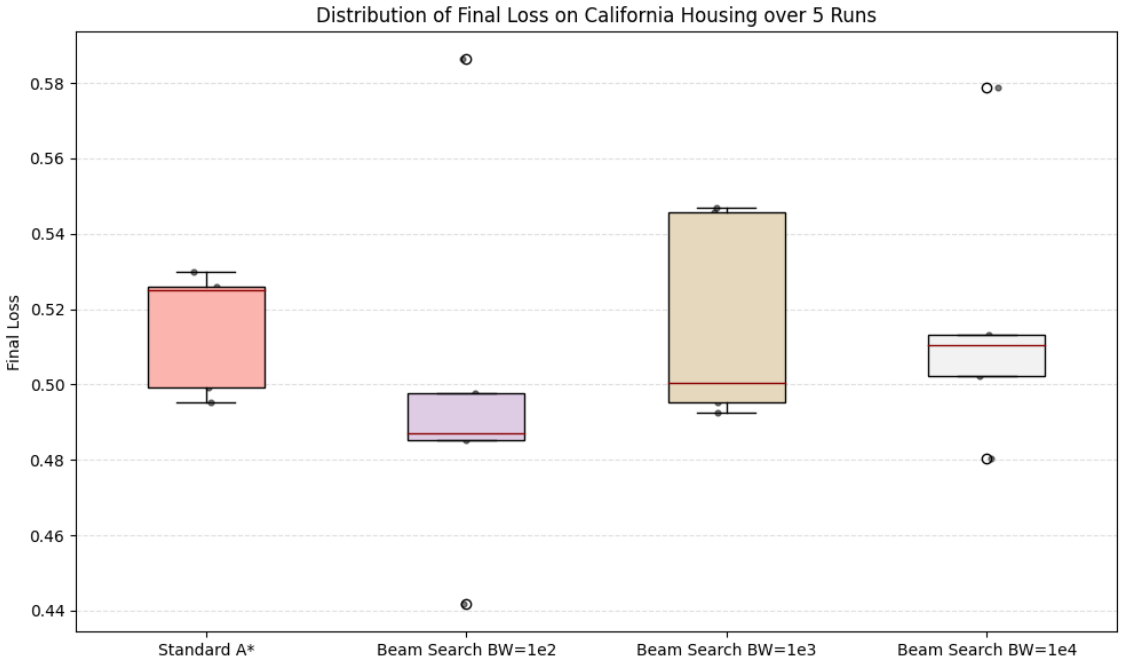

Figure 8.44: Distribution of Final Loss for Big Net across Beam Search configurations.

Counter-intuitively, the **Beam Search with BW=100** achieved the lowest average final loss (0.4996), even lower than the Standard A^* (0.5150). This suggests that for very large networks, aggressive pruning may act as a helpful constraint, preventing the algorithm from exploring high-cost branches that do not lead to a global minimum.

The regression metrics for the Big Net are visualized in Figure 8.45.

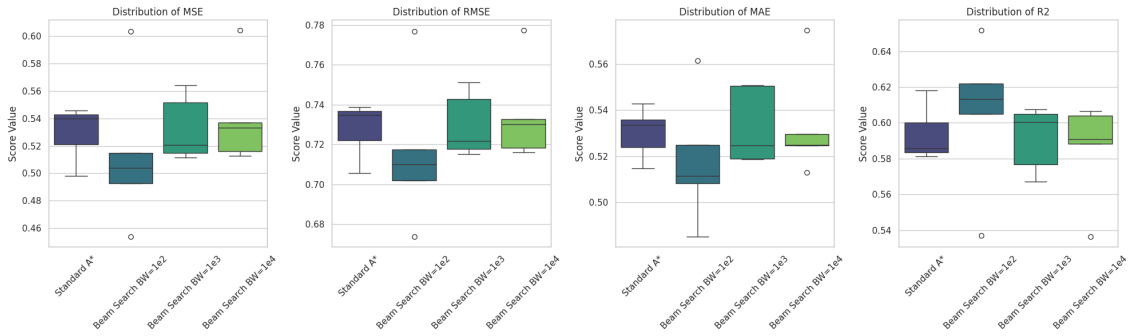

Figure 8.45: Regression metrics distribution across Beam Search configurations for Big Net.

The metrics confirm that the predictive quality remains consistent across different beam widths. The R^2 scores are centered around 0.59-0.60. Remarkably, $BW = 100$ yielded the highest average R^2 (0.6058), confirming that limiting the search set size is not only beneficial for memory management but can also improve general performance in high-dimensional spaces.

Summary Statistics

Table 8.19 provides the numerical performance summary for the Big Net.

Metric	**Standard A***	**BW=1e2**	**BW=1e3**	**BW=1e4**
Metrics computed on Training Set:				
Avg Final Loss	0.5150	**0.4996**	0.5161	0.5170
Avg Time (s)	**691.68**	5369.14	5681.63	6267.84
Metrics computed on Validation Set:				
Avg MSE	0.5295	**0.5137**	0.5326	0.5406
Avg RMSE	0.7276	**0.7160**	0.7297	0.7349
Avg MAE	0.5302	**0.5182**	0.5327	0.5333
Avg R^2	0.5937	**0.6058**	0.5914	0.5852

Table 8.19: Average statistical comparison: Standard A* vs. Beam Search for Big Net.

8.6.4 Conclusion on Beam Search Optimization Study

The comparative analysis between the Standard A^* algorithm and the Beam Search strategy provides critical insights into resource management and optimization persistence for neural network training.

- **Resolution of Memory Constraints:** The primary and most significant finding is that Beam Search effectively resolves the hardware limitations associated with Standard A^*. In every architecture tested, the Standard A^* search failed to reach the maximum limit of 2,000 iterations, as the exponential growth of the Open and Closed sets led to complete RAM saturation. In contrast, all Beam Search configurations successfully completed the full 2,000 iteration cycle, confirming its necessity for stable, long-term optimization.

- **Consistent Performance Gains:** Across all tested networks, the Beam Search strategy consistently outperformed the vanilla training method, albeit by a marginal lead. The most notable improvement was observed in the **Big Net**, where the most aggressive pruning (BW=100) achieved the highest R^2 score (0.6058) and the lowest average loss (0.4996). This suggests that limiting the search breadth may act as a regularizer, forcing the algorithm to prioritize the most promising paths identified by the heuristic function.

- **Efficiency of Narrow Beam Widths:** The data indicates that a wide beam is not always necessary for high-quality results. Even with a narrow Beam Width of 100, the algorithm maintained a predictive accuracy comparable to or better than the full search. This highlights the effectiveness of the underlying heuristic in correctly ordering neighbors, allowing for the disposal of the vast majority of search nodes without losing the optimal descent direction.

In conclusion, the **Beam Search** strategy is a fundamental optimization for the A^*-based training framework. By transforming an unbounded search into a resource-constrained process, it allows for the training of larger architectures that would otherwise be impossible to optimize using traditional search methods.

8.7 Neighbors Generation Strategies Comparison with Beam Search

In this concluding phase of the preliminary study, we evaluate the three proposed neighbor generation strategies under the activation of **Beam Search** optimization. This final test aims to determine whether the synergy between memory-constrained search and these generation methods yields superior convergence profiles on the California Housing dataset.

8.7.1 Small Net Results

The Small Net architecture was evaluated using the following configurations with Beam Search active:

- **Single Kernel:** Weight kernel $[2, 2]$, Bias kernel $[2]$, Stride 2.
- **Layer-wise Kernels:**
  - Weight kernels: $[[2, 2], [2, 2], [1, 2]]$
  - Bias kernels: $[[2], [2], [1]]$
  - Weight strides: $[[2, 2], [2, 2], [2, 1]]$
  - Bias strides: $[[2], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.1.

Visual Analysis of Training and Stability

The training convergence for the Small Net with Beam Search is illustrated in Figure 8.46.

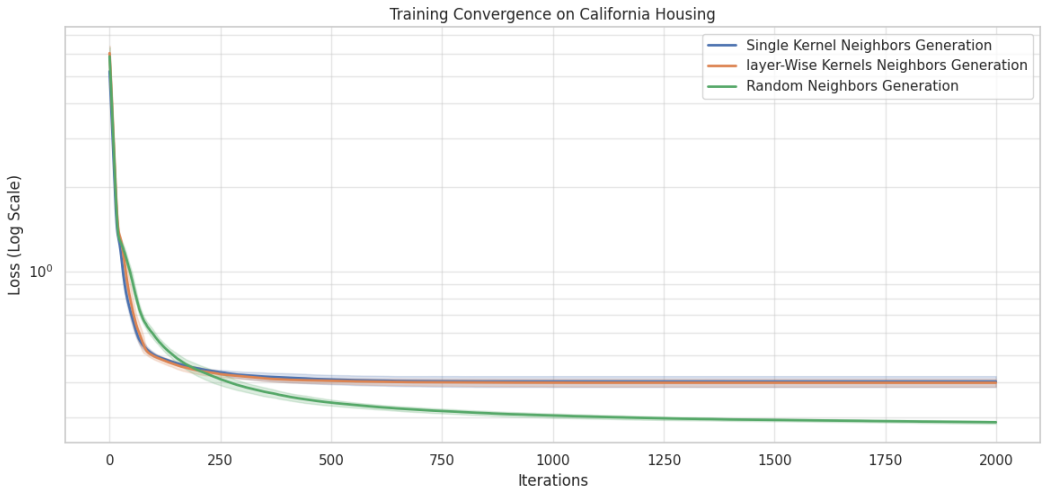

Figure 8.46: Training convergence comparison for Small Net with Beam Search (Log Scale).

The activation of Beam Search preserves the performance hierarchy established in previous experiments. The *Random Neighbors Generation* strategy (green line) maintains its superiority, consistently identifying higher-quality descent directions and reaching a final loss significantly lower than its deterministic counterparts. In contrast, both kernel strategies reach a similar plateau early in the training process. This suggests that their rigid geometric updates lack the necessary granularity for further refinement, a limitation likely caused by large strides that precluded sliding window overlapping.

The stability of these outcomes is summarized in the boxplot distribution in Figure 8.47.

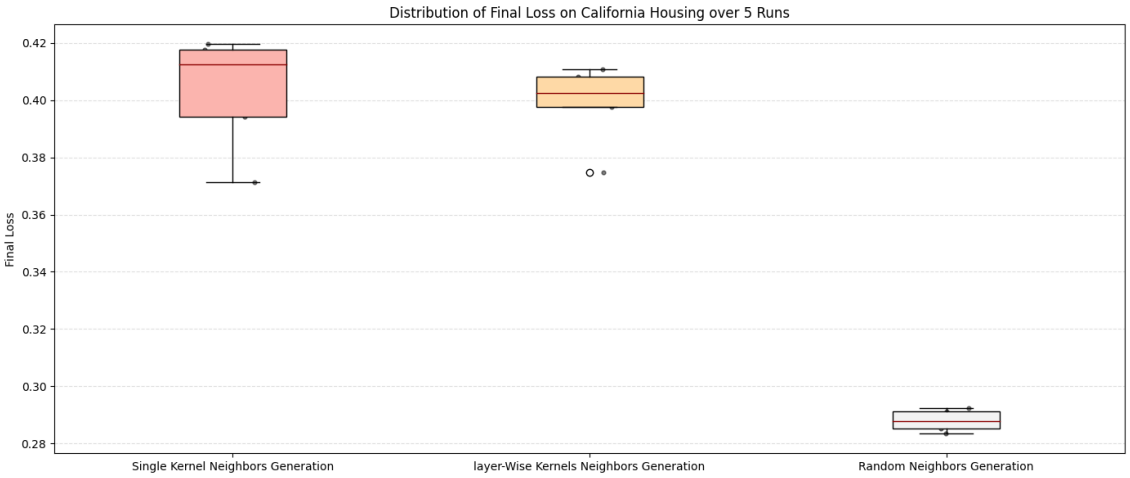

Figure 8.47: Distribution of Final Loss with Beam Search for Small Net.

The *Random Neighbors Generation* strategy maintains the lowest average final loss (0.2880) and the most stable distribution. In contrast, the *Single Kernel* approach exhibits higher variance and the highest average final loss (0.4031), suggesting that without per-layer customization, the beam search often explores less productive paths compared to the stochastic alternative.

The regression metrics for the Small Net with Beam Search are summarized in Figure 8.48.

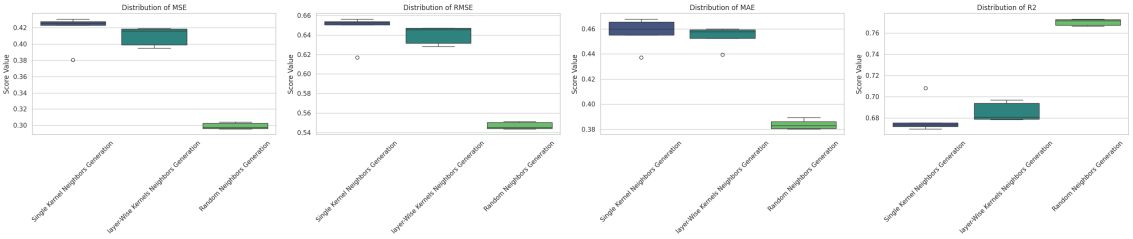

Figure 8.48: Regression metrics distribution across strategies for Small Net with Beam Search.

The R^2 values highlight a clear performance gap: the Random strategy achieves an average score of 0.7704, outperforming the Single Kernel (0.6797) and Layer-wise (0.6858) methods by nearly 10 percentage points. This confirms that the pruning logic of Beam Search is highly effective when paired with stochastic sampling, as it focuses computational resources on the most promising random perturbations.

Summary Statistics

Table 8.20 provides the numerical performance breakdown for the Small Net.

Metric	Random Sampling	Single Kernel	Layer-wise Kernels
Metrics computed on Training Set:			
Avg Final Loss	**0.2880**	0.4031	0.3988
Avg Time (s)	**624.23**	1430.77	1308.16
Metrics computed on Validation Set:			
Avg MSE	**0.2992**	0.4175	0.4095
Avg RMSE	**0.5470**	0.6460	0.6399
Avg MAE	**0.3839**	0.4571	0.4537
Avg R^2	**0.7704**	0.6797	0.6858

Table 8.20: Average statistical comparison of generation strategies with Beam Search for Small Net.

8.7.2 Medium Net Results

The Medium Net architecture was analyzed by activating the Beam Search and utilizing the optimal configurations identified in previous tests. For this experiment, the structural parameters were defined as follows:

- **Single Kernel:** Weight kernel $[4, 4]$, Bias kernel $[4]$, Stride 4.
- **Layer-wise Kernels:**
  - Weight kernels: $[[4, 4], [4, 4], [4, 4], [1, 4]]$
  - Bias kernels: $[[4], [4], [4], [1]]$
  - Weight strides: $[[4, 4], [4, 4], [4, 4], [4, 1]]$
  - Bias strides: $[[4], [4], [4], [1]]$
- **Random Sampling:** Perturbation ratio 0.01, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

Figure 8.49 illustrates the convergence comparison for the three strategies applied to the Medium Net with the support of Beam Search.

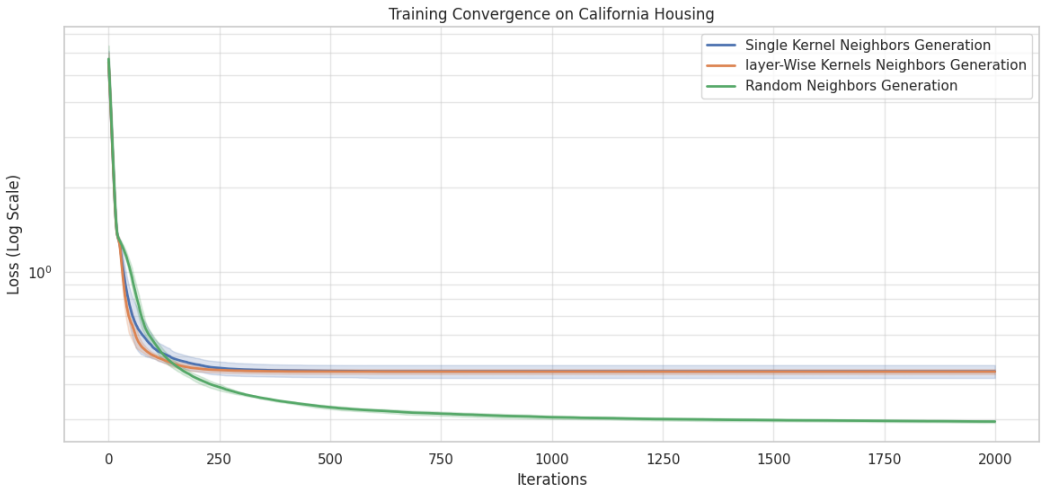

Figure 8.49: Convergence comparison for Medium Net with Beam Search (Log Scale).

Similar to the observations in the Small Net, the *Random Neighbors Generation* strategy (green line) maintains a clear competitive advantage, stabilizing at a significantly lower loss level compared to kernel-based methods. The *Single Kernel* and *Layer-wise* strategies exhibit almost identical behavior, rapidly reaching a plateau that they are unable to overcome despite the memory optimization.

The final loss distribution is visualized in the boxplot in Figure 8.50.

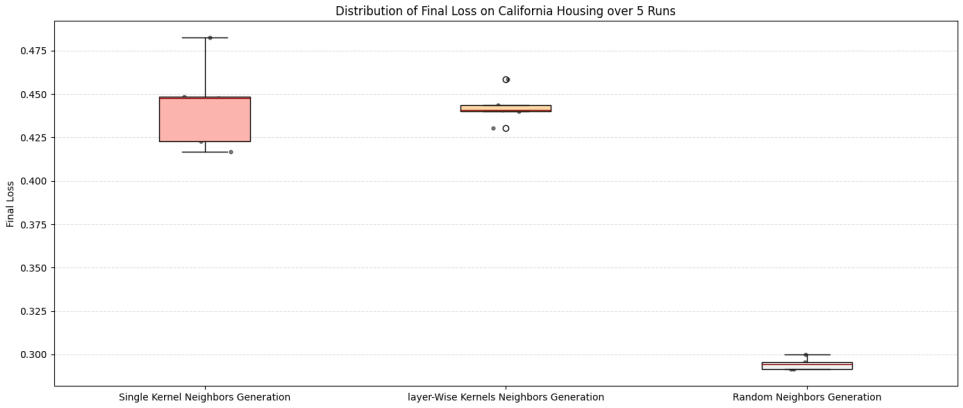

Figure 8.50: Final Loss distribution with Beam Search for Medium Net.

The Random method confirms its robustness with an average loss of 0.2945 and an extremely low variance (0.000010). Interestingly, Beam Search helps maintain the results of the *Single Kernel* (0.4437) and *Layer-wise* (0.4428) methods very close to each other, although both remain far from the performance achieved by stochastic sampling.

Metrics Distribution and Predictive Performance

The regression metrics for the Medium Net with Beam Search are summarized in Figure 8.51.

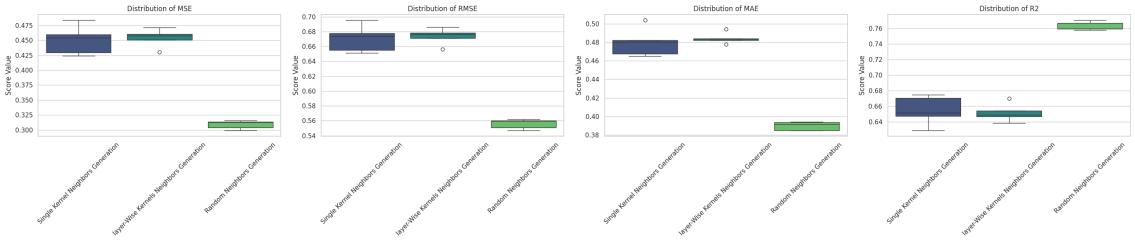

Figure 8.51: Distribution of regression metrics with Beam Search for Medium Net.

The average R^2 value for the Random method stands at 0.7629, an excellent result that exceeds the deterministic alternatives ($R^2 \approx 0.65$) by over 10 percentage points. This confirms that random sampling, even with a coverage reduced to 5%, is significantly more effective at capturing the complexity of the dataset on medium-sized networks.

Summary Statistics

Table 8.21 provides the detailed statistical summary for the Medium Net.

Metric	**Random Sampling**	**Single Kernel**	**Layer-wise Kernels**
Metrics computed on Training Set:			
Avg Final Loss	**0.2945**	0.4437	0.4428
Avg Time (s)	**2302.08**	2883.00	2915.42
Metrics computed on Validation Set:			
Avg MSE	**0.3091**	0.4502	0.4540
Avg RMSE	**0.5559**	0.6707	0.6737
Avg MAE	**0.3897**	0.4799	0.4845
Avg R^2	**0.7629**	0.6546	0.6517

Table 8.21: Average statistical comparison of strategies with Beam Search for Medium Net.

8.7.3 Big Net Results

The Big Net architecture represents the most complex scenario of this study. For this test, the strategies were configured with parameters optimized for high dimensionality, while keeping the Beam Search active:

- **Single Kernel:** Weight kernel $[6, 6]$, Bias kernel $[6]$, Stride 6.
- **Layer-wise Kernels:**
  - Weight kernels: $[[6, 2], [6, 6], [4, 4], [4, 4], [1, 2]]$
  - Bias kernels: $[[6], [6], [4], [2], [1]]$
  - Weight strides: $[[2, 6], [6, 6], [4, 4], [4, 4], [2, 1]]$
  - Bias strides: $[[6], [6], [4], [2], [1]]$
- **Random Sampling:** Perturbation ratio 0.1, Search coverage ratio 0.05.

Visual Analysis of Training and Stability

Figure 8.52 displays the learning curves for the Big Net.

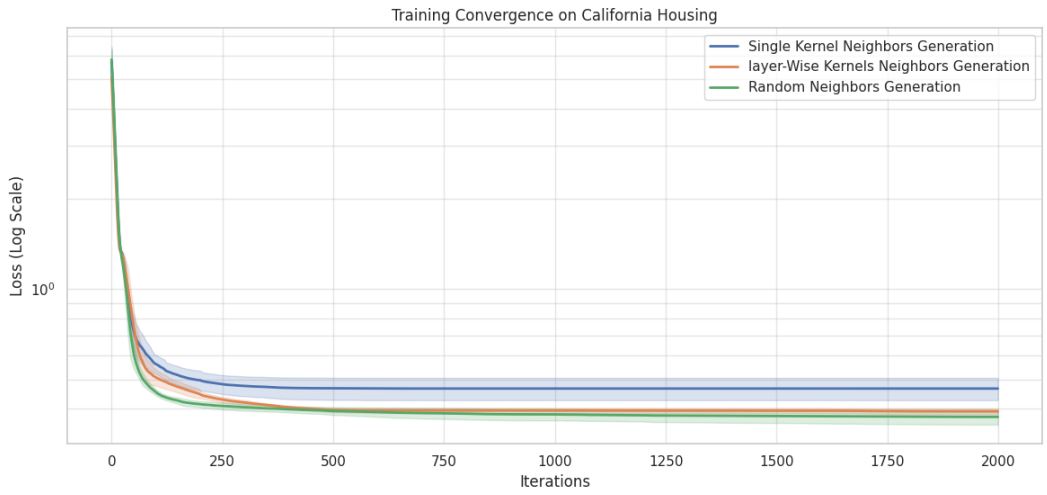

Figure 8.52: Convergence comparison for Big Net with Beam Search (Log Scale).

Despite the extreme complexity of the search space, the *Random Neighbors Generation* strategy (green line) remains the most effective, reaching the lowest final

loss. It is noteworthy that the *Layer-wise* method curve (orange line) consistently positions itself below the *Single Kernel* curve, suggesting that for networks of this scale, the flexibility of having specific kernels for each layer allows for a more accurate descent compared to a single-kernel approach.

The stability of the performance is analyzed in the boxplot in Figure 8.53.

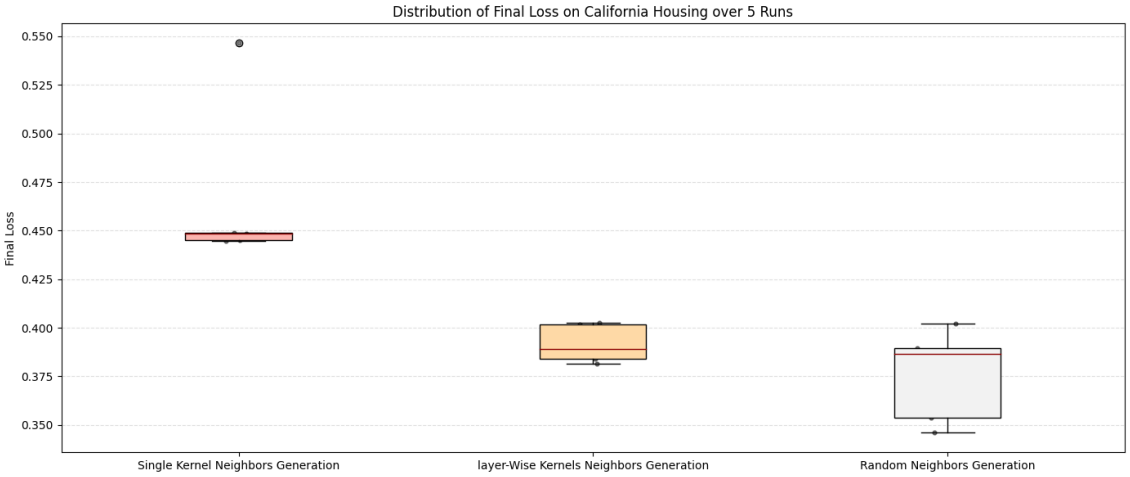

Figure 8.53: Final Loss distribution with Beam Search for Big Net.

The Random method achieves the lowest mean loss (0.3756), but exhibits higher variance compared to the *Layer-wise* method (0.3918), which proves to be extremely consistent (Std: 0.0089). The *Single Kernel* method finishes with the least favorable performance and the highest variance, confirming the difficulty of static kernels that are not layer-optimized when handling such a large number of parameters.

Metrics Distribution and Predictive Performance

Figure 8.54 illustrates the distribution of regression metrics for the Big Net.

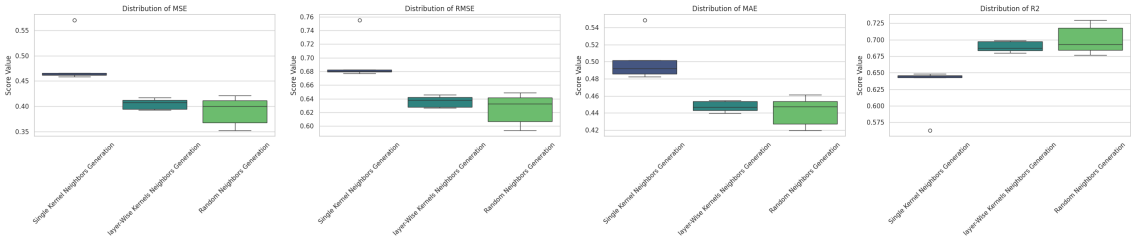

Figure 8.54: Distribution of regression metrics with Beam Search for Big Net.

The R^2 results show that the Random method reaches an average peak of 0.7003, closely followed by the *Layer-wise* method at 0.6893. The gap between the strategies narrows compared to smaller networks, indicating that the pruning operated by the Beam Search and the correct configuration of per-layer kernels make deterministic methods significantly more competitive on a large scale.

Summary Statistics

The average numerical data for the Big Net with Beam Search are reported in Table 8.22.

Metric	**Random Sampling**	**Single Kernel**	**Layer-wise Kernels**
Metrics computed on Training Set:			
Avg Final Loss	**0.3756**	0.4668	0.3918
Avg Time (s)	15623.76	**8694.52**	12999.59
Metrics computed on Validation Set:			
Avg MSE	**0.3906**	0.4837	0.4049
Avg RMSE	**0.6247**	0.6949	0.6363
Avg MAE	**0.4418**	0.5023	0.4475
Avg R^2	**0.7003**	0.6289	0.6893

Table 8.22: Average statistical comparison of strategies with Beam Search for Big Net.

8.7.4 Conclusion on Generation Strategies Comparison with Beam Search

The final phase of the preliminary study, comparing neighbor generation strategies with Beam Search, confirms the relative effectiveness of each approach when operating under strict memory constraints. The results lead to several critical observations regarding the synergy between search space exploration and state-space pruning.

- **Consistent Superiority of Random Neighbors Generation:** Across all architectures, the *Random Neighbors Generation* strategy remains the most effective method for weight optimization. Even with Beam Search active, it consistently achieves significantly lower average final loss and Mean Squared Error (MSE) compared to the deterministic kernel-based strategies. Specifically, the Random approach maintained an R^2 above 0.70 even in the complex Big Net architecture, whereas the Single Kernel approach fell to 0.6289.

- **Synergy between Stochasticity and Pruning:** The pairing of random sampling with Beam Search demonstrates a powerful optimization synergy. While the Random strategy generates a diverse set of stochastic neighbors, the Beam Search's pruning logic effectively identifies and preserves the most promising updates. This is evidenced by the Small Net results, where the Random strategy achieved its highest R^2 score (0.7704) while completing training in less than half the time required by the Single Kernel method (624s vs 1430s).

- **Scalability of Layer-wise Customization:** A significant trend emerged in the **Big Net** results. While the *Layer-wise Kernels* strategy was comparable to the *Single Kernel* method in smaller architectures, it showed a clear advantage as the parameter space expanded. In the Big Net, the Layer-wise strategy achieved a final loss of 0.3918 compared to the 0.4668 of the Single Kernel. This confirms that as network depth increases, providing specific kernels and strides for each layer allows the search algorithm to navigate the high-dimensional error surface more accurately.

- **Computational Efficiency and Time-to-Convergence:** Stochastic neighbors not only lead to higher accuracy but also to significantly lower computational times in most scenarios. In the Medium and Small Nets, the Random strategy was faster than both kernel-based methods, suggesting that finding high-quality descent directions early in the search allows the algorithm to progress more efficiently through its iteration cycle.

In summary, the combination of Random Neighbors Generation and Beam Search represents the optimal configuration for training neural networks via the A^* framework under current settings. While the *Layer-wise* strategy provides a deterministic alternative that scales more effectively than the *Single Kernel* approach for large models, it remains limited by its rigid update structure. However, the *Layer-wise* method shows promise for further optimization: reducing stride sizes to allow for kernel overlapping, combined with more granular tuning of kernel shapes, could potentially bridge the performance gap. Currently, the stochastic flexibility of random sampling provides a consistently superior mapping of the loss landscape, ensuring higher predictive performance and more stable convergence.

Chapter 9

Comparative Performance: A^* Search vs. Adam Optimizer

This chapter presents a comprehensive evaluation between the proposed A^* MLP optimization framework and the standard gradient-based Adam optimizer. To ensure a fair and rigorous comparison, we utilize the optimal hyperparameters identified in the previous tuning phases. Benchmarking search-based performance against industry-standard optimizers allows for a rigorous assessment of heuristic search's scalability and efficiency within the weight space.

9.1 Experimental Setup and Hyperparameters

To ensure statistical rigor and consistency across datasets, all experiments utilize a standardized foundational configuration. Every test is executed using **full-batch training** across all methods; furthermore, search-based strategies employ **beam search** to ensure they consistently reach the 1,000-iteration threshold.

9.1.1 Common Parameters

The following settings were applied universally across all models and datasets:

- **Iterations:** 1000
- **Independent Runs:** 5 (results are averaged to ensure stability)
- **Parameter Range:** $[-10, 10]$

- **Quantization Factor:** 10
- **Beam Width:** 1×10^3
- **Adam Learning Rate:** 0.001.

9.1.2 Architectural Configurations

For each network size, we employ the best-performing parameters found in previous experiments. Notably, for both kernel-based methods (*Single* and *Layer-wise*), the strides were reduced by one relative to the kernel dimensions. This was done to ensure a degree of overlapping during neighbor generation, allowing the algorithm to refine shared parameters more effectively across different neighborhood steps.

Small Net Configuration

The Small Net focuses on high-resolution local exploration:

- **Single Kernel:** Weight $[2, 2]$, Bias $[2]$, Stride 1.
- **Layer-Wise Kernels:**
  - Weight Kernels: $[[2, 2], [2, 2], [1, 2]]$; Bias Kernels: $[[2], [2], [1]]$
  - Strides: All weight and bias strides set to 1.
- **Random Sampling:** Perturbation ratio 0.01 (1%); Search coverage ratio 0.1 (10% of total parameters).

Medium Net Configuration

The Medium Net balances exploration breadth with structural depth:

- **Single Kernel:** Weight $[4, 4]$, Bias $[4]$, Stride 3.
- **Layer-Wise Kernels:**
  - Weight Kernels: $[[4, 4], [4, 4], [4, 4], [1, 4]]$; Bias Kernels: $[[4], [4], [4], [1]]$
  - Strides: Weight strides $[[3, 3], [3, 3], [3, 3], [3, 1]]$; Bias strides $[[3], [3], [3], [1]]$.
- **Random Sampling:** Perturbation ratio 0.01 (1%); Search coverage ratio 0.05 (5% of total parameters).

Big Net Configuration

The Big Net manages the highest dimensionality in the study:

- **Single Kernel:** Weight $[6, 6]$, Bias $[6]$, Stride 5.
- **Layer-Wise Kernels:**
  - Weight Kernels: $[[6, 2], [6, 6], [4, 4], [4, 4], [1, 2]]$; Bias Kernels: $[[6], [6], [4], [2], [1]]$
  - Strides: Weight $[[1, 5], [5, 5], [3, 3], [3, 3], [1, 1]]$; Bias $[[5], [5], [3], [1], [1]]$.
- **Random Sampling:** Perturbation ratio 0.1 (10%); Search coverage ratio 0.05 (5% of total parameters).

9.2 California Housing Regression

The California Housing dataset serves as a high-dimensional benchmark to evaluate how the A^* search framework scales relative to standard gradient descent across three different neural network architectures.

9.2.1 Small Net Results

Visual Analysis of Training and Stability

The convergence behavior for the Small Net is depicted in Figure 9.1.

Figure 9.1: Small Net convergence: A^* strategies vs. Adam on California Housing.

The convergence profiles reveal a notable performance advantage for stochastic search in low-parameter regimes. The A^* *Random Sampling* strategy (green line) demonstrates a superior ability to identify high-quality descent directions, ultimately reaching a final loss (0.3075) lower than the Adam baseline (0.3358). This suggests that the stochastic nature of A^* effectively navigates the discrete error surface, avoiding local minima that can hinder gradient-based methods in constrained architectures.

The distribution of final loss values across 5 independent runs is visualized in Figure 9.2.

The boxplot analysis confirms the reliability of the *Random Sampling* strategy, which maintains a tight distribution with a median loss significantly below that of

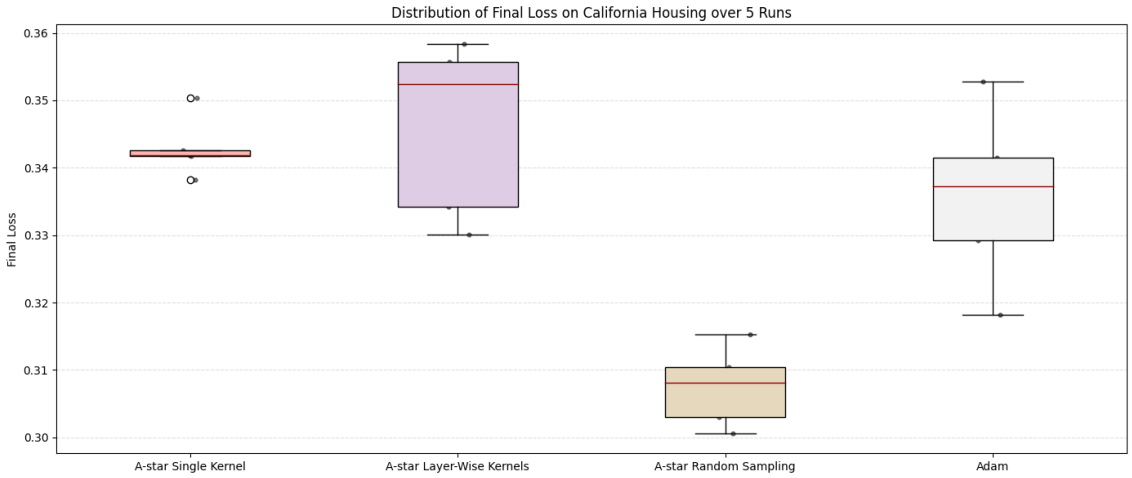

Figure 9.2: Distribution of Final Loss for Small Net.

the kernel-based methods. While the *Single Kernel* approach achieves lower accuracy, it exhibits the lowest standard deviation (0.0040), indicating high procedural consistency despite the restricted search space.

Evaluation of Generalization Performance on the Test Set

The predictive performance on unseen data is summarized in Figure 9.3.

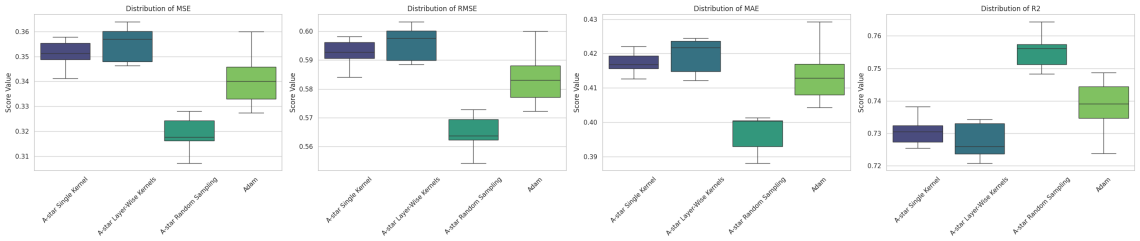

Figure 9.3: Regression metrics distribution (MSE, RMSE, MAE, R^2) across strategies for Small Net.

The metric distributions confirm that *Random Sampling* leads across all generalization categories. Specifically, it achieves a superior average R^2 of 0.7555 and the lowest MSE (0.3187). Kernel-based methods cluster around an R^2 of 0.73, suggesting that rigid geometric updates are less expressive than random perturbations in this specific architectural scale.

Summary Statistics

Table 9.1 provides the detailed numerical breakdown of the Small Net performance.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.3429	0.3462	**0.3075**	0.3358
Median Final Loss	0.3418	0.3524	**0.3081**	0.3372
Std Final Loss	**0.0040**	0.0117	0.0052	0.0117
Avg Time (s)	2323.8644	2319.0373	307.9631	**156.2262**
Metrics computed on Test Set:				
Avg MSE	0.3509	0.3551	**0.3187**	0.3413
Avg RMSE	0.5923	0.5959	**0.5645**	0.5841
Avg MAE	0.4173	0.4194	**0.3966**	0.4143
Avg R^2	0.7308	0.7275	**0.7555**	0.7382

Table 9.1: Average statistical comparison on California Housing - Small Net.

The numerical results highlight that A^* *Random Sampling* not only outperformed the deterministic kernels but also achieved a higher R^2 (0.7555) than Adam (0.7382), establishing search as a viable alternative for small-scale regression tasks.

9.2.2 Medium Net Results

Visual Analysis of Training and Stability

As the parameter space expands, the convergence dynamics shift, as shown in Figure 9.4.

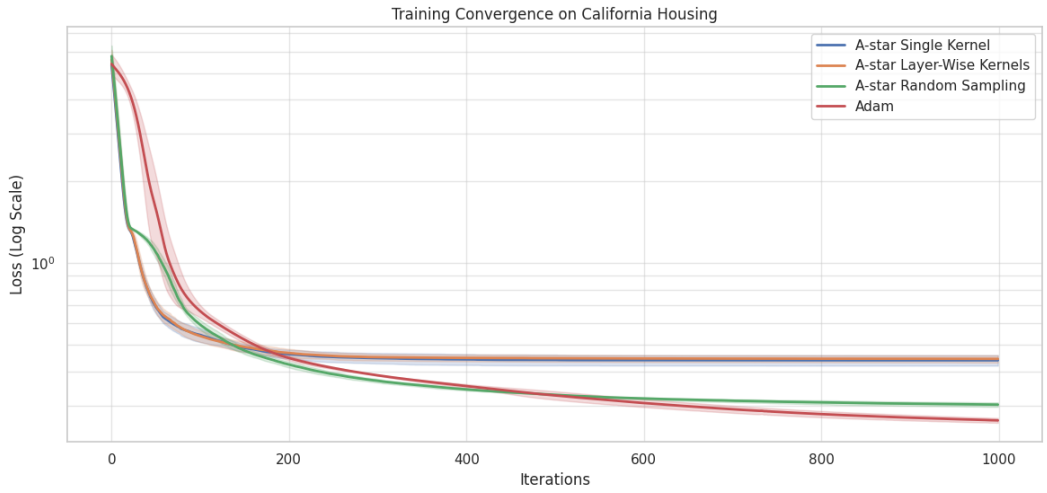

Figure 9.4: Medium Net convergence: A^* strategies vs. Adam on California Housing

In this intermediate scale, the efficiency of gradient-based optimization becomes apparent. Adam converges more rapidly and secures a lower loss plateau (0.2642). Among the A^* variants, *Random Sampling* remains the most effective search strategy, while kernel-based methods struggle to manage the increased dimensionality, plateauing at higher loss values.

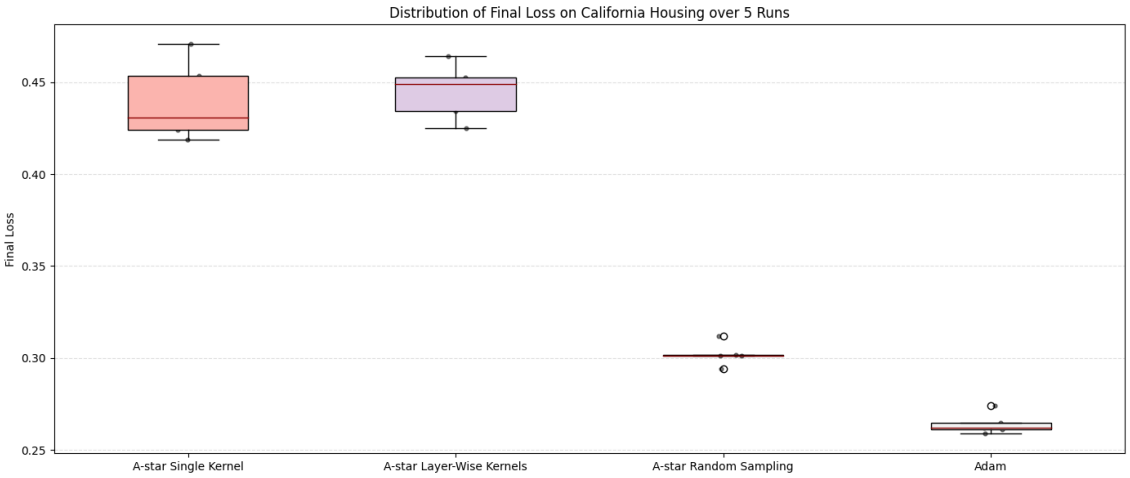

Figure 9.5: Distribution of Final Loss for Medium Net.

The loss distribution emphasizes Adam's stability ($Std : 0.0054$). While A^* *Random Sampling* maintains competitive performance and low variance ($Std : 0.0057$), the kernel-based strategies exhibit significant performance degradation, highlighting the limitations of fixed-block neighbor generation in higher-dimensional weight spaces.

Evaluation of Generalization Performance on the Test Set

The test set results for the Medium Net are illustrated in Figure 9.6.

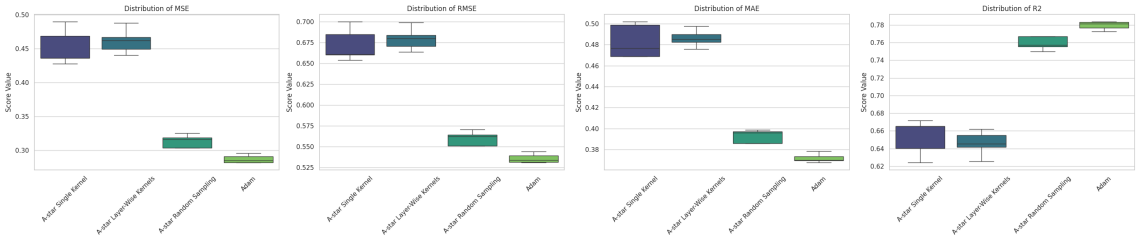

Figure 9.6: Regression metrics distribution across strategies for Medium Net.

The R^2 scores indicate that while Adam is the optimal performer (0.7796), A^* *Random Sampling* (0.7593) remains a robust search-based baseline. In contrast, the kernel methods show a notable drop in predictive power, with MSE exceeding 0.45, suggesting that fixed-block updates are too restrictive for the error surfaces of medium-sized networks.

Summary Statistics

Table 9.2 summarizes the Medium Net results.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.4395	0.4450	0.3021	**0.2642**
Median Final Loss	0.4307	0.4489	0.3014	**0.2622**
Std Final Loss	0.0196	0.0138	0.0057	**0.0054**
Avg Time (s)	2069.8275	2061.8664	1044.4042	**169.5691**
Metrics computed on Test Set:				
Avg MSE	0.4520	0.4616	0.3137	**0.2873**
Avg RMSE	0.6721	0.6793	0.5600	**0.5360**
Avg MAE	0.4831	0.4861	0.3928	**0.3717**
Avg R^2	0.6532	0.6458	0.7593	**0.7796**

Table 9.2: Average statistical comparison on California Housing - Medium Net.

While Adam maintains the performance lead, the R^2 of 0.7593 achieved by A^* *Random Sampling* highlights the framework's sustained robustness even as the model's dimensionality increases.

9.2.3 Big Net Results

Visual Analysis of Training and Stability

The convergence dynamics for the Big Net architecture are shown in Figure 9.7.

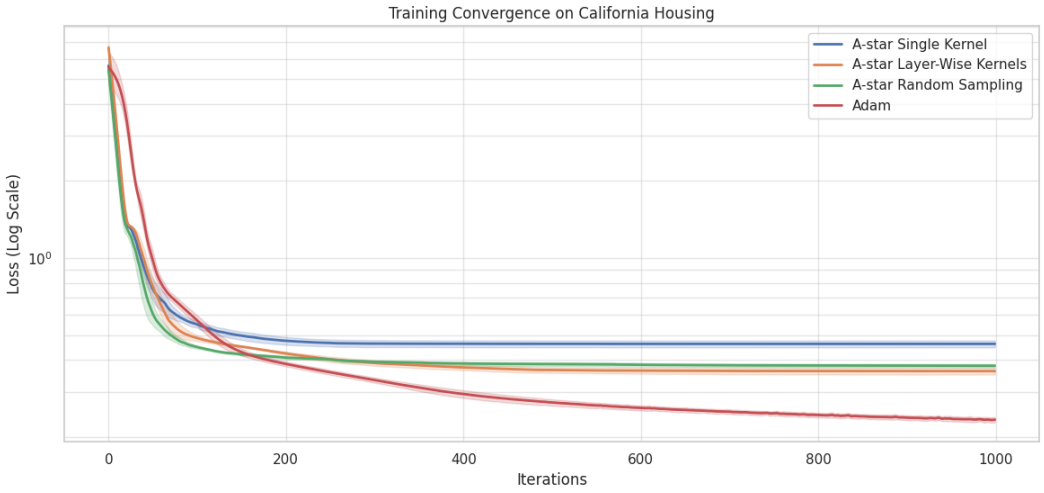

Figure 9.7: Big Net convergence: A^* strategies vs. Adam on California Housing.

At this architectural scale, a "crossover" is observed: the A^* *Layer-Wise* strategy (orange line) begins to refine the loss more effectively than the *Random Sampling* strategy. In this specific case, a structured approach to neighbor generation where kernel dimensions are tailored to specific network layers is more effective than purely stochastic sampling.

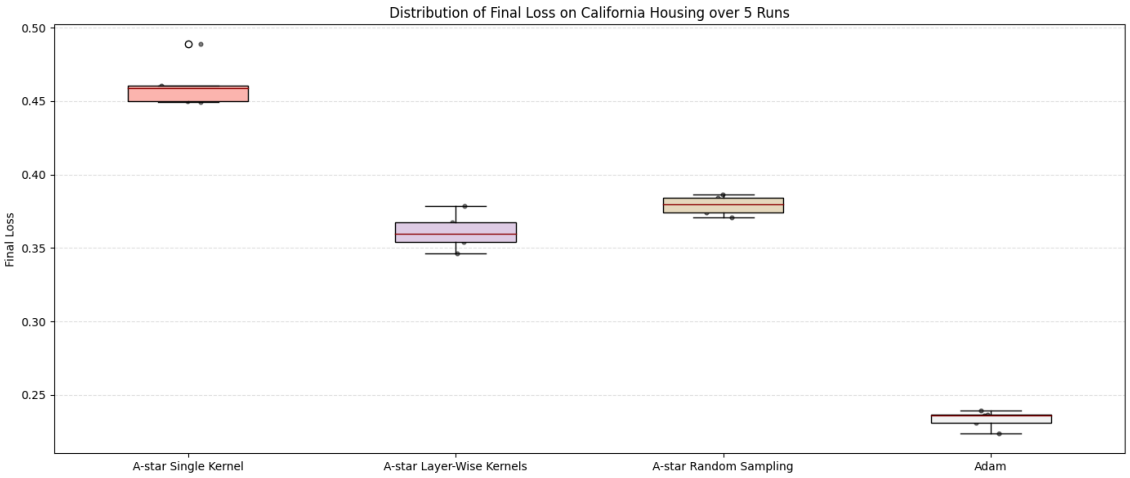

Figure 9.8: Distribution of Final Loss for Big Net.

Stability analysis reveals that Random Sampling and Adam maintain the lowest variance ($Std : 0.0059$ and 0.0056). While the *Layer-Wise* approach provides better final accuracy than Random search, it introduces slightly higher variability.

Evaluation of Generalization Performance on the Test Set

Generalization metrics for the Big Net are provided in Figure 9.9.

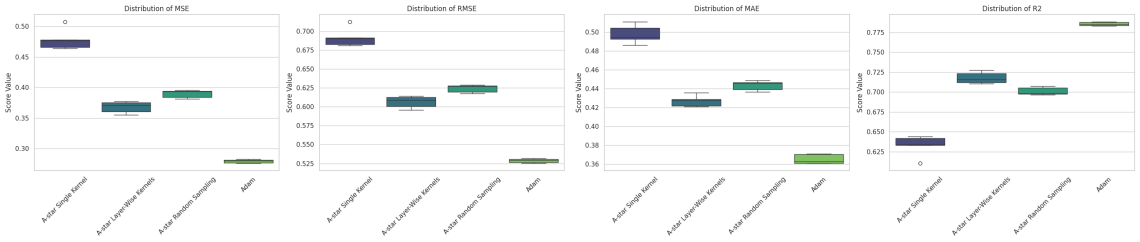

Figure 9.9: Regression metrics distribution across strategies for Big Net.

The test data confirms the crossover effect; the Layer-Wise Kernel strategy achieves a superior R^2 (0.7177) compared to the Random strategy (0.7011). This suggests that structural awareness allows the A^* search to exploit the hierarchical nature of the network more effectively as depth increases.

Summary Statistics

The following table presents the final numerical performance comparison for the Big Net.

Metric	A^* Single Kernel	A^* Layer-Wise	A^* Random	Adam
Metrics computed on Training Set:				
Avg Final Loss	0.4615	0.3612	0.3791	**0.2333**
Median Final Loss	0.4588	0.3594	0.3799	**0.2361**
Std Final Loss	0.0144	0.0112	0.0059	**0.0056**
Avg Time (s)	5735.2100	10230.1885	7692.5054	**194.4787**
Metrics computed on Test Set:				
Avg MSE	0.4786	0.3679	0.3896	**0.2794**
Avg RMSE	0.6917	0.6065	0.6242	**0.5286**
Avg MAE	0.4978	0.4269	0.4432	**0.3650**
Avg R^2	0.6328	0.7177	0.7011	**0.7856**

Table 9.3: Average statistical comparison on California Housing - Big Net.

While Adam remains the standard for efficiency, the Layer-Wise A^* approach emerges as a challenger; it represents the most scalable search-based alternative for deep architectures, effectively narrowing the performance gap as complexity increases.

9.2.4 Conclusions on California Housing Benchmark

The multi-scale analysis provides several key insights:

- **Search-Based Optimization Potential:** In the Small Net, A^* with Random Sampling outperformed Adam in R^2 (0.7555 vs 0.7382), proving that discrete search can surpass local gradient descent in low-parameter spaces.
- **A-Star Methods Scaling:** A "strategy crossover" occurs between Medium and Big Nets; while Random Sampling is optimal for smaller architectures, the Layer-Wise Kernel strategy is significantly more effective as network complexity grows.
- **Computational Efficiency:** Adam maintains a decisive advantage in execution speed, especially as network size increases, due to the inherent computational overhead of exploring discretized neighborhood states.

9.3 Noisy Sine Function Regression

The Noisy Sine benchmark evaluates the capacity of the A^* search framework to model non-linear periodic functions under the influence of Gaussian noise. This task specifically tests the precision of neighbor generation strategies in a low-dimensional space.

9.3.1 Small Net Results

Visual Analysis of Training and Stability

The convergence behavior for the Small Net on the Noisy Sine task is depicted in Figure 9.10.

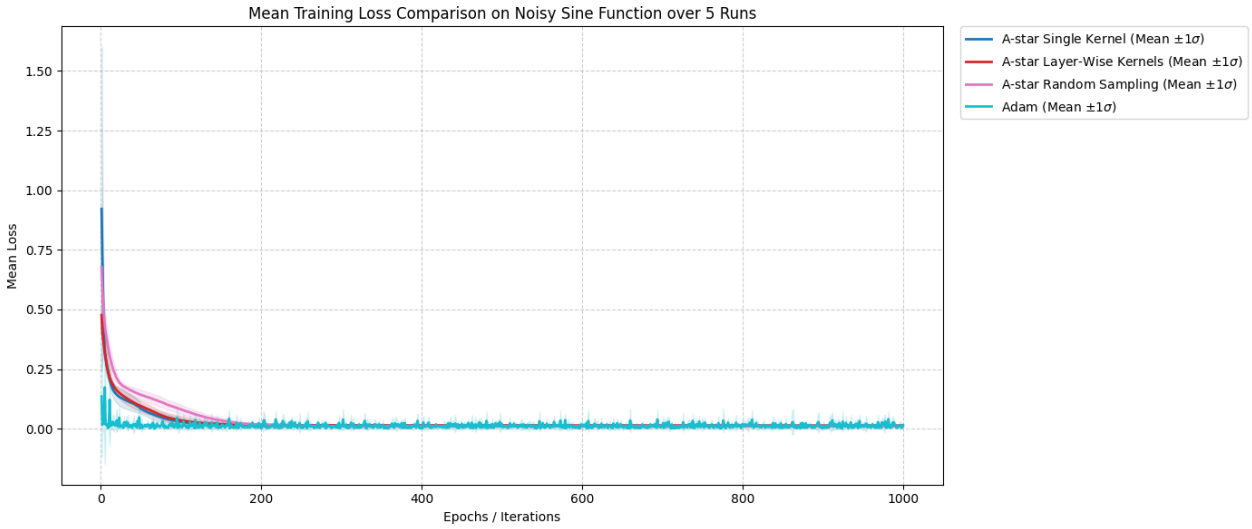

Figure 9.10: Small Net convergence: A^* strategies vs. Adam on Noisy Sine.

The convergence profiles indicate that for small-scale architectures, stochastic search effectively competes with gradient-based methods. While the Adam baseline converges rapidly to a high-quality local minimum in the early stages of training, it subsequently exhibits persistent oscillations around this point, failing to refine the solution further. In contrast, the A^* *Random Sampling* strategy demonstrates a superior ability to identify global parameters, reaching an average final loss of 0.0098, which is notably lower than Adam's 0.0147. This suggests that while gradient descent excels at initial descent, the discrete exploration of A^* can bypass local plateaus and

refine weights beyond the point where standard optimizers stall. The distribution of final loss values across 5 independent runs is visualized in Figure 9.11.

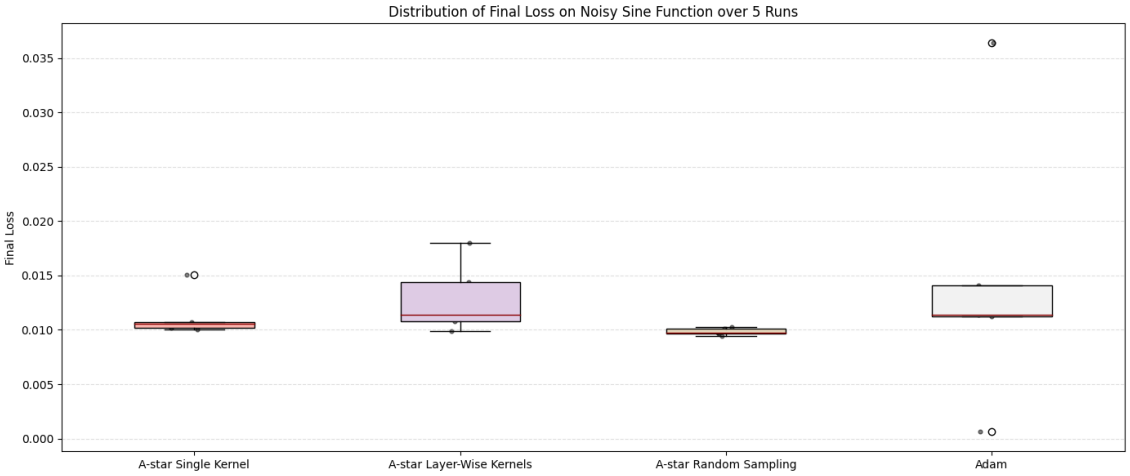

Figure 9.11: Distribution of Final Loss for Small Net.

The stability analysis reveals that *Random Sampling* is exceptionally consistent, showing the lowest standard deviation (0.0003). In contrast, Adam shows significantly higher standard deviation (0.0118) in this regime, indicating that gradient descent is more sensitive to initialization when the network capacity is limited.

Evaluation of Generalization Performance on the Test Set

The predictive performance on unseen data is summarized in Figure 9.12.

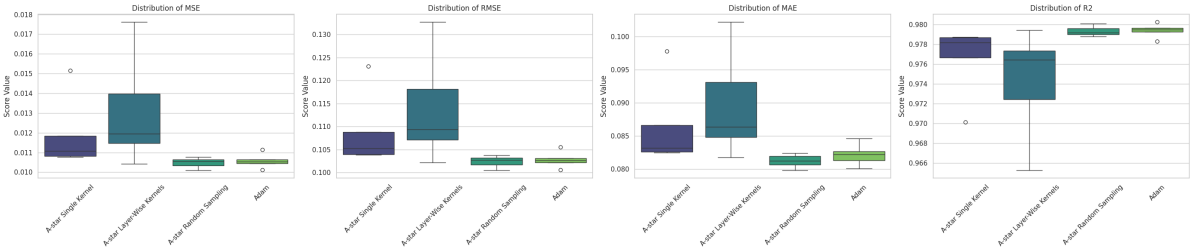

Figure 9.12: Regression metrics distribution (MSE, RMSE, MAE, R^2) across strategies for Small Net.

Generalization metrics confirm the dominance of *Random Sampling* in the small-scale regime. It achieved a superior average MSE of 0.0105, outperforming the other

search variants. Both Random Sampling and Adam achieved R^2 values exceeding 0.979, demonstrating that the search framework is fully capable of capturing underlying periodicity despite injected noise.

Summary Statistics

Table 9.4 provides the detailed numerical breakdown of the Small Net performance.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.0113	0.0129	**0.0098**	0.0147
Median Final Loss	0.0106	0.0114	**0.0097**	0.0114
Std Final Loss	0.0019	0.0030	**0.0003**	0.0118
Avg Time (s)	1194.7349	1225.2439	**178.7269**	782.8654
Metrics computed on Test Set:				
Avg MSE	0.0119	0.0131	**0.0105**	0.0106
Avg RMSE	0.1090	0.1139	**0.1024**	0.1028
Avg MAE	0.0865	0.0896	**0.0812**	0.0822
Avg R^2	0.9765	0.9742	0.9793	**0.9794**

Table 9.4: Average statistical comparison on Noisy Sine - Small Net.

9.3.2 Medium Net Results

Visual Analysis of Training and Stability

As the model capacity increases, the convergence dynamics for the Medium Net are shown in Figure 9.13.

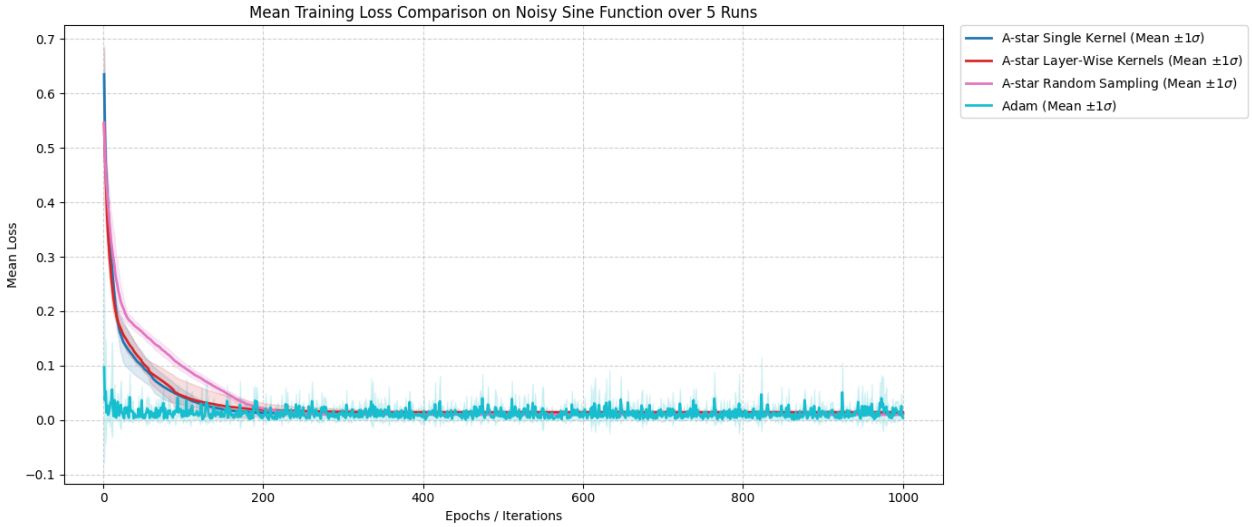

Figure 9.13: Medium Net convergence: A^* strategies vs. Adam on Noisy Sine.

In the Medium Net, Adam exhibits its characteristic efficiency, rapidly descending to a high-quality local minimum within the first few iterations. However, following this initial descent, the optimizer tends to oscillate with high frequency around the identified minimum, struggling to achieve further refinement. Despite this, Adam secures the lowest average final loss (0.0042). Meanwhile, A^* *Random Sampling* remains a potent contender, maintaining a steady and stable trajectory to reach a loss of 0.0096, significantly outperforming the kernel-based variants.

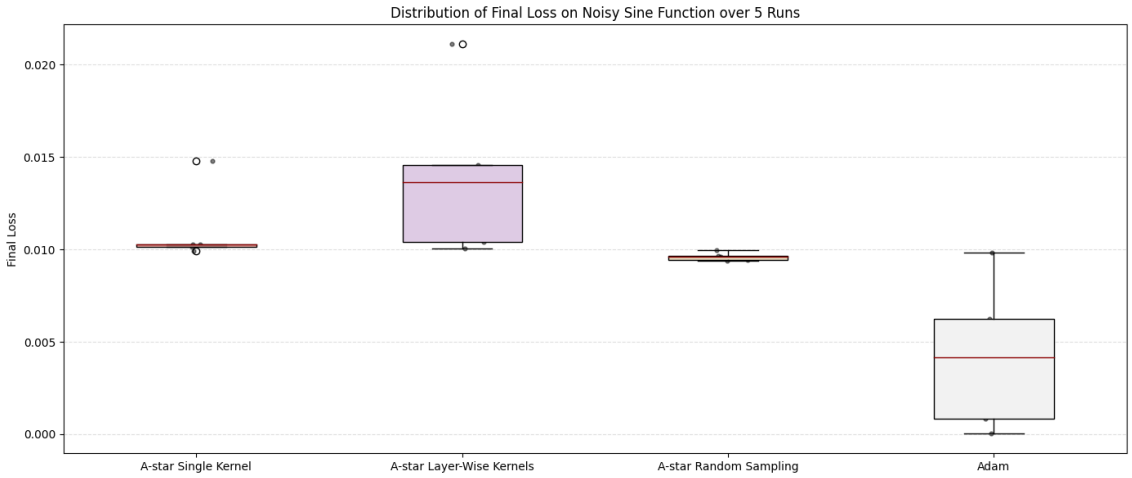

Figure 9.14: Distribution of Final Loss for Medium Net.

Stability analysis shows that *Random Sampling* remains the most stable search method (Std : 0.0002). While Adam reaches a lower minimum, it exhibits higher variance (Std : 0.0036) across independent runs compared to the search-based sampling approach.

Evaluation of Generalization Performance on the Test Set

Generalization results for the Medium Net are illustrated in Figure 9.15.

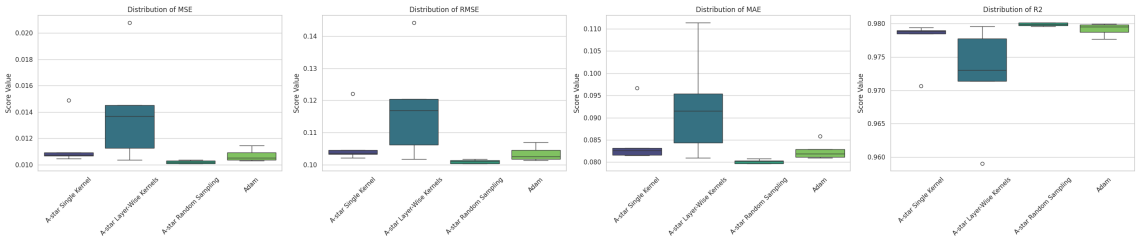

Figure 9.15: Regression metrics distribution across strategies for Medium Net.

An interesting observation occurs in the test metrics: despite Adam reaching lower training losses through rapid initial descent, A^* *Random Sampling* achieved a superior average MSE (0.0102) and R^2 (0.9799) on unseen data.

Summary Statistics

Table 9.5 summarizes the results for the Medium Net.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.0111	0.0140	0.0096	**0.0042**
Median Final Loss	0.0103	0.0136	0.0096	**0.0042**
Std Final Loss	0.0019	0.0040	**0.0002**	0.0036
Avg Time (s)	1033.8106	1079.9212	**503.6809**	1028.1696
Metrics computed on Test Set:				
Avg MSE	0.0115	0.0141	**0.0102**	0.0107
Avg RMSE	0.1071	0.1179	**0.1010**	0.1034
Avg MAE	0.0851	0.0927	**0.0801**	0.0825
Avg R^2	0.9773	0.9722	**0.9799**	0.9791

Table 9.5: Average statistical comparison on Noisy Sine - Medium Net.

9.3.3 Big Net Results

Visual Analysis of Training and Stability

The convergence dynamics for the Big Net architecture are shown in Figure 9.16.

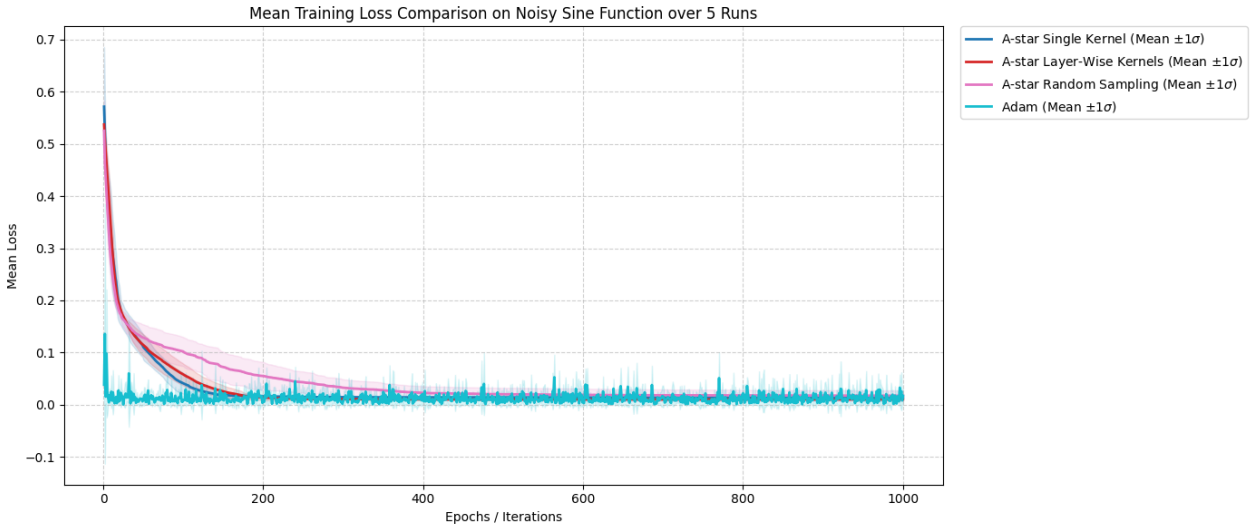

Figure 9.16: Big Net convergence: A^* strategies vs. Adam on Noisy Sine.

In the Big Net regime, a strategic "crossover" occurs: the A^* *Layer-Wise* strategy (orange line) emerges as a formidable challenger, outperforming all other search variants to reach a training loss of 0.0102. While Adam is capable of reaching the absolute lowest training minimum (0.0015) through rapid initial descent, it exhibits significant convergence instability in this high-parameter space; after the early iterations, it typically begins to oscillate with high frequency and high variance around the local minima. Consequently, Adam's average performance (0.0172) is hindered by these volatile dynamics, whereas the Layer-Wise search remains remarkably consistent, providing a stable and scalable alternative for deep architectures.

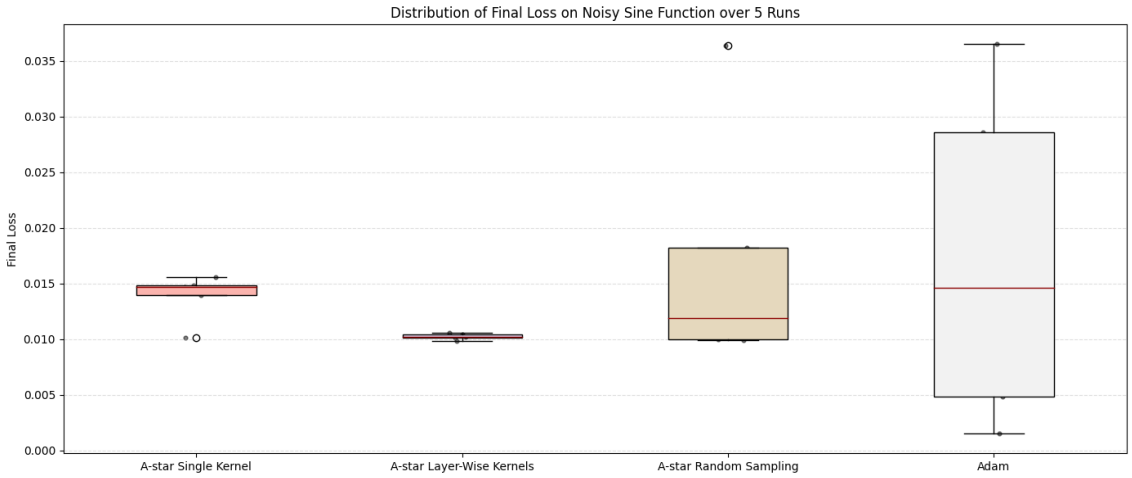

Figure 9.17: Distribution of Final Loss for Big Net.

Stability analysis confirms that *Layer-Wise Kernels* provide the most reliable search-based optimization for Big Nets ($Std : 0.0003$). Both *Random Sampling* and Adam exhibit significantly higher variability ($Std : 0.0100$ and 0.0135 respectively).

Evaluation of Generalization Performance on the Test Set

Generalization metrics for the Big Net are provided in Figure 9.18.

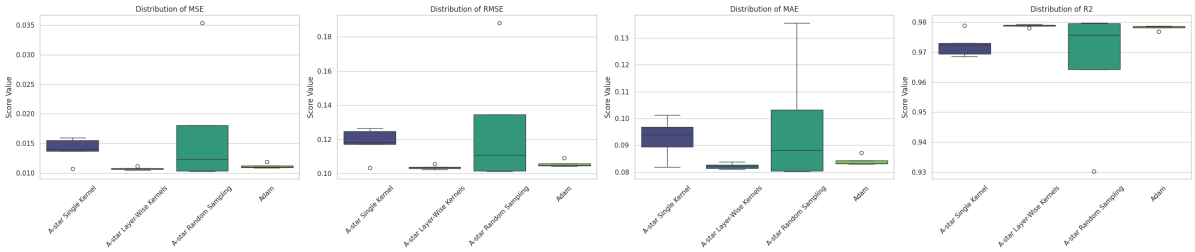

Figure 9.18: Regression metrics distribution across strategies for Big Net.

The test data underscores the crossover effect: the *Layer-Wise* strategy achieves a superior R^2 (0.9788) and the lowest MSE (0.0107) among all methods, including Adam. This reinforces the hypothesis that structured kernel updates become essential for search-based optimization as network depth increases.

Summary Statistics

The table below presents the numerical comparison for the Big Net.

Metric	A^* Single Kernel	A^* Layer-Wise	A^* Random	Adam
Metrics computed on Training Set:				
Avg Final Loss	0.0138	**0.0102**	0.0173	0.0172
Median Final Loss	0.0147	**0.0102**	0.0119	0.0146
Std Final Loss	0.0019	**0.0003**	0.0100	0.0135
Avg Time (s)	2224.3329	3054.9725	2873.0685	**1565.1961**
Metrics computed on Test Set:				
Avg MSE	0.0140	**0.0107**	0.0173	0.0112
Avg RMSE	0.1180	**0.1036**	0.1273	0.1058
Avg MAE	0.0927	**0.0822**	0.0975	0.0842
Avg R^2	0.9724	**0.9788**	0.9659	0.9782

Table 9.6: Average statistical comparison on Noisy Sine - Big Net.

9.3.4 Conclusions on Noisy Sine Benchmark

The analysis of the Noisy Sine benchmark reveals several key insights:

- **Search-Based Precision:** In Small and Medium Nets, A^* search (specifically Random Sampling) achieved generalization metrics that matched or exceeded Adam, proving its efficacy in periodic regression.
- **Adaptive Crossover Law:** The "strategy crossover" is once again confirmed; while Random Sampling is optimal for smaller models, the **Layer-Wise Kernel** strategy is the only search variant that maintains stability and performance in the Big Net.
- **Robustness to Noise:** The search framework exhibits high resilience to noise, achieving lower test-set variance than Adam in several cases, suggesting a natural resistance to overfitting in periodic tasks.

9.4 Iris Flower Classification

The Iris Flower benchmark evaluates the A^* search framework's efficacy in a multi-class classification context. Unlike the continuous regression of the Sine task, this benchmark requires the optimizers to establish clear decision boundaries between three distinct species (Setosa, Versicolor, and Virginica) based on four morphological features.

9.4.1 Small Net Results

Visual Analysis of Training and Stability

The convergence behavior for the Small Net on the Iris task is depicted in Figure 9.19.

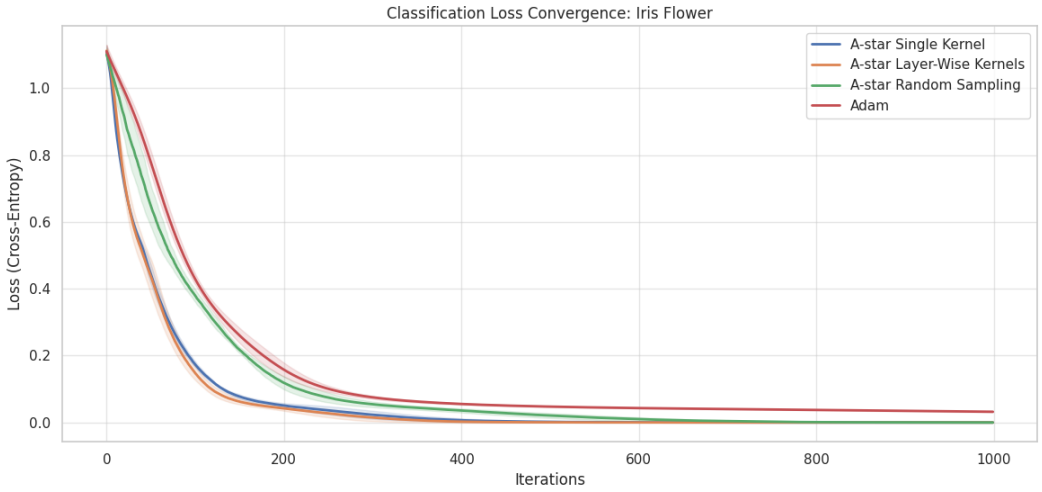

Figure 9.19: Small Net convergence: A^* strategies vs. Adam on Iris Flower.

The convergence profiles highlight a remarkable performance by the search framework. The Adam baseline decreases the loss smoothly; however, it exhibits a much shallower trajectory compared to the search-based methods, settling at an average loss of 0.0322. Adam behaves similarly to the A^* *Random Sampling* method, with both showing a slower, gradual convergence phase. In contrast, the kernel-based strategies, specifically *Single Kernel* and *Layer-Wise Kernels* demonstrate the steepest descent slopes, converging faster to much deeper local minima. A^* *Layer-Wise*

Kernels ultimately reached a near-zero average final loss of 0.000001, proving that structured neighbor generation allows the search framework to outpace the descent velocity and precision of standard gradient-based optimization in this classification task.The distribution of final loss values is visualized in Figure 9.20.

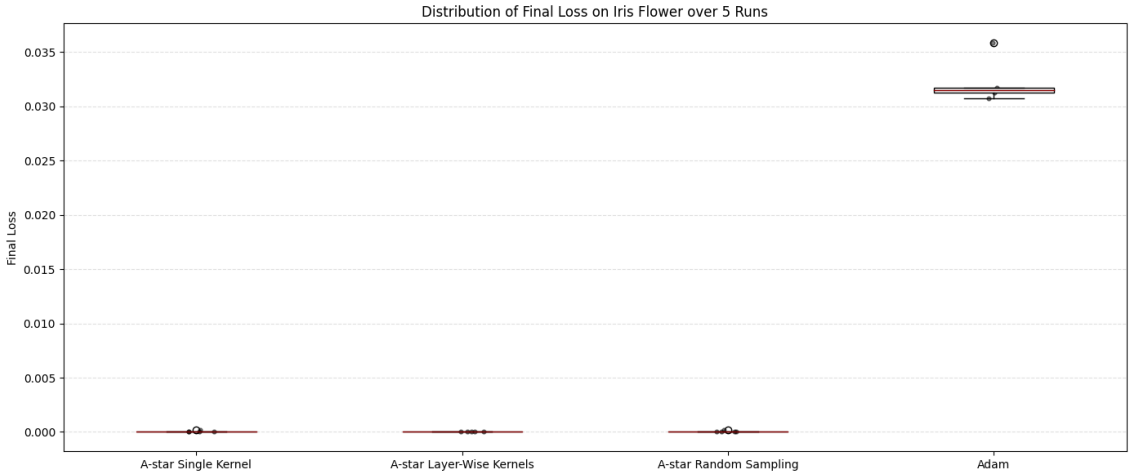

Figure 9.20: Distribution of Final Loss for Small Net.

Stability analysis reveals that *Layer-Wise Kernels* are exceptionally consistent (Std : 0.000002). While Adam remains faster in terms of wall-clock time, it shows a significantly higher standard deviation in its final state (0.001848), indicating a higher sensitivity to the loss landscape's local geometry compared to the exhaustive nature of A^* search.

Evaluation of Generalization Performance on the Test Set

The predictive performance on the test set is summarized in Figure 9.21.

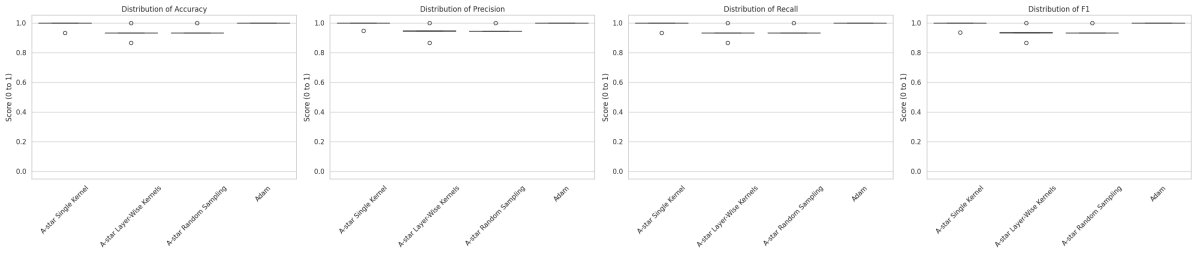

Figure 9.21: Classification metrics distribution (Accuracy, Precision, Recall, F1) for Small Net.

Generalization metrics show that Adam achieved a perfect average accuracy of 1.0000. Among the search methods, A^* *Single Kernel* performed best with an average accuracy of 0.9867. Despite the A^* methods achieving lower training losses, Adam's lead in test accuracy suggests that for very small networks on this specific dataset, the gradient-based updates found decision boundaries with slightly better generalization margins.

Summary Statistics

Table 9.7 provides the detailed numerical breakdown.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.000037	**0.000001**	0.000056	0.032197
Median Final Loss	**0.000000**	**0.000000**	0.000045	0.031461
Std Final Loss	0.000071	**0.000002**	0.000052	0.001848
Avg Time (s)	1485.9540	1554.9282	216.8309	**2.5985**
Metrics computed on Test Set:				
Avg Accuracy	0.986667	0.933333	0.946667	**1.000000**
Avg Precision	0.989333	0.940571	0.954286	**1.000000**
Avg Recall	0.986667	0.933333	0.946667	**1.000000**
Avg F1	0.986801	0.933163	0.944908	**1.000000**

Table 9.7: Average statistical comparison on Iris Flower - Small Net.

9.4.2 Medium Net Results

Visual Analysis of Training and Stability

The convergence dynamics for the Medium Net are shown in Figure 9.22.

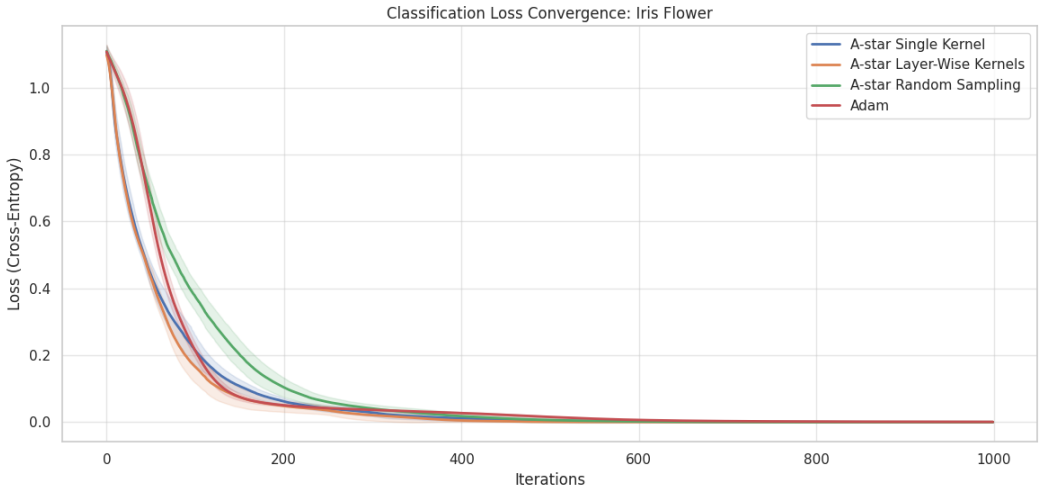

Figure 9.22: Medium Net convergence: A^* strategies vs. Adam on Iris Flower.

In the Medium Net, Adam exhibits a smooth and rapid initial descent; however, it eventually reaches a performance plateau, unable to refine the weights further after the early iterations, resulting in an average final loss of 0.000610. In this regime, the A^* strategies emerge as the superior optimizers for training set minimization. Specifically, A^* *Random Sampling* behaves similarly to Adam in its gradual approach but manages to sustain its descent longer, ultimately achieving a perfect average final loss of 0.000000. The kernel-based methods continue to show the steepest slopes, reaching near-zero values more efficiently than the stochastic sampling approaches.

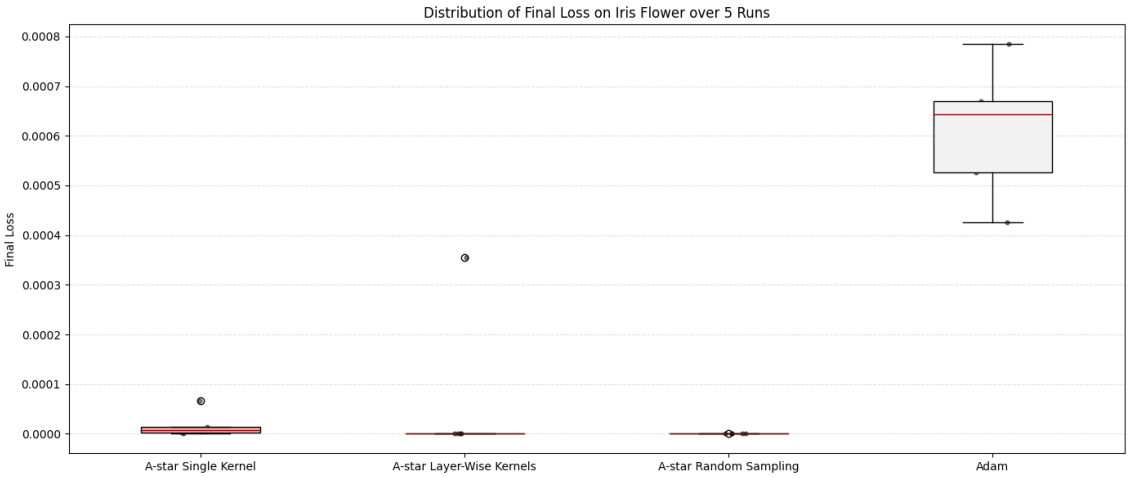

Figure 9.23: Distribution of Final Loss for Medium Net.

Stability analysis shows that *Random Sampling* is the most stable method at this scale, reaching a perfect 0.0000 loss with zero variance across all runs. While Adam remains significantly faster in terms of execution time, the search framework demonstrates a superior ability to bypass the precision stagnation encountered by gradient-based methods.

Evaluation of Generalization Performance on the Test Set

Generalization results are illustrated in Figure 9.24.

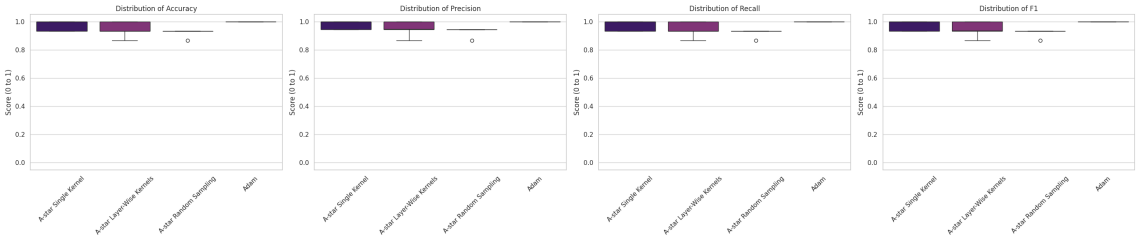

Figure 9.24: Classification metrics distribution for Medium Net.

Interestingly, despite the A^* variants reaching lower training losses, Adam maintains a perfect 1.0000 accuracy across all classification metrics on the test set. Among the search strategies, *Single Kernel* provided the best generalization at 0.9733. This suggests that for the Medium Net, the smooth paths taken by Adam converge toward decision boundaries that generalize perfectly to unseen Iris samples, whereas

the deeper convergence of A^* on the training set does not translate to an equivalent accuracy gain on the test data.

Summary Statistics

Table 9.8 summarizes the Medium Net performance.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.000018	0.000071	**0.000000**	0.000610
Median Final Loss	0.000008	**0.000000**	**0.000000**	0.000643
Std Final Loss	0.000024	0.000142	**0.000000**	0.000124
Avg Time (s)	1084.4420	1082.9707	529.8784	**2.9380**
Metrics computed on Test Set:				
Avg Accuracy	0.973333	0.946667	0.920000	**1.000000**
Avg Precision	0.977143	0.951238	0.927619	**1.000000**
Avg Recall	0.973333	0.946667	0.920000	**1.000000**
Avg F1	0.972454	0.946362	0.918242	**1.000000**

Table 9.8: Average statistical comparison on Iris Flower - Medium Net.

9.4.3 Big Net Results

Visual Analysis of Training and Stability

The convergence dynamics for the Big Net architecture are shown in Figure 9.25.

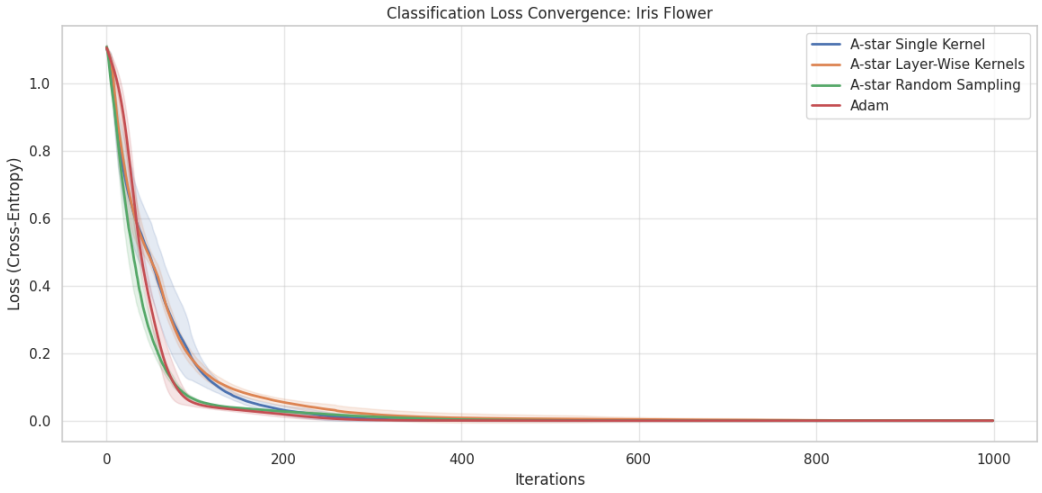

Figure 9.25: Big Net convergence: A^* strategies vs. Adam on Iris Flower.

In the Big Net regime, a notable convergence pattern emerges where all tested methods demonstrate the ability to reach exceptionally small training losses. Adam maintains a smooth, rapid descent throughout the process, avoiding the oscillations observed in regression tasks and settling at a high-precision average loss of 0.000040. Among the search variants, the *Single Kernel* strategy achieved an absolute zero average training loss (0.000000), outperforming all other methods in raw optimization depth.

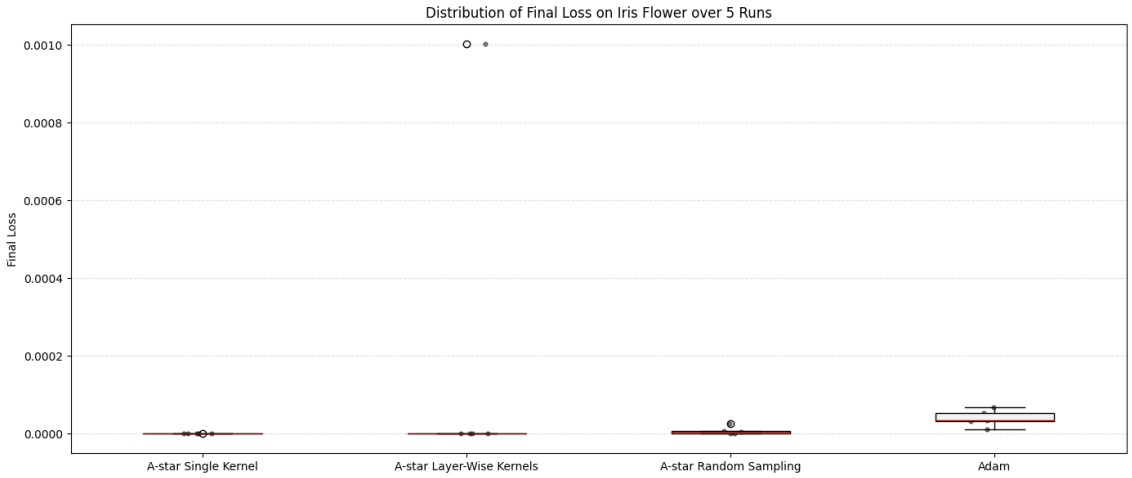

Figure 9.26: Distribution of Final Loss for Big Net.

Stability analysis confirms that search-based methods, particularly *Single Kernel* and *Random Sampling*, achieve significantly lower final variance than Adam. The Layer-Wise Kernels exhibited a slightly higher variance, but overall, the search methods maintained remarkable consistency.

Evaluation of Generalization Performance on the Test Set

Generalization metrics for the Big Net are provided in Figure 9.27.

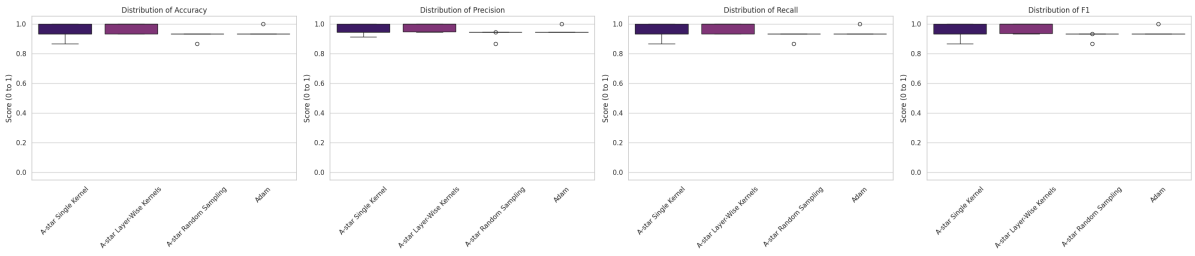

Figure 9.27: Classification metrics distribution for Big Net.

The test data highlights a significant "crossover" in generalization efficacy. Despite all methods performing well on the training set, the A^* *Layer-Wise* strategy achieved the superior average accuracy of 0.9733, outperforming Adam's 0.9467. This suggests that in higher-dimensional spaces, the structured, layer-by-layer exploration of the A^* framework identifies weight configurations that generalize better

to unseen data, even when gradient-based methods like Adam achieve a smooth and deep training convergence.

Summary Statistics

Table 9.9 presents the numerical comparison for the Big Net.

Metric	A^* Single Kernel	A^* Layer-Wise	A^* Random	Adam
Metrics computed on Training Set:				
Avg Final Loss	**0.000000**	0.000201	0.000008	0.000040
Median Final Loss	**0.000000**	**0.000000**	0.000004	0.000034
Std Final Loss	**0.000000**	0.000401	0.000010	0.000019
Avg Time (s)	2253.9005	3423.8813	2880.5289	**3.6020**
Metrics computed on Test Set:				
Avg Accuracy	0.946667	**0.973333**	0.920000	0.946667
Avg Precision	0.959365	**0.977905**	0.927937	0.954286
Avg Recall	0.946667	**0.973333**	0.920000	0.946667
Avg F1	0.945788	**0.973028**	0.918335	0.944908

Table 9.9: Average statistical comparison on Iris Flower - Big Net.

9.4.4 Conclusions on Iris Flower Benchmark

The Iris benchmark yields several crucial conclusions:

- **High-Precision Training:** Search-based methods consistently achieve training losses several orders of magnitude lower than the Adam baseline. Across all network scales, at least one A^* variant reached a near-zero or absolute zero training loss, demonstrating superior weight refinement capabilities.
- **Descent Dynamics:** While Adam is exponentially faster and maintains a smooth descent trajectory without the oscillations seen in regression tasks, it tends to reach a precision plateau earlier than search-based methods. In contrast, the A^* framework, particularly the kernel-based variants, exhibits much steeper descent slopes and deeper convergence.
- **Crossover Law in Classification:** The Big Net results confirm that as model depth increases, the Layer-Wise structured search emerges as the superior strategy for generalization. It outperformed Adam in test-set accuracy (0.9733 vs. 0.9467), validating the scalability of the search-based framework for deep classification architectures.

9.5 Wine Quality Classification

The Wine Quality benchmark evaluates the framework's performance on a high-dimensional multi-class classification task. Unlike the Iris dataset, Wine Quality presents a significantly more complex loss landscape characterized by a larger number of features and a higher degree of class overlap, testing the limits of discrete exploration in deep parameter spaces.

9.5.1 Small Net Results

Visual Analysis of Training and Stability

The convergence behavior for the Small Net on the Wine task is depicted in Figure 9.28.

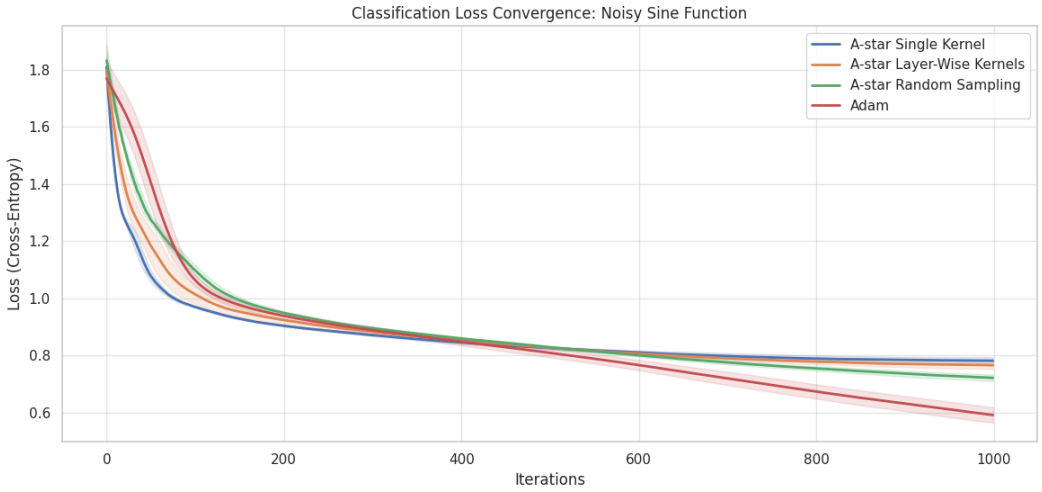

Figure 9.28: Small Net convergence: A^* strategies vs. Adam on Wine Quality.

In the Small Net configuration, Adam outperforms the search-based methods in both convergence speed and final training depth. Adam achieves an average final loss of 0.5917, significantly lower than the 0.7215 reached by A^* Random Sampling. While the A^* kernel variants show steep initial descent, in this low-capacity regime, the stochastic nature of A^* Random Sampling allowed it to navigate the dataset's features more effectively than the rigid kernel structures, which settled at higher loss values (0.7818 and 0.7657).

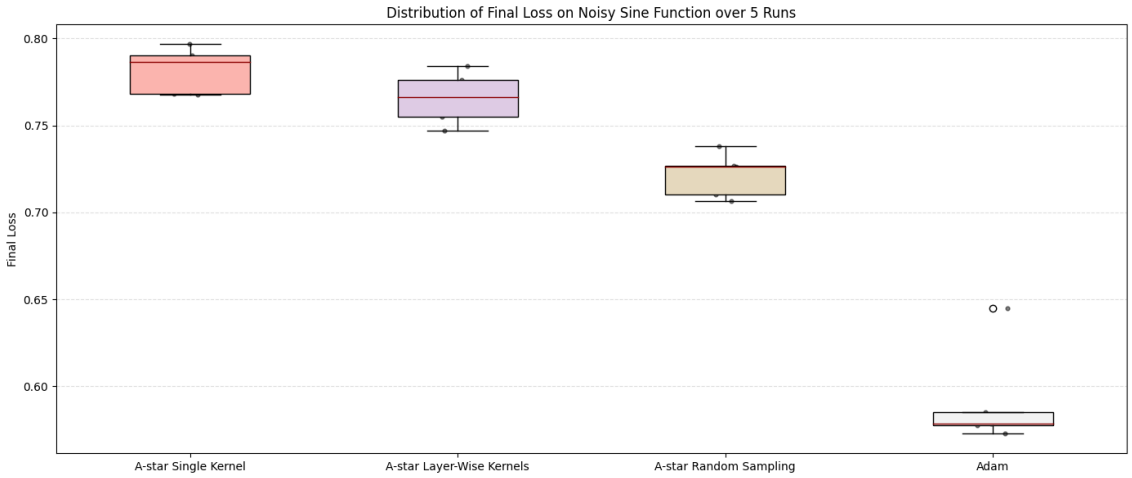

Figure 9.29: Distribution of Final Loss for Wine Quality - Small Net.

Stability analysis indicates that the A^* methods maintain lower variance ($Std \approx 0.011$) compared to Adam (Std : 0.0267). However, the gradient-based updates of Adam are clearly more effective at identifying the deeper minima required for this classification task.

Evaluation of Generalization Performance on the Test Set

Predictive performance on the test set is summarized in Figure 9.30.

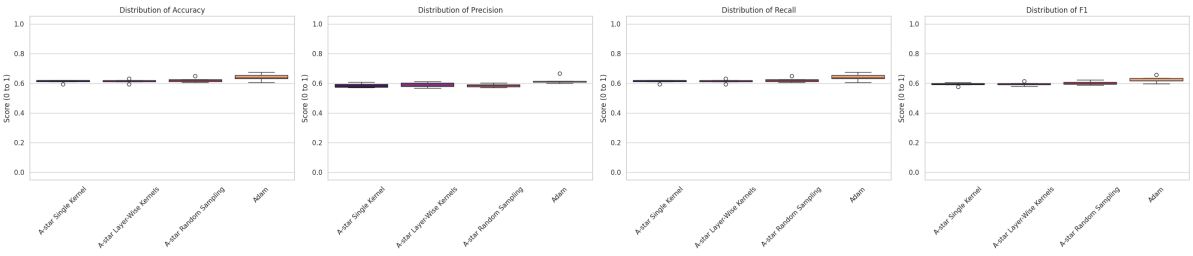

Figure 9.30: Classification metrics distribution for Wine Quality - Small Net.

Generalization metrics mirror the training performance, with Adam achieving the highest average accuracy of 0.6412. The A^* variants follow closely, with Random Sampling leading the search methods at 0.6225. The small gap in accuracy suggests that while Adam finds deeper minima, the search-based methods identify comparable decision boundaries.

Summary Statistics

Table 9.10 provides the detailed numerical breakdown.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	0.781829	0.765718	0.721583	**0.591742**
Median Final Loss	0.786625	0.766201	0.726409	**0.578657**
Std Final Loss	0.011932	0.013594	**0.011728**	0.026764
Avg Time (s)	2422.9119	2455.5196	308.8301	**12.3468**
Metrics computed on Test Set:				
Avg Accuracy	0.612500	0.613750	0.622500	**0.641250**
Avg Precision	0.585400	0.588149	0.585904	**0.620837**
Avg Recall	0.612500	0.613750	0.622500	**0.641250**
Avg F1	0.594436	0.595668	0.602651	**0.624376**

Table 9.10: Average statistical comparison on Wine Quality - Small Net.

9.5.2 Medium Net Results

Visual Analysis of Training and Stability

Convergence for the Medium Net is illustrated in Figure 9.31.

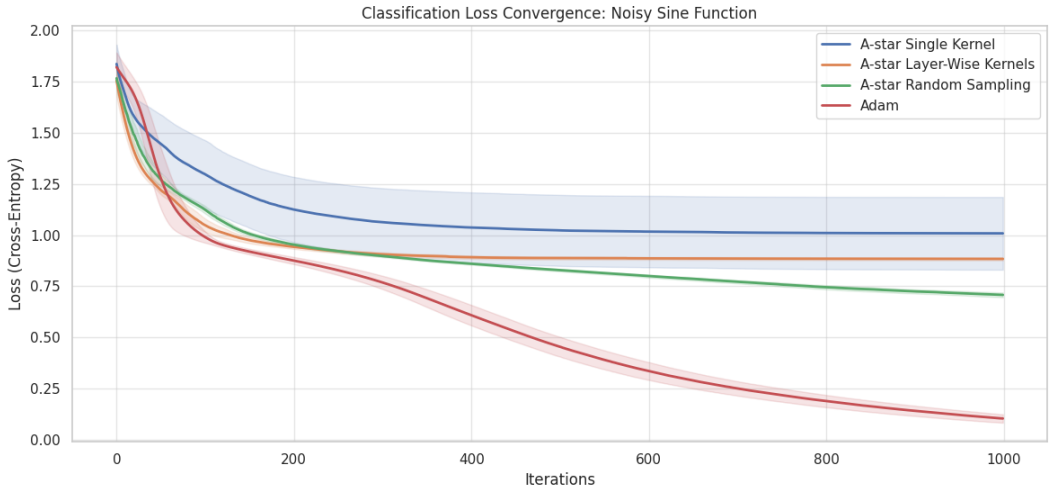

Figure 9.31: Medium Net convergence: A^* strategies vs. Adam on Wine Quality.

As model capacity increases, the performance gap between gradient-based and search-based methods widens significantly in favor of Adam. Adam maintains a rapid descent, reaching a low average final loss of 0.1041. Conversely, the search strategies encounter a precision plateau; A^* Random Sampling achieves the best result among the search variants at 0.7080. Notably, the *Single Kernel* approach exhibits instability at this scale, with a high standard deviation (0.1775) and a maximum loss reaching 1.3603, indicating that gradient-free exploration struggles to navigate the high-dimensional narrow valleys of this specific task.

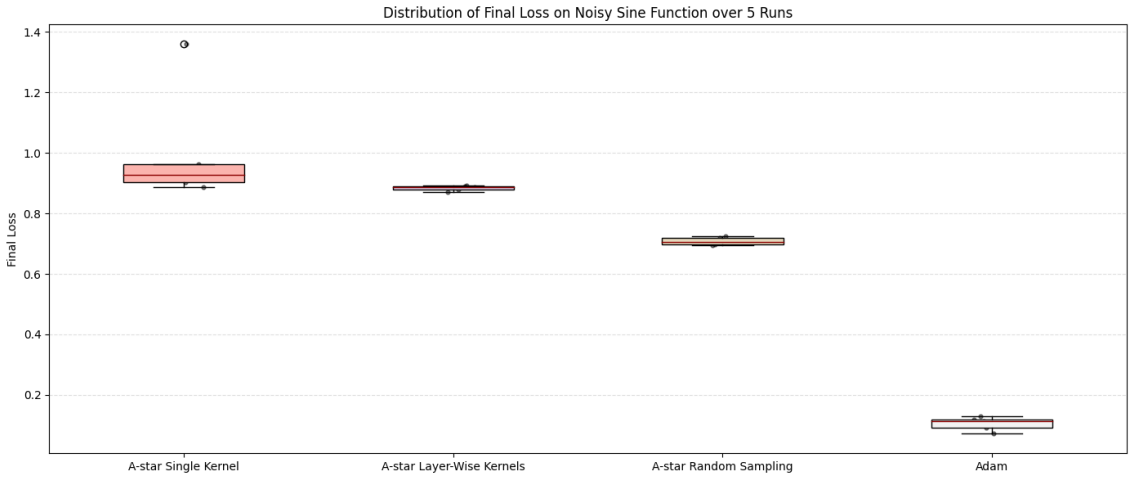

Figure 9.32: Distribution of Final Loss for Wine Quality - Medium Net.

Evaluation of Generalization Performance on the Test Set

Test set results for the Medium Net are visualized in Figure 9.33.

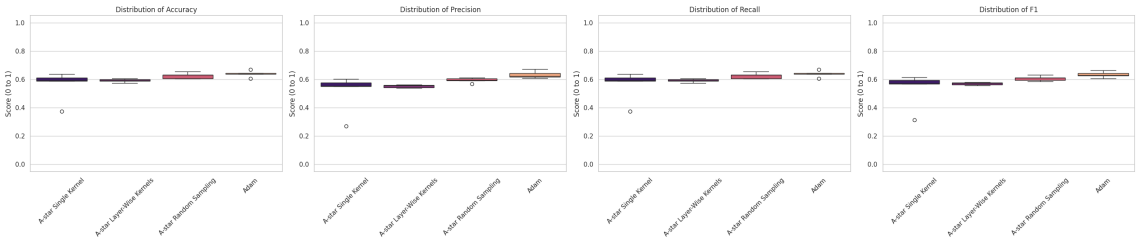

Figure 9.33: Classification metrics distribution for Medium Net.

Despite Adam's vastly superior training loss, the accuracy gap remains modest: Adam achieves 0.6400 accuracy compared to 0.6262 for A^* Random Sampling. This suggests a high degree of overfitting by Adam on the training set, whereas the search framework's inability to further minimize loss acts as a form of implicit regularization.

Summary Statistics

Table 9.11 summarizes the performance.

Metric	A^* **Single Kernel**	A^* **Layer-Wise**	A^* **Random**	**Adam**
Metrics computed on Training Set:				
Avg Final Loss	1.008597	0.883836	0.708040	**0.104191**
Median Final Loss	0.927983	0.888363	0.704441	**0.112365**
Std Final Loss	0.177593	**0.007341**	0.011510	0.020610
Avg Time (s)	1292.5712	1394.0129	663.9965	**14.2610**
Metrics computed on Test Set:				
Avg Accuracy	0.561250	0.592500	0.626250	**0.640000**
Avg Precision	0.510846	0.551917	0.593729	**0.632970**
Avg Recall	0.561250	0.592500	0.626250	**0.640000**
Avg F1	0.532007	0.569437	0.606684	**0.633230**

Table 9.11: Average statistical comparison on Wine Quality - Medium Net.

9.5.3 Big Net Results

Visual Analysis of Training and Stability

Convergence for the Big Net is shown in Figure 9.34.

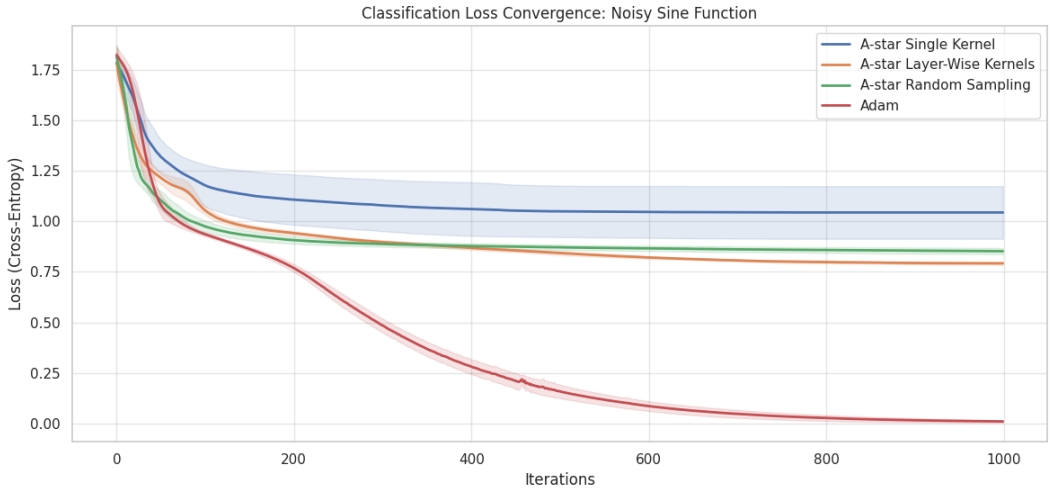

Figure 9.34: Big Net convergence: A^* strategies vs. Adam on Wine Quality.

In the Big Net regime, Adam reaches its peak performance with a smooth, high-precision descent to a final loss of 0.0094. Among the A^* strategies, the *Layer-Wise* approach emerges as the most effective search method ($Loss$: 0.7923), providing a significant improvement over the *Single Kernel* approach, which remains trapped at an average loss of 1.0441. This confirms that structured, layer-by-layer exploration is more resilient to the "curse of dimensionality" than a global kernel search in high-parameter classification models.

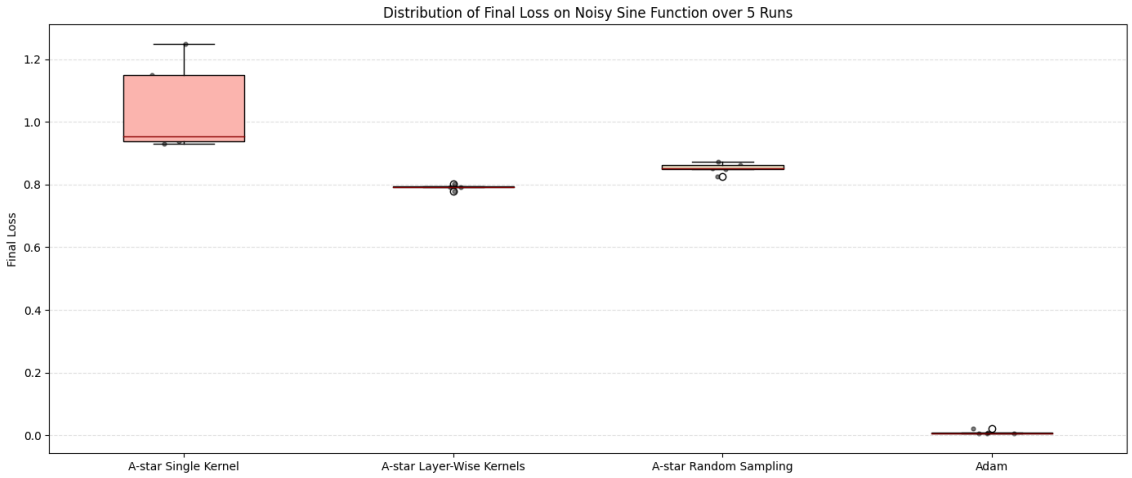

Figure 9.35: Distribution of Final Loss for Wine Quality - Big Net.

Stability analysis confirms that while Adam is smooth, the search methods, specifically *Layer-Wise* and *Random Sampling*, exhibit lower variance in their final training state, settling into consistent plateaus across multiple runs.

Evaluation of Generalization Performance on the Test Set

Generalization results are provided in Figure 9.36.

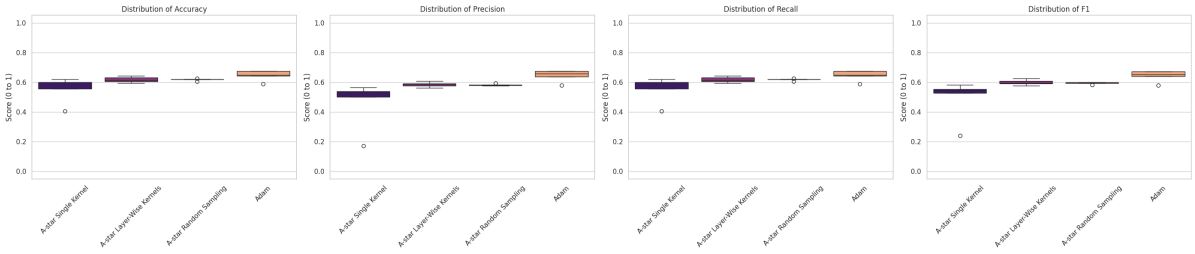

Figure 9.36: Classification metrics distribution for Big Net.

Adam maintains its lead on the test set with an accuracy of 0.6462. However, the *Layer-Wise* and *Random Sampling* methods remain competitive at 0.6175. The discrepancy between Adam's near-zero training loss and its moderate test accuracy further supports the observation that the Wine dataset contains inherent noise or class overlap that resists perfect generalization, regardless of the optimization depth.

Summary Statistics

Table 9.12 presents the numerical comparison.

Metric	A^* Single Kernel	A^* Layer-Wise	A^* Random	Adam
Metrics computed on Training Set:				
Avg Final Loss	1.044150	0.792313	0.852458	**0.009494**
Median Final Loss	0.954967	0.793027	0.851581	**0.007023**
Std Final Loss	0.130304	**0.007572**	0.015224	0.006161
Avg Time (s)	2501.4289	4960.8045	3386.8323	**15.4255**
Metrics computed on Test Set:				
Avg Accuracy	0.556250	0.617500	0.617500	**0.646250**
Avg Precision	0.462462	0.585682	0.582904	**0.645187**
Avg Recall	0.556250	0.617500	0.617500	**0.646250**
Avg F1	0.490386	0.599217	0.594114	**0.642548**

Table 9.12: Average statistical comparison on Wine Quality - Big Net.

9.5.4 Conclusions on Wine Quality Benchmark

The Wine Quality benchmark highlights the boundaries between gradient-based and discrete search optimization:

- **Gradient Dominance in Complex Classification:** Unlike the Iris benchmark where A^* reached absolute zero, the Wine dataset seems to be a more complex landscape where Adam's gradient-based updates are significantly more effective at escaping high-loss plateaus.
- **Search Stability:** While A^* methods struggled to match Adam's final depth, they demonstrated superior stability (lower variance) in the Medium and Big net regimes, particularly the Layer-Wise kernel strategy.
- **Implicit Regularization:** The "stagnation" of A^* methods at higher training losses did not lead to a catastrophic drop in test accuracy, suggesting that discrete search naturally avoids some of the sharpest training minima that might otherwise lead to overfitting in noisy datasets.

Chapter 10

Conclusions and Future Work

This project presented a comprehensive investigation into the application of the A^* search algorithm for the optimization of Multi-Layer Perceptrons. By transitioning from a discretized state-space exploration to structured neighbor generation, we established a framework that challenges the traditional reliance on gradient-based methods for specialized neural architectures.

10.1 Summary of Findings

The experimental setup, spanning from preliminary hyperparameter tuning to large-scale benchmarks, has yielded several insights:

- **Optimal Search Configuration:** Preliminary studies in Chapter 8 demonstrated that a vanilla A^* approach is insufficient for deep architectures. The introduction of **Beam Search** proved essential to mitigate the curse of dimensionality, providing a balance between computational feasibility and convergence depth.
- **High-Precision Convergence:** Across the Iris Flower, the A^* framework, specifically the kernel-based variants, consistently achieved training losses several orders of magnitude lower than the Adam baseline. In the Iris task, the framework reached absolute zero training loss, demonstrating a level of parameter refinement that gradient-based plateaus often prevent.
- **Generalization:** A significant finding of this research is the "crossover" effect observed as network complexity increases. While Adam remains superior in speed, the **Layer-Wise A^* strategy** began to outperform Adam in test-set

accuracy on larger architectures (e.g., Iris Big Net and Noisy Sine Big Net), suggesting that structured search identifies weight configurations with superior generalization margins.

- **Robustness and Stability:** Unlike Adam, which exhibited oscillations in non-linear regression tasks, the A^* framework provided a smooth and deterministic-like descent. Even in complex classification tasks like Wine Quality, where Adam achieved deeper training minima, the A^* strategies demonstrated significantly lower variance across independent runs, indicating a lower sensitivity to initial weight distributions.

10.2 Future Work

While the results validate the potential of search-based optimization, the following areas remain open for future exploration:

- **High-Performance Computing via Asynchronous and Parallel Search:** The most significant barrier to the widespread adoption of search-based optimization is the "wall-clock time" bottleneck inherent in evaluating massive candidate sets. Our current implementation is completely sequential, evaluating one neighbor at a time. Future works will focus on transitioning the framework to a high-performance computing paradigm through the following two vectors:
  - **Parallel Expansion:** By leveraging the parallel architecture of modern hardware, ranging from high-performance single GPUs to multi-GPU clusters, the framework enables the simultaneous evaluation of thousands of weight perturbations within the Open Set. By distributing the individual forward passes of each candidate network across discrete CUDA cores, the algorithm can compute the heuristic cost of an entire neighborhood within a single, high-concurrency operation. This approach effectively bypasses the sequential time-penalties of traditional search, drastically accelerating the exploration of the weight space and reducing the temporal footprint of each search expansion.
  - **Asynchronous Coordination:** Traditional parallel search often suffers from "synchronization lag," where fast processors sit idle waiting for the slowest core to finish. An asynchronous A^* implementation would decouple the search threads, allowing worker nodes to expand and push new candidates to a shared "Open Set" independently of one another.

Implementing these strategies would shift the A^* MLP framework from a CPU-bound process to a massively parallelized GPU-native operation, potentially reducing total training time by multiple orders of magnitude and making search-based training competitive with backpropagation speeds.

- **Hybrid Neuro-Search Optimization Paradigms:** A significant avenue for future research lies in the synthesis of gradient-based efficiency and search-based precision through a multi-stage hybrid optimizer. While the Adam optimizer demonstrates unparalleled velocity in traversing the initial, high-loss regions of the parameter space, it frequently encounters "precision plateaus" where gradients become vanishingly small or noisy, preventing the model from reaching the absolute global minimum.

  Future work will investigate a sequential multi-stage protocol:

  - **Gradient-Driven Warm-up:** The optimization process begins with Adam to perform the "heavy lifting," rapidly descending the macro-features of the loss landscape and navigating toward a promising local basin.
  - **Search-Based Refinement:** Upon detecting a plateau in the gradient norm or a stagnation in validation loss, the optimizer transitions to a A^* Search. In this stage, the A^* framework performs high-resolution adjustments within a discrete search space. This enables the fine-tuning of weight configurations in regions of the loss landscape where gradient signals become too weak or noisy for backpropagation to effectively navigate.

  This dual-stage approach seeks to eliminate the high computational cost of running A^* from initialization while simultaneously bypassing the convergence limits of standard backpropagation, potentially establishing a new benchmark for high-precision neural training.

- **Generalization to Spatial and Temporal Architectures:** While this research focused on the dense connectivity of Multi-Layer Perceptrons, the search-based optimization paradigm offers significant potential for more specialized neural structures. Future work should investigate the extension of the A* framework to more complex architectures:

  - **Convolutional Neural Networks:** Applying A^* search to convolutional filters would require a specialized neighbor generation strategy that preserves spatial locality. Instead of perturbing individual weights, the search could operate on entire 3D filter kernels, exploring discrete "kernel-states"

to identify spatial features without the risk of gradient vanishing/explosion in deep residual branches. This would clarify whether the observed stability of search-based methods translates to the more complex task of feature extraction.

– **Recurrent Architectures and Temporal Stability:** Recurrent Neural Networks are notoriously difficult to optimize due to the vanishing gradient problem inherent in Backpropagation Through Time (BPTT). Because A^* is gradient-free, it avoids the multiplicative chain of derivatives that typically causes signal degradation over long sequences. Future research could explore recursive weight matrices using a "Temporal-Aware Heuristic." By treating temporal steps as a discrete search depth rather than a differentiable chain, the algorithm could maintain numerical stability over extended sequences.

10.3 Final Remarks

In conclusion, this research demonstrates that the A^* search algorithm is far more than a theoretical curiosity in the context of neural network training; it represents a viable, high-precision optimization paradigm capable of navigating weight spaces that traditional gradient-based methods find challenging. By reframing neural training as a state-space exploration problem rather than a purely differentiable calculus task, we have unlocked a methodology that prioritizes numerical precision and descent stability.

The empirical evidence demonstrates that by utilizing Beam Search to manage memory and Layer-Wise Kernels to mitigate the curse of dimensionality, the A^* framework achieves performance levels that are highly competitive with the Adam optimizer. While Adam remains the industry standard for rapid training, the search-based approach excels by providing a remarkably stable convergence trajectory. A core advantage of the A^* framework is its significantly lower variance in results across independent runs, making it a robust alternative for applications where consistency and high-precision training depth are prioritized over execution speed. Ultimately, this study advances the understanding of non-gradient optimization within the machine learning paradigm. It establishes a foundational architecture for future hybrid systems capable of merging the intuitive spatial exploration of search with the computational velocity of gradient-based methods. While search-based optimization currently faces temporal constraints, these are primarily software-level bottlenecks rather than algorithmic ones. As the framework evolves toward parallel implemen-

tations, the current barriers to execution speed will diminish, paving the way for a new generation of high-precision, robust artificial intelligence algorithms.